源荷与电价不确定性下微网电力调控策略的随机优化与风险控制

摘要

随着分布式光伏的快速普及，微网快速发展，其目的在于就地消纳与降低用能成本。但光伏与小区负载的早晚双峰严重错配、光伏预报存在误差、外网电价实时波动且极端价格时有出现，**微网购电决策**面临多重不确定性。为此，本文基于该微网的分时电价、负载与光伏预测、全年逐日源荷数据、四个时点光伏预报与全年实时电价，建立数学模型，为购电申报与储能调度提供依据。

对于问题一，本文对负载、光伏与电价数据做可视化与箱线图检验后，构建以全天购电费用最小为目标的**线性规划模型**，以 **10 分钟**离散全天，设置母线功率平衡、供电可靠性、荷电状态递推、储电量区间、充放电功率上限、日循环边界与弃光上限共 **7 个**约束，并证明最优解不同时充放电。最优解给出日购电量 **57 526.24 kWh**、日购电费 **33 801.5 元**；影子价格验证了储能的必要性，可节省 **29.66%** 的电费并消纳 **6 247.96 kWh** 富裕光电量。

对于问题二，本文以一年实际负载的相似日加权点预测构造残差池，有放回抽样生成**等概率情景**，并设**预热期**传递初值；随后构建**两阶段随机规划模型**，目标为购电与紧急购电费用期望之和最小。沿用 **HiGHS** 逐日**滚动求解**，总费用 **13 234 106.71 元**。最后以报童模型校验，表明情景集宽度与模型结构无系统偏差。

对于问题三，本文先统一四个时点光伏预报语义并完成**降尺度映射**，误差画像显示预报近似无偏、单点误差可观且午后低估。随后用非负辅助变量把 **0.5 倍**与 **1.5 倍**双向计价的偏差罚则等价写成线性约束，以非预期性约束限定各阶段信息集，构建**多阶段滚动决策模型**，在四个时点求解剩余时段子问题。接着设计 **A、B、C** 三重控制变量实验，分离决策机会价值与信息更新价值。说明仅增加改签机会多花 **53.1 万元**，信息更新节省 **62.6 万元**，净增益 **9.5 万元**。分发布时刻的边际价值图显示，**12:00** 预报价值最高、**6:00** 次之、**18:00** 几乎为零，说明发布频率不等于信息量。

对于问题四，本文先诊断**实时电价**形态；随后为电价、负荷与光伏构造残差池并按同一历史日对齐抽样，形成**三源联合情景**，另设独立抽样对照，在同一随机种子与情景数下运行四种方案；在问题二、三模型上叠加**风险控制**，以**条件风险价值 CVaR** 为惩罚项，取置信水平 **95%** 下的情景平均费用，构建 **CVaR 购电策略优化模型**，求得最佳策略：风险厌恶系数取 **0.2** 时尾部风险下降 **37.9%**、期望费用上升 **14.1%**，费用与风险曲线在 **0.15** 后出现明显拐点，为保守程度取舍提供定量依据。后悔值分析还显示，按均值电价制定计划、按实时电价结算的费用反而**低于**按实时电价优化，说明均值电价计划带有**对冲价值**。

关键词：微网电力调控；两阶段随机规划；滚动时域优化；信息价值分解；电价不确定性建模；条件风险价值

一、引言

1.1 问题背景

光伏等分布式电源近年来在居民小区和工业园区中越来越常见[1,2]。这类电源就地发电、就地消纳，减少了远距离输电的损耗，但光伏产生的电量完全由天气决定，中午前后往往远高于本地负载，傍晚和夜间又降为零，与小区自身的用电节奏并不匹配。为缓解这一矛盾，工程上通常把光伏、储能设备与本地负载组成一个微网，再通过公共连接点与外部电网相连，当光伏有富余时给储能充电，光伏不足时由储能放电或从外网购电，这样既能保证正常供电，也能省下一部分电费。

同时外网电价还带有明显的分时特征，一天之内高低电价时段交替出现，不同季节的价格水平也有差别，这给微网运行留出了优化空间。在电价低或光伏出力大的时段多购电、多充电，在电价高的时段少购电、多放电，把购电费用压下来。另外，实际运行中还有两类额外成本要考虑，当供电不足时向外网紧急购电，价格远高于正常电价；但是购电计划临时变动，也会产生违约或加价费用[3]。所以微网购电策略的核心，是在满足负载需求和储能运行约束的前提下，对每个时段的购电量与充放电量做出合理安排，使全天总费用尽可能低。本题正是围绕这一问题展开。

1.2 问题重述

结合题目所给的数据与要求，需要解决的问题可以归纳为以下四个。

问题 1：电价与小区负载每天相同，并已知全天的光伏发电预测功率。要求建立每天 0:00 制定当天计划购电策略的数学模型，且微网提供的电能不低于小区负载、储能设备在 0:00 与 24:00 的储电量相同。再根据附件 1 和附录 1 的数据给出具体方案：指定时段的购电量、全天购电量与购电费（表 1 格式），以及指定时段的充放电量和 0:00、24:00 的储电量（表 2 格式）。

问题 2：电价不变，负载和光伏发电功率则逐日变化；如果微网提供的电能低于负载，不足部分需按交易时刻电价的 5 倍紧急购电，其余购电费用仍按计划购电量计算。要求在每天 0:00 制定当天的计划购电策略，依据附件 2 的数据完成 2025 年 2 月 1 日至 12 月 31 日的计算，并在论文中给出 2025.3.20、2025.6.21、2025.9.23、2025.12.21 四天的购电量、充放电量及紧急购电结果。

问题 3：光伏和负载预报，每天 0:00、6:00、12:00、18:00 可获得未来 24 小时整点的光伏发电功率预报，既可以在 0:00 制定当天计划，也可以在后续时刻根据新预报调整。费用规则相应更复杂：调整后的购电量低于计划时，差额部分按交易时刻电价的 50% 计违约电价；高于计划时，超出部分按 1.5 倍电价计费；总费用由计划购电费用、紧急购电费用和调整购电量的相关费用组成。要求建立相应的数学模型，利用附件 3 的预报数据制定计划与调整策略，并分析是否需要引入其他时刻的预报。

问题 4：考虑外网电价实时波动的情形，针对波动电价建立微网当天购电策略的数学模型，并基于附件 4 的电价数据重新计算问题二和问题三的结果。

二、 总体分析

本题围绕微网与外部电网之间的电力调控策略展开，要求在满足小区负荷供电可靠性的前提下，通过优化计划购电量与储能充放电策略，实现购电总费用的最小化。题目由浅入深地设置了四个递进式问题：问题一在完全信息条件下求解单日日前调度方案，构建了全题的基准模型框架；问题二引入紧急购电机制与信息不完全约束，将问题拓展为不对称惩罚下的最优承诺量决策，其数学结构与报童模型同构；问题三进一步放宽调度信息条件，要求基于实时预测滚动更新调度策略；问题四则在实时电价实际情况下，考察模型在多时间尺度、多情景下的泛化能力与工程实用性。

本题的核心是挖掘储能设备的时间转移价值。设备以充放电效率为代价，把电能从低谷时段搬到高峰时段，从而利用峰谷价差获取套利收益；而光伏出力与负荷需求在时间上的错配，又决定了储能调度必须与购电决策、弃光决策协同确定。供电可靠性在本模型中是刚性约束，不是被权衡的对象；真正的取舍落在计划量上，如果承诺得少，缺电时要按惩罚性高价补购，承诺得多，未用的部分仍要照付。四个问题实际上是在不同的信息条件下反复面对这一取舍。

整体来看，四个问题共用同一套线性规划基础，随着问题深入，模型沿时间维度、信息维度与价格不确定性三条轴逐层扩展。

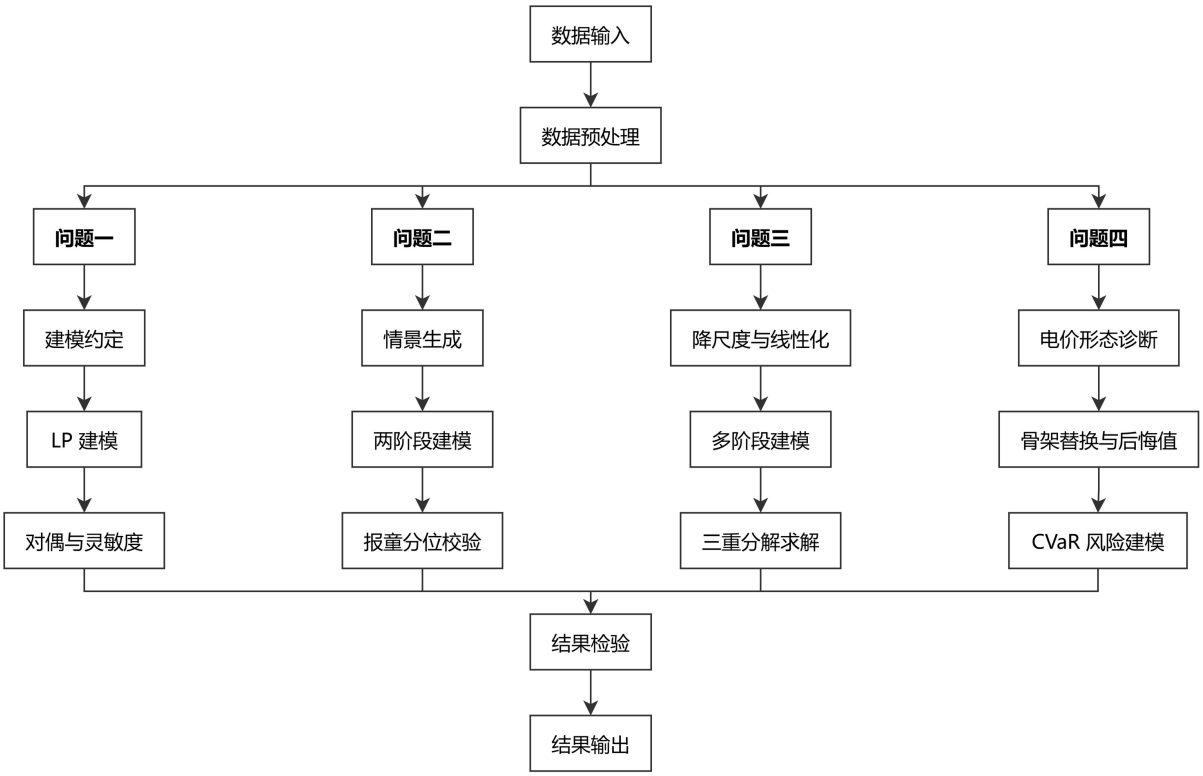


图 **1**　总体流程图

三、 模型假设

简化问题并突出主要矛盾[8]，本文做出以下假设：

• 假设 1：微网只向外部电网购电，不向外部电网售电。

• 假设 2：储能往返效率为 90%，即充、放电效率各取 $\eta_c = \eta_d = \sqrt{0.9}$；同时不考虑电池循环寿命损耗、折旧成本与容量衰减。

• 假设 3：弃光量为富余光伏发电量。

• 假设 4：假设全天按 10 分钟等分为 144 个时段，各时段内负载、光伏与电价保持恒定。

四、 符号说明

为便于查阅，下表汇总模型建立与求解过程中使用的全局符号。按集合与索引、参数、决策变量、状态与中间变量、目标变量五类列出。

关于日期下标：表中决策变量在问题 2～4 中均增加日期下标 d，即 $g_t \to g_{d,t}$、$c_t \to c_{d,t}$、$d_t \to d_{d,t}$、$s_t \to s_{d,t}$、$e_t \to e_{d,t}$、$u_t \to u_{d,t}$、$v_t \to v_{d,t}$、$g_t^{\text{plan}} \to g_{d,t}^{\text{plan}}$、$g_t^{\text{adj}} \to g_{d,t}^{\text{adj}}$；问题 1 为单日模型，不带日期下标。

表 **1**　集合与索引

符号	含义	单位
$t \in \mathcal{T}$	时段索引，$\mathcal{T} = \{1, 2, \ldots, 144\}$	—
$d \in \mathcal{D}$	日期索引，共 365 天	—
$d^{\text{rep}} \in \mathcal{D}^{\text{rep}}$	报告期日期索引，共 334 天	—
$k \in \mathcal{K}$	预报发布时刻索引，$\mathcal{K} = \{0, 1, 2, 3\}$ 对应 0:00、6:00、12:00、18:00	—
$s \in \mathcal{S}$	情景索引，$\mathcal{S} = \{1, 2, \ldots, S\}$	—
τ	决策时点索引，$\tau \in \{0, 36, 72, 108\}$	—

表 **2**　参数

符号	含义	单位
Δt	时段长度，取值 $1/6$	h
π_t	时刻 t 的电价	元/kWh
L_t	时刻 t 的小区负载功率	kW
pv_t	时刻 t 的光伏发电功率	kW
$\widehat{pv}_t$	降尺度到 144 段后的光伏预报功率	kW
ξ_t	光伏预报误差，$\xi_t = \widehat{pv}_t - pv_t^{\text{act}}$	kW
η_c	充电效率，取值 $\sqrt{0.9} \approx 0.9487$	—
η_d	放电效率，取值 $\sqrt{0.9} \approx 0.9487$	—

符号	含义	单位
$P_{\max}$	储能最大充放电功率，取值 5 000	kW
$E^{\max}$	单时段最大充放电电量，$E^{\max} = P_{\max}\Delta t = 833.33$	kWh
S_{cap}	储能额定容量，取值 12 000	kWh
$S_{\min}$	储电量运行下界，取值 1 200	kWh
$S_{\max}$	储电量运行上界，取值 10 800	kWh
E_0	初始储电量，取值 6 000	kWh
β_{emg}	紧急购电电价倍数，取值 5.0	—
β_{def}	违约（少买）电价倍数，取值 0.5	—
β_{exc}	超额（多买）电价倍数，取值 1.5	—
c_{cyc}	电池循环成本（仅用于灵敏度扫描）	元/kWh

表 **3**　决策变量

符号	含义	单位
g_t	计划购电量	kWh
c_t	电池充电量	kWh
d_t	电池放电量	kWh
s_t	弃光电量	kWh
e_t	紧急购电量	kWh
g_t^{plan}	基准计划购电量	kWh
g_t^{adj}	调整后购电量	kWh
u_t	违约电量，即计划量高于调整量的部分	kWh
v_t	超额电量，即调整量高于计划量的部分	kWh

表 **4**　状态变量与中间变量

符号	含义	单位
E_t	第 t 时段末的储电量（SOC），由 E_0 与全部充放电量唯一确定	kWh
$E_{d,t}$	第 d 天第 t 时段末的储电量	kWh

表 **5**　目标变量

符号	含义	单位
C^{plan}	计划购电费用	元
C^{emg}	紧急购电费用	元
C^{def}	违约费用	元
C^{exc}	超额费用	元

符号	含义	单位
C^{total}	全天或全周期总购电费用	元
G^{day}	全天总购电量	kWh
VOI	预报信息价值	元
R	后悔值	元
VSS	随机解价值	元

五、 模型的建立与求解

5.1 问题一的模型的建立和求解

5.1.1 具体分析

针对问题一题目提供完整一天的电价、负载和光伏预测，且均在附件 1 里，每天电价和小区负载相同，所以这是确定性条件下的单日调度问题，要求解每段的购电量与充放电量。本文将采用线性规划模型，建立单日确定性调度模型，以获得全局最优解，并通过求解器返回的对偶变量解释最优策略。

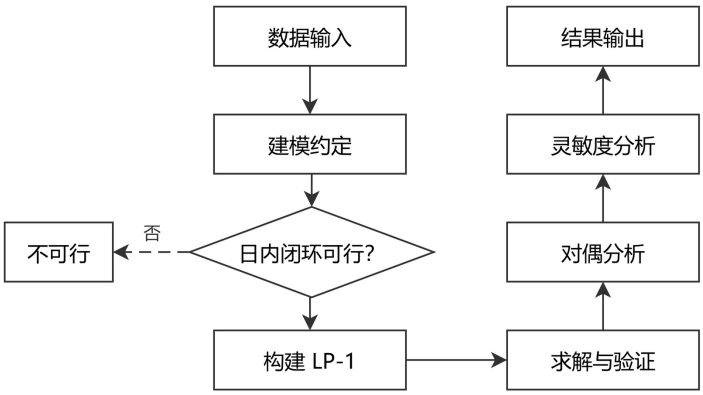


图 **2**　问题一分析流程图

5.1.2 模型准备

1. 数据预处理

根据附件 1 的数据生成可视化图片来观察数据的整体情况，如图 3 附件 1 典型日曲线图所示，该图为典型日 24h 分时电价、小区负荷与光伏预测出力时序变化，电价呈阶梯峰谷特性，负荷具有早晚双峰，光伏仅日间发电。

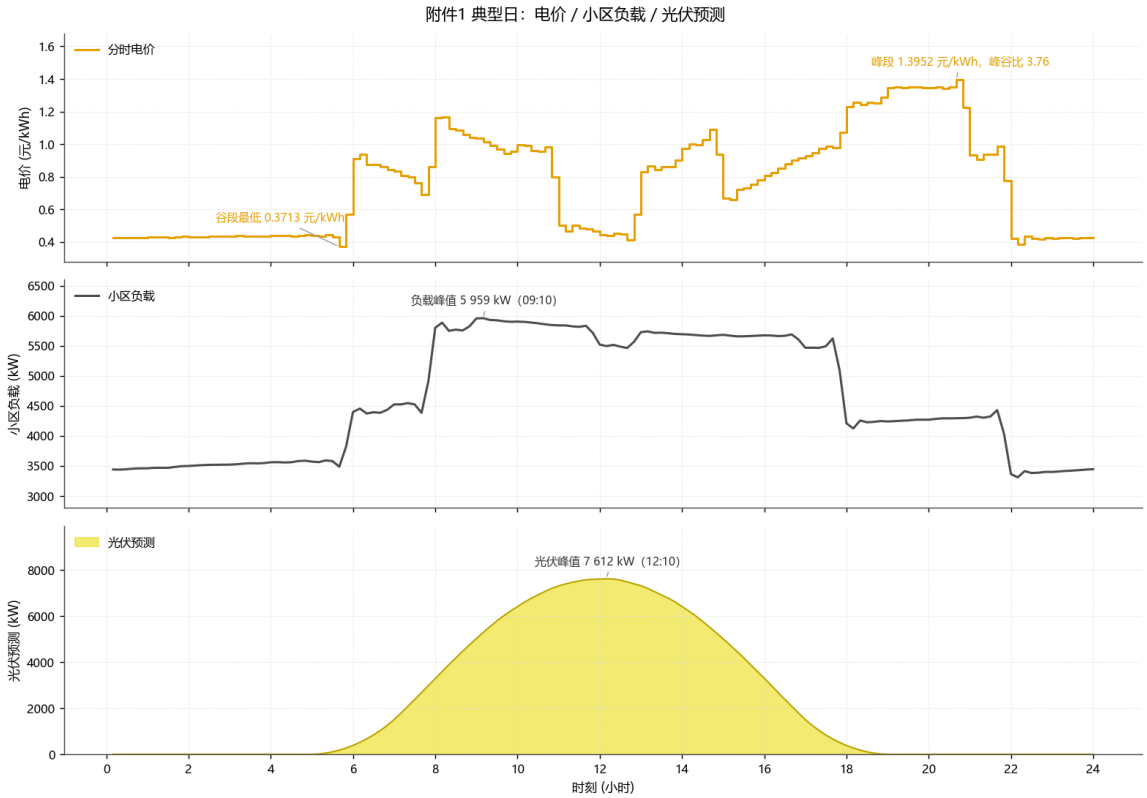


图 **3**　附件 **1** 典型日曲线图

接下来是对数据做一个箱线图测试，如图 4 为附件 1 三变量箱线图，其中包含了典型日电价、小区负载与光伏预测功率箱线图，直观反映各变量的数据分布、中位数与波动区间，电价体现明显峰谷价差，负载集中在中位附近且上下对称，光伏高度右偏且四分位以下全为零，但三图都没有离群点，没有异常值和缺失值，可认为附件 1 的数据基本上不需要做更多的数据预处理。

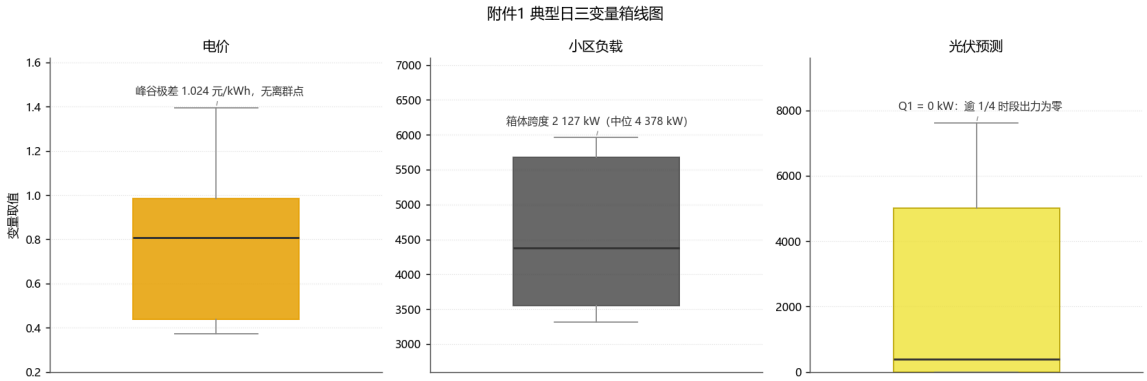


图 **4**　附件 **1** 三变量箱线图

2. 算法选择

针对问题一，附件 1 已给出了电价、负载与光伏等完备信息，属完全信息的确定性单日调度问题，若采用启发式智能算法将引入随机性和近似解，无法保证全局最优。本文考虑到模型若引入 0-1 整数变量将退化为混合整数规划，徒增计算负担，故采用纯线性规划求解，以

10 分钟为时间步长离散得到 144 个时段，构建确定性线性规划模型[4]。同时，本文经过数学推导证明最优解不会出现同一时段同时充放电的情况，故无需引入 0-1 整数变量，本文将沿用 HiGHS 稀疏求解器以单纯形法或内点法完成全局最优求解，并提取对偶变量开展对偶分析。从而推导储能充放电的盈亏阈值规则，辅以约束可行性校验、KKT 互补松弛检验与能量守恒闭环校验检验模型。

5.1.3 模型建立

步骤 1：定义决策变量

本文以全天 144 个时段内微网向外部电网的计划购电量、储能设备的充电量、放电量与弃光电量作为决策变量，储能设备的储电量作为状态变量由递推关系确定，设：

$$g_t \ \geq \ 0 \ (1)$$

其中，g_t 表示第 t 个时段微网向外部电网的计划购电量（kWh）；t = 1, 2, …, 144 表示时段序号，其数字对应于第 t 个时段 ((t−1)Δt, tΔt]，时段长度 Δt = 1/6 h，即 10 分钟；附件中的时间标签视为时段终点，故第 1 段为 (0:00, 0:10]、第 144 段为 (23:50, 24:00]，144 段时间段覆盖全天。

$$c_t \ \geq \ 0 \ (2)$$

其中，c_t 表示第 t 个时段储能设备的充电量（kWh），按母线侧计量，即该时段从母线取用、用于给储能充电的电量；其取值上限为充放电功率上限约束。

$$d_t \ \geq \ 0 \ (3)$$

其中，d_t 表示第 t 个时段储能设备的放电量（kWh），同样按母线侧计量，即该时段储能向母线输出的电量；其取值上限同样为充放电功率上限约束。

$$w_t \ \geq \ 0 \ (4)$$

其中，w_t 表示第 t 个时段的弃光电量（kWh），即该时段光伏发电量中未被负载消纳、也未能充入储能的多余电量；其取值上限为弃光上限约束。

$$e_0 \ = \ 6000 \ (5)$$

其中，e_t 表示第 t 个时段末储能设备的储电量（kWh）。e_t 为状态变量而非独立决策变量，给定初始储电量与全部充放电量后，其取值由储电量递推方程唯一确定。当前问题一是由题目附录 1 给定 2025 年 1 月 1 日 0:00 的储电量为 6000 kWh，故 e_0 = 6000。

步骤 2：确立目标函数

微网优化调度以当天购电费用最小为目标。首先考虑第 t 个时段的购电费用，有：

$$C_t \ = \ p_t \, g_t \ (6)$$

其中，C_t 表示第 t 个时段的购电费用（元）；p_t 表示第 t 个时段的电价（元/kWh）。

模型的目标函数为：

$$min\ Z\ =\ \sum_{t\,=\,1}^{144} p_t\ g_t\ (7)$$

其中，Z 表示微网当天的购电总费用（元）。

步骤 3：设立约束条件

约束 1：母线功率平衡。

$$g_t\ +\ P_t^{pv}\ \Delta t\ +\ d_t\ =\ L_t\ \Delta t\ +\ c_t\ +\ w_t\ ,\ \ t\ =\ 1\ ,\ \ldots\ ,144\ (8)$$

其中 L_t 表示第 t 个时段的小区负载功率（kW），$L_t\Delta t$ 为该时段小区负载用电量（kWh）；P_t^{pv} 表示第 t 个时段的光伏发电功率（kW），$P_t^{pv}\Delta t$ 为该时段光伏发电量（kWh）。由于弃光变量 $w_t \geq 0$ 的存在，等式形式可满足题目中微网提供的电能不可低于小区负载的要求，故式 (8) 取等式而非不等式。如果出现供给大于需求的情况，它们之间的差额即为 w_t 。

约束 2：供电可靠性。

$$g_t\ +\ P_t^{pv}\ \Delta t\ +\ d_t\ \geq\ L_t\ \Delta t\ +\ c_t\ (9)$$

其中对式 (8) 移项并利用 $w_t \geq 0$，即得题目所给供电可靠性要求的显式形式。

约束 3：储能荷电状态递推。

$$e_t\ =\ e_{t-1}\ +\ \eta_c\ c_t\ -\ \frac{d_t}{\eta_d}\ (10)$$

其中充电时储能内部实际增加 $\eta_c \cdot c_t$，本文假设返效率 $\eta_c \cdot \eta_d = 90\%$，即放电时储能内部实际减少 d_t/η_d。$\eta_c = \eta_d = \sqrt{0.9}$ 约 0.9487 为对称的单边充放电效率。

约束 4：储电量运行区间。

$$1200\ \leq\ e_t\ \leq\ 10800\ (11)$$

其中题目指明了储能设备的电量必须保持在 1200–10800 kWh 之间，即为储电量的上下限。

约束 5：充放电功率上限。

$$0\ \leq\ c_t\ \leq\ 833.33\ ,\ \ 0\ \leq\ d_t\ \leq\ 833.33\ (12)$$

其中题目给出最大充放电功率 5000 kW，须折算为单时段电量上限，5000 × 1/6 = 833.33 kWh。若误将 5000 kWh 直接作为上限，会严重高估电池的调节能力。

约束 6：日循环边界条件。

$$e_{144}\ =\ e_0\ =\ 6000\ (13)$$

其中问题一已指明需保证单日的 0:00 和 24:00 的储电量相同。

约束 7：弃光上限与非负约束。

$$0\ \leq\ w_t\ \leq\ P_t^{pv}\ \Delta t\ (14)$$

其中弃光电量 w_t 的上限取为该时段的光伏发电量 $P_t^{pv}\Delta t$（kW • h），即弃光只能来自光伏发电，不能来自外网购电。

汇总以上所有部分，针对问题一的单日确定性调度模型的完整模型如式（15）

$$
\begin{aligned}
&\min Z = \sum_{t=1}^{144} p_t\, g_t \\
\text{st.}\quad &\begin{cases}
g_t + P_t^{pv}\Delta t + d_t = L_t\Delta t + c_t + w_t \\
e_t = e_{t-1} + \eta_c\, c_t - \dfrac{d_t}{\eta_d} \\
1200 \le e_t \le 10800 \\
0 \le c_t \le 833.33, \qquad 0 \le d_t \le 833.33 \\
0 \le w_t \le P_t^{pv}\Delta t, \qquad g_t \ge 0 \\
e_0 = 6000, \qquad e_{144} = e_0
\end{cases}
\end{aligned}
\tag{15}
$$

5.1.4 模型求解

步骤 1：数据准备与参数设定

读取附件 1 的 144 段电价、负载与光伏预测数据，折算为电量，每段电量 = 功率 $\times 1/6$；设定储能参数，充放电效率 $\eta_c = \eta_d = \sqrt{0.9}$、储电量上下界 1200 与 10800 kWh、单段充放电量上限 833.33 kWh、初始储电量 6000 kWh。

步骤 2：变量排列与目标函数构造

按 $[g; c; d; e; w]$ 的分块顺序排列决策变量，目标函数系数向量取 $(p_t,\ 10^{-6},\ 10^{-6},\ 0,\ 0)$，即前 144 个系数为电价，中间两组 10^{-6} 为吞吐量正则项，令其量级远小于电价，不影响最优费用，用于在等价最优解中选出曲线最平滑的。

步骤 3：约束矩阵构建

用 COO 稀疏格式直接构造 288 行等式约束矩阵，前 144 行为母线功率平衡，后 144 行为储能荷电状态递推；变量上下界按步骤 1 的参数设定，末端储电量 e_{144} 的上下界收紧为 $[6000, 6000]$，以实现日终闭环。

步骤 4：调用求解器

调用 linprog 中的 HiGHS 求解器，返回状态码 0 表示求得全局最优，模型规模为 720 个变量、288 条等式约束。

步骤 5：结果校验与输出

将解代回全部约束计算残差；用对偶单纯形与内点法两条独立算法路径交叉验证目标值；读取对偶变量用于结果分析。按题目要求聚合出指定时段购电量与 4 小时区间充放电量，写入 result1.xlsx。

5.1.5 结果分析

表 **6**　求解结果

指标	数值
全天购电量	57 526.24 kWh

指标	数值
全天购电费	33 801.50 元
弃光电量	0.00 kWh
储电量轨迹	最低 1200.0 kWh，最高 10800.0 kWh
0:00 / 24:00 储电量	6000.00 / 6000.00 kWh
求解状态	status = 0

表 **7**　指定时段的计划购电量

时间段	购电量 (kWh)	时间段	购电量 (kWh)
10:00–10:10	0.0000	16:00–16:10	445.4317
12:00–12:10	480.4124	18:00–18:10	64.5344
14:00–14:10	0.0000	20:00–20:10	0.0000
全天合计	57 526.2435	全天购电费	33 801.4955 元

表 **8**　储能充放电量与储电量

时间段	充电量 (kWh)	放电量 (kWh)
0:00–4:00	4 226.3109	0.0000
4:00–8:00	833.3333	6 956.9814
8:00–12:00	4 448.6339	1 702.9970
12:00–16:00	5 274.7881	91.1014
16:00–20:00	0.0000	6 247.4915
20:00–24:00	5 059.6443	2 859.8681
0:00 储电量	6000.0000	
24:00 储电量	6000.0000	

如图 5 最优调度结果图。上图可见储能装置在电价低谷时段充电、高峰时段放电；中间图对比光伏出力、小区负荷与购电功率，光伏充足时可减少购电；下图为储能装置的 SOC 曲线，SOC 在上下限区间波动，始末电量保持一致。整体可看出该方案做到低电价储电、高电价释能，有效降低微网购电总成本。

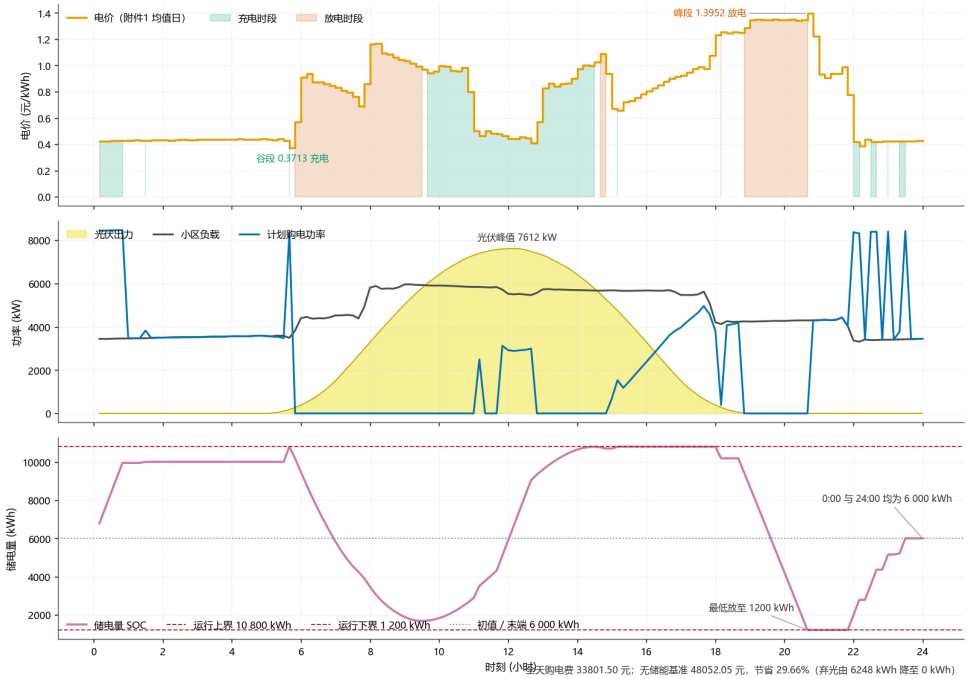


图 **5**　最优调度结果图

如图 6 边际电价分析图所示，左图是对偶变量与实际电价，右图是把对偶变量和实际电价散点对比图，整合起来分析可得对偶变量在多数时段等于实际电价，说明这些时段的电由电网按边际价格供给；而在电池放电时段低于电价，说明那一段的边际供电成本由电池放电的机会成本决定，而不是电网购电价

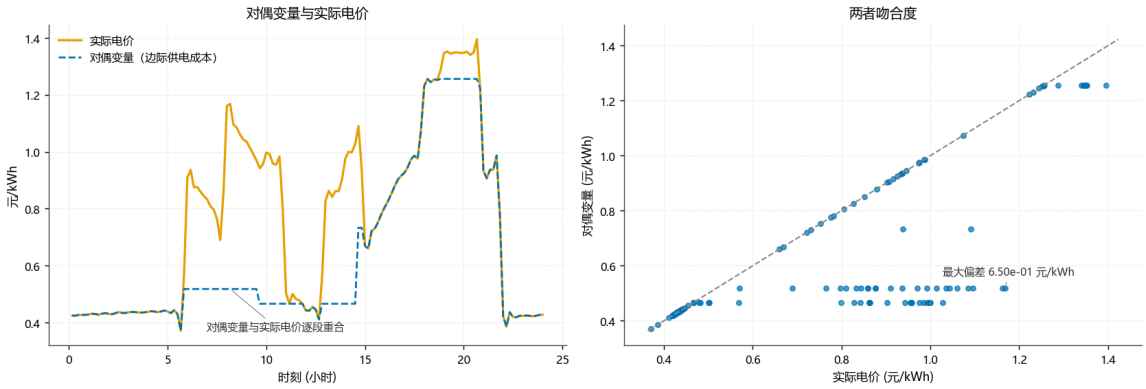


图 **6**　边际电价分析

如图 7 储能经济价值量化所示，左图为储能带来的费用下降，右图为电量结构对比图，可分析得出有储能时全天购电费显著低于无储能情形，且弃光量由 6 248kWh 降至 0。说明储能装置不仅能有效减少购电，而且能把丢弃的光伏电量减少为 0。

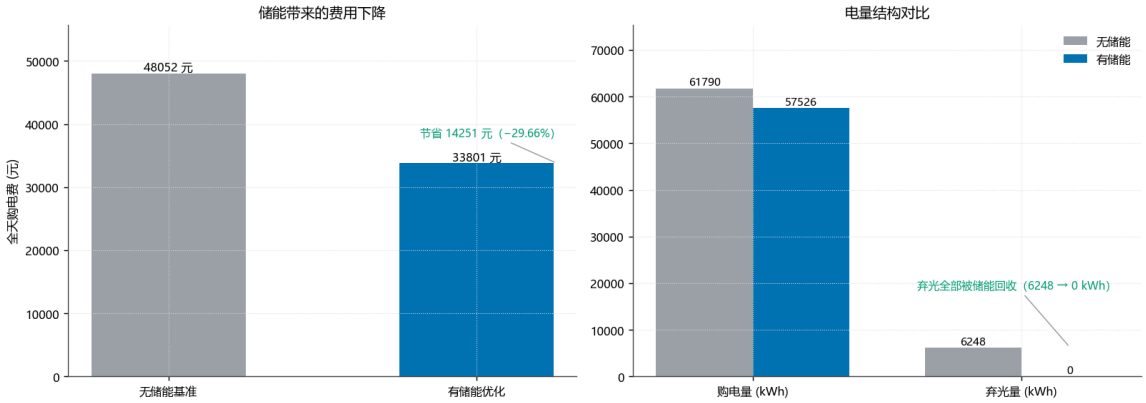


图 **7**　储能经济价值量化

5.2 问题二的模型的建立和求解

5.2.1 具体分析

针对问题二本问将时间尺度从一天扩展到一年，电价依据附件 1 的电价数据，每天的电价相同，但负载与光伏改为逐日、逐时段变化。本文作出对应的修改，即决策变量需改为 (d,t) 的二维索引，且储能设备的荷电状态需跨日连续传递，不沿用问题一的储能设备的跨日储电量相同的假设。

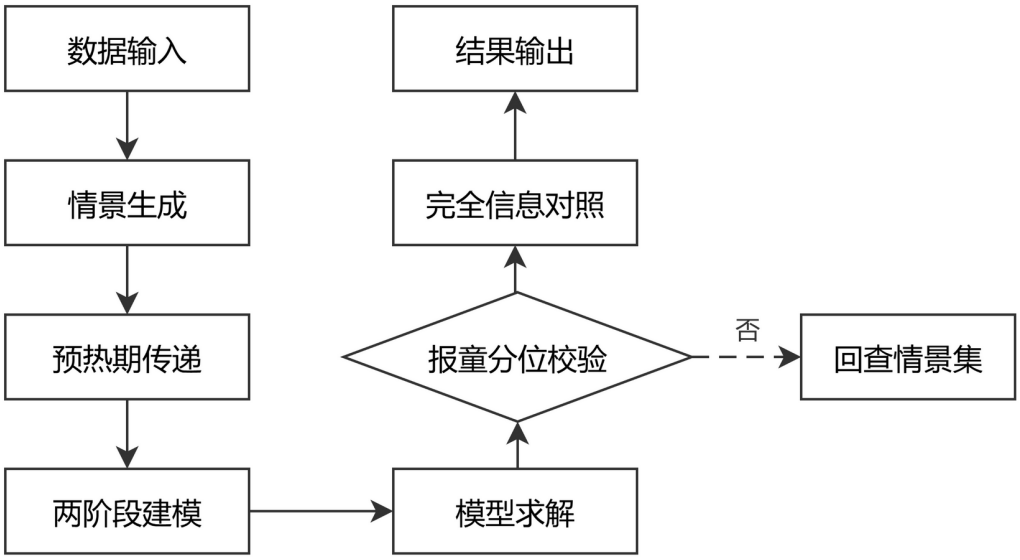


图 **8**　问题二分析流程图

5.2.2 模型准备

1. 数据预处理

针对问题二，考虑 0:00 制定购电计划时，当日光伏、负荷尚未可知，若采用附件 2 的完全信息建模会使最优解下的紧急购电恒为零，导致惩罚机制失效。为解决这个问题，本文构建两阶段随机规划模型[5]，将单日调度模型拓展至 365 天时序仿真，设置 1 月共 31 天作为预

热期，得到 2 月 1 日真实初始储能状态，消除初值人为假设带来的偏差。

2. 算法选择

针对问题二在 0:00 制定计划时当日的光伏和负载数据尚未可知，若按完全信息建模将使紧急购电恒为零、5 倍惩罚机制失效；本文考虑到鲁棒优化过于保守，难以量化期望费用。故采用两阶段随机线性规划求解，基于历史预报误差经验分布生成等概率情景，第一阶段在每日 0 点确定计划购电量，第二阶段在依据不同情景决策储能充放电、弃光与紧急购电，沿用 HiGHS 求解器逐日求解，并借助报童模型临界分位开展独立校验，验证模型结果合理性。

5.2.3 模型建立

步骤 1：定义决策变量

本文的决策变量分两个阶段。第一阶段变量为全天逐时段的计划购电量，它与情景无关，须在当天 0:00 前确定，设：

$$g_t^{plan} \geq 0, \quad (16)$$

其中，计划购电量为第一阶段决策，对全部情景取同一值；

第二阶段变量为每个源荷情景下的充电量、放电量、弃光电量与紧急购电量，由情景实现后决定，设：

$$c_{\omega,t} \geq 0, \quad d_{\omega,t} \geq 0, \quad s_{\omega,t} \geq 0, \quad e_{\omega,t} \geq 0 (17)$$

其中分别表示情景 ω 下的充电量、放电量、弃光电量与紧急购电量（kWh）。

步骤 2：确立目标函数

在每日 0:00 决策计划购电量[6]，最小化计划购电费用与紧急购电费用的期望总和：

$$min\ C^{\text{total}} = \sum_d \sum_t \pi_t\, g_{d,t} \ + \ \beta_{\text{emg}} \sum_d \sum_t \pi_t\, e_{d,t} \qquad (18)$$

其中第一项按计划购电量计费，第二项是情景 s 下缺口按 5 倍电价补齐的费用即 $\beta_{\text{emg}} = 5$。

步骤 3：设立约束条件

约束 1：各情景下的母线平衡

$$g_{d,t} + e_{d,t} + pv_{d,t}\Delta t + d_{d,t} = L_{d,t}\Delta t + c_{d,t} + s_{d,t} \qquad (19)$$

其中等式左侧为供给侧，右侧为需求侧，在原本的母线平衡公式中，在供给侧增加了紧急购电量 $e_{d,t}$，由于 $e_{d,t} \geq 0$ 且其价格是正常电价的 5 倍，最优解中只在计划电量不足时才启用，符合题意。

约束 2：跨日储电量传递

$$E_{d,0} = E_{d-1,144} \qquad (20)$$

其中把 365 个单日问题串联成一个整体，避免跨日储电量重置的问题。

$$E_{1,0} = 6000 \qquad (21)$$

其中根据题目要求给出 1 月 1 日的初始储电量 6000 kWh。

约束 3：日内 SOC 递推与运行边界

$$E_{s,t} = E_{s,t-1} + \eta_c\, c_{s,t} - \frac{d_{s,t}}{\eta_d} \quad (22)$$

其中，储电量表示情景 s 下第 t 个时段末的电池电量（kWh）。问题二要求跨日连续传递，即当日 24:00 的储电量等于次日 0:00 的初始储电量。

约束 4：储电量运行区间与充放电功率上限

$$1200 \le E_{d,t} \le 10800, \qquad 0 \le c_{d,t} \le 833.33, \qquad 0 \le d_{d,t} \le 833.33 \quad (23)$$

其中，储电量须始终保持在 1200～10800 kWh 的运行区间内；单时段充、放电量均不超过 5000 kW 折算的电量上限 833.33 kWh（Δt = 1/6 h）。

约束 5：取用量松弛与变量非负

计划量是计费量，实际取自电网的电量可以低于承诺的计划量即买多了照付但不必全部取用，该松弛关系与变量非负要求分别为：

$$g_{\omega,t}^{draw} \le g_t, \quad \forall\, \omega, t \quad (24)$$

$$g_t \ge 0, \quad c_{\omega,t} \ge 0, \quad d_{\omega,t} \ge 0, \quad s_{\omega,t} \ge 0, \quad e_{\omega,t} \ge 0 \quad (25)$$

其中，取用量不超过承诺量这一松弛避免了夜间光伏为零而计划量偏高时、因电池单段吸收能力有限而导致的模型不可行。

汇总以上所有部分，针对问题二的两阶段随机规划模型的完整模型如式 (26)。为便于排版，记情景 s 下第 n 天第 t 段的净负荷电量 $N_{n,t}^{(s)} = (L_{n,t}^{(s)} - P_{n,t}^{pv,(s)})\Delta t$，即负载电量与光伏发电量之差；式中 $t = 1, \ldots, 144$，$s = 1, \ldots, S$，n 遍历全部仿真日。

$$\min \sum_t \pi_t g_t \;+\; \frac{1}{S} \sum_{\omega=1}^{S} \sum_t \beta_{\text{emg}}\, \pi_t\, e_{\omega,t}$$

$$\text{st.} \quad \begin{cases} g_{\omega,t}^{\text{draw}} + e_{\omega,t} + pv_{\omega,t}\Delta t + d_{\omega,t} = L_{\omega,t}\Delta t + c_{\omega,t} + s_{\omega,t} \\ E_{\omega,t} = E_{\omega,t-1} + \eta_c c_{\omega,t} - d_{\omega,t}/\eta_d, \qquad E_{\omega,0} = E_d^0 \\ g_{\omega,t}^{\text{draw}} \le g_t \\ 0 \le c_{\omega,t},\, d_{\omega,t} \le 833.33 \\ \qquad 0 \le s_{\omega,t} \le pv_{\omega,t}\Delta t \\ 1200 \le E_{\omega,t} \le 10800, \qquad e_{\omega,t} \ge 0 \end{cases} \quad (26)$$

5.2.4 模型求解

以 HiGHS 为求解后端。由于两阶段随机规划的变量规模随情景数线性增长，全年整体求解将产生数百万个变量，因此采用逐日滚动求解策略：把跨日耦合通过 SOC 初值传递实现，第 t 天求解时以第 t−1 天的末储电量作为本日初值，求解完成后把本日末储电量传给第 t+1 天。这种策略下可以有效地控制单次求解效率，提高模型的计算效率[7]。

步骤 1：数据准备与分型。

读取附件 1 电价、附件 2 全年 365 天的负载与光伏实际功率、附录 1 电池参数；按” 工作日 / 休息日” 对全年日期分型。费用统计报告期为 2025 年 2 月 1 日至 12 月 31 日；1 月 31 天作为预热期，储能初始值取 6000 kWh，经逐日递推后得到 2 月 1 日 0:00 的起始储电量。

步骤 2：情景集生成。

为第 t 天生成情景时只使用第 t 天之前的数据，对同型历史日做几何衰减加权，得到负载与光伏的点预测；从同型历史日的相对残差池中有放回抽取 S=20 组残差在夜间光伏接近 0 时以 ε kW 平滑，避免除零，再将点预测与残差还原为功率曲线，并将光伏出力截断为非负值，最终得到 20 个等概率源荷情景。

步骤 3：情景展开 LP 构建。

构建两阶段情景展开线性规划模型：第一阶段变量为计划购电量，第二阶段变量为各情景下的取用量、充放电量、储电量、弃光量与紧急购电量。目标为计划购电费与紧急购电费期望之和；约束为各情景下的母线平衡、储能递推与容量 / 功率边界，以及取用量不超过计划量。

步骤 4：预热期仿真。

从 2025-01-01 起进入预热期，逐日调用 HiGHS 求解一次步骤 3 的情景展开 LP；求解后按当日实际负载与光伏做实时结算，把当日末储电量递推为次日初值。连续仿真 1 月 31 天，储能从初始 6000 kWh 滚动至 2 月 1 日 0:00，该阶段仅用于确定报告期初始 SOC，不计入费用统计。

步骤 5：报告期逐日滚动求解与实时结算。

以预热期末的储电量为初始值，对 2 月 1 日至 12 月 31 日共 334 天逐日求解；每天调用 HiGHS 求解一次情景展开 LP，求解后按当日实际负载与光伏做实时结算，得到最终的取用量、充放电量与紧急购电量明细，并把当日末储电量递推为次日初值，全年 365 天依次进行。

步骤 6：校验与结果汇总输出。

计算最优计划量的隐含分位并与 0.8 对照，校验结果合理性；截取报告期 334 天的结果，汇总计划购电费、紧急购电费与紧急购电量明细，写入 result2.xlsx。

5.2.5 结果分析

表 **9**　问题二报告期结果汇总

指标	数值
情景数	$S = 20$
求解规模	约 1.74 万变量/日，全年 365 天逐日求解
报告期（334 天）总费用	13 234 106.71 元
— 计划购电费	12 279 178.37 元
— 紧急购电费	954 928.34 元
报告期计划购电量	20 137 477.9 kWh

指标	数值
紧急购电量	329 097.35 kWh（183 天，占购电量 1.6%）
无储能基准（均值电价）	16 407 319.63 元
相对无储能节省	3 173 212.93 元（19.34%）
完全信息对照解（334 天）	11 796 484.28 元
信息不完全的代价	1 437 622.43 元（占完全信息费用的 12.19%）

下表 10 选取春分、夏至、秋分、冬至四个典型节气，对比各日计划购电量、紧急购电量与当日总费用。分析可得夏至光伏资源充足，紧急购电需求最低；冬至光照弱，模型选择提高计划购电量，当日无紧急购电。春分、秋分的光伏和负载波动较大，产生较多紧急购电，推高当日用电成本。

表 **10**　指定日期的紧急购电量

日期	紧急购电量 (kWh)	计划购电量 (kWh)	当日费用 (元)
2025.3.20（春分）	2 849.13	61 850.83	45 159.58
2025.6.21（夏至）	876.44	30 916.30	19 172.67
2025.9.23（秋分）	3 719.01	60 707.23	48 442.37
2025.12.21（冬至）	0.00	92 635.50	58 999.35

如图 9 2025 年逐日购电费日历热力图所示，颜色深浅代表当日购电费用高低。其中峰值为 6.05 万元，估值 1.14 万元可以看出全年逐日费用呈冬夏高、春秋低的季节性波动，最高与最低日相差 5 倍以上。结合问题二电价每天相同这一前提可知，费用差异完全来自光伏资源与负载水平的日间差别，而不是电价。

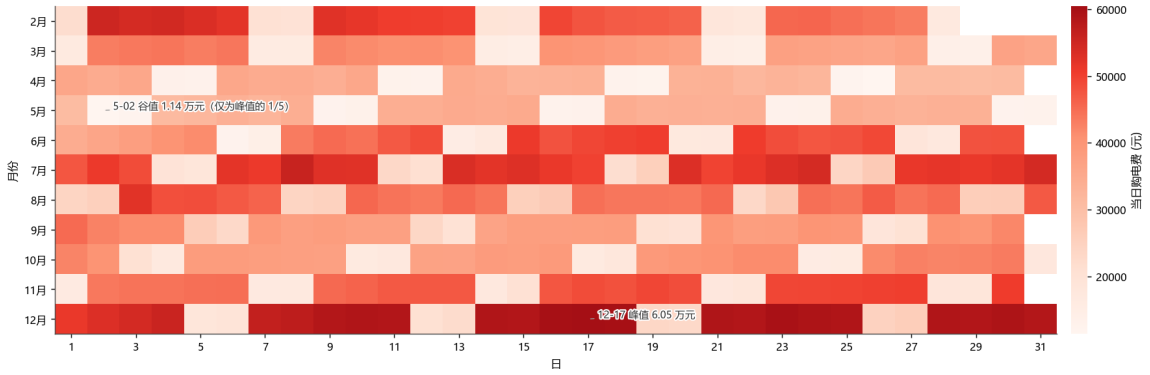


图 **9**　**2025** 年逐日购电费日历热力图

如图 10 月度电量与费用结构图所示，月度购电量与费用同步波动，12 月电量最高和单位成本最高叠加，把费用推到全年峰值；而 4–5 月因光伏充足、净负荷低，电量与费用双低。此处平均电价实为月度结算的单位购电成本，其波动来自购电时段结构而非电价本身。

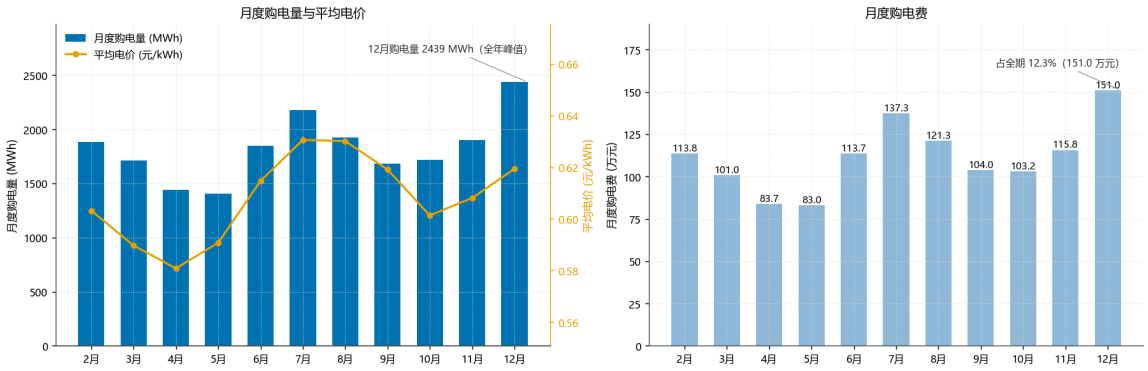


图 **10**　月度电量与费用结构

如图 11 四季典型日调度对比所示，第一列为电价与充放电图，第二列为光伏，负载和购电功率对比图，第三列为当日的 SOC 轨迹。分析可得四个指定日期的调度形态差异显著，差异完全由当天的光伏资源与负载水平决定，即夏至光伏充足、全天购电最少，冬至光伏发电效能减弱，同时购电最高，是为夏至的 3 倍；储能则在每一天内部完成两次满区间循环，白天吸收光伏富余、峰段顶替购电，起到削峰填谷的作用。

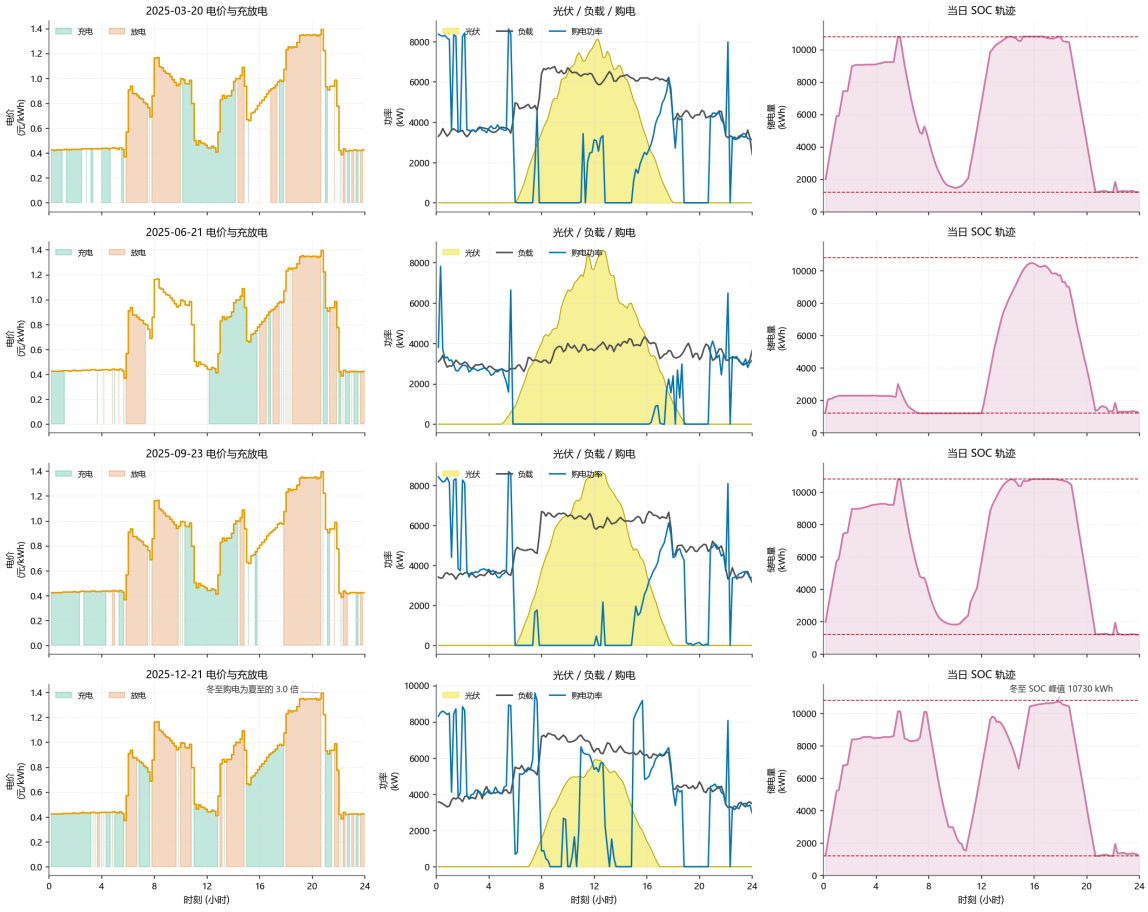


图 **11**　四季典型日调度对比

如图 12 全年储电量轨迹与电池循环强度所示，上图为全年 SOC 运行区间，下图为每日充放电量的校检，其中由于时间跨度大，SOC 运行区间基本保持在 8000 以上，但实际上每天

的电量都会回落到出电量的下界，即 1200kWh，说明在当前参数下电池容量是被用尽的紧约束。而每日的充放电校检图也说明储能装置确实在承担跨越时段搬运电量的工作。

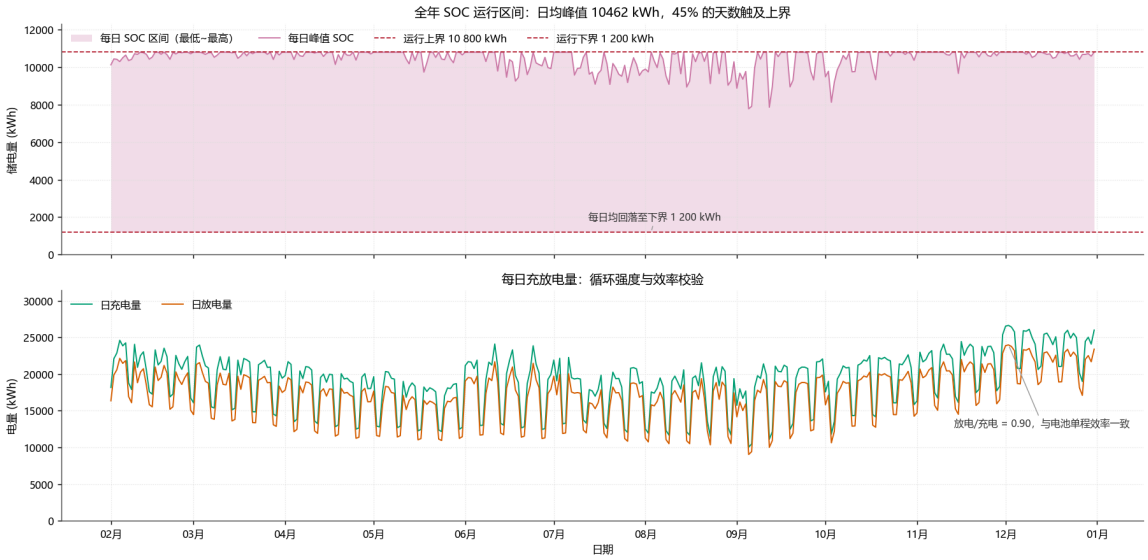


图 **12**　全年储电量轨迹与电池循环强度

如图 13 完美信息的价值与来源分解所示，完美信息的价值在均值电价环境下为 151.76 万元，在实时电价环境下放大到 248.77 万元，约为前者的 1.6 倍；新增的近百万价值来自价格波动这一独立的不确定性来源，说明知道明天的价格信息与知道明天的光伏和负载信息是两类不同的信息。

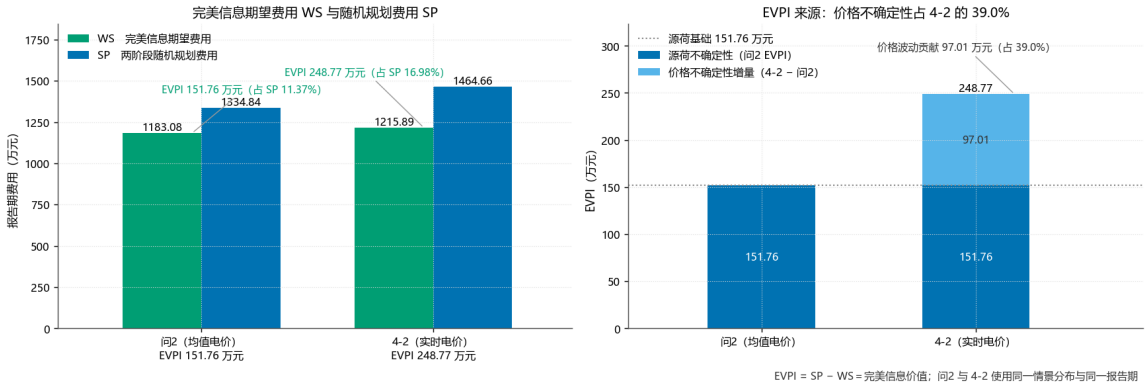


图 **13**　完美信息的价值与来源分解

5.3 问题三的模型的建立和求解

5.3.1 具体分析

问题三在问题二的基础上添加了未来 24 小时整点的光伏发电功率预报，分别以 0:00、6:00、12:00、18:00 为节点各发布一版未来 24 小时整点的光伏功率预报，并且允许在 0:00 之后按新预报调整购电方案，同时修改了惩罚规则，调整后的购电量低于计划时，差额按交易时刻电价的 50% 计违约电价，高于计划时，超出部分按 1.5 倍电价计费，总费用由计划购电

费用、紧急购电费用和调整购电量的相关费用三部分构成。所以本文认为问题三是一种序贯决策问题。

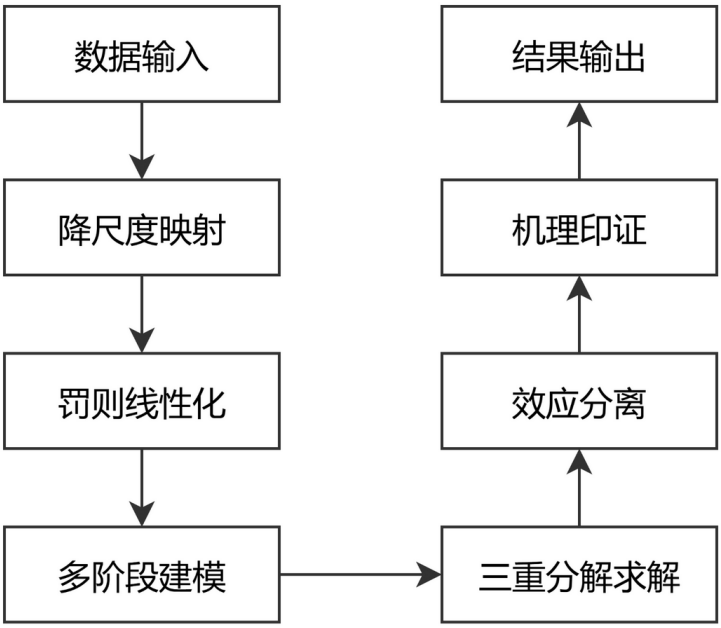


图 **14**　问题三分析流程图

5.3.2 模型准备

1. 数据预处理

针对问题三，本文的数据处理集中在光伏预报上，包含预报语义与降尺度、预报误差画像、惩罚机制的确定三项工作。

步骤 1：预报语义与降尺度。

设第 k 个发布时刻的预报序列为 $F_k = (f_{k,1}, \ldots, f_{k,24})$，其映射到当天整点 h 的值为

$$f_k(h) = f_{k,\, off(h,k)} \qquad (27)$$

$$off(h,k) = max\left\{1,\ (h - \tau_k)\ mod\ 24\right\} \qquad (28)$$

再按分段常数展开为 144 段：

$$P_t^{pv,fc} = f_k\left(\lfloor (t-1)\Delta t \rfloor\right), \qquad t = 1, \ldots, 144. \qquad (29)$$

式 (27) 把“预报 j 小时”翻译成“目标整点的功率值”，式 (29) 把整点值扩到 10 分钟粒度。降尺度方式会影响逐段的偏差罚则触发次数，故在模型检验中另做线性插值的对照。

附件 3 是某天某发布时刻对未来 24 小时的整点预报，日期列在同一天只写一次，其余三行留空，所以向下填充可得到完整预报库。为消除语义歧义，本文统一采用“预报 j 小时 = 发布时刻后 j 小时”的假设，发布时刻本身没有预报值 $\square j = 0$ 的位置，用“预报 1 小时”的值顶替；第 k 版预报映射到当天整点 h 的位置为 $off(h,k) = max\{1, (h - \tau_k)\ mod\ 24\}$，其中 $\tau_k \in \{0, 6, 12, 18\}$。然后再把整点值展开成 144 段，用分段常数作为每段取所属整点的值，同

时另设线性插值作为对照。

步骤 2：预报误差画像。

把降尺度后的预报与附件 2 的实际上报值相减，得到光伏预报误差 $\xi_t = P_t^{pv,fc} - P_t^{pv}$（kW）。白天时段的误差均值为 -63 kW、平均绝对误差 618 kW；分发布版本看，误差均值在 $-70 \sim +38$ kW 之间、标准差 612∼821 kW，整体近似无偏但离散度可观，其中午后存在 12%∼13% 的系统性低估。这条画像说明预报整体不偏、但单点不准的情况，所以本文提出滚动修正误差[9]。

步骤 3：惩罚机制设定

将违约电量与超额电量定义为 $u_{n,t} = (g_{n,t}^{plan} - g_{n,t}^{adj})^+$、$v_{n,t} = (g_{n,t}^{adj} - g_{n,t}^{plan})^+$。由于目标函数中 u、v 的系数 $0.5p_t$ 与 $1.5p_t$ 均为正，最优解中两者会自动取到下界，从而恒有 $u \cdot v = 0$，无需引入绝对值或取大这类非线性运算。

2. 算法选择。

针对问题三，分析可得该问属于信息随时间到达的序贯决策问题，若采用确定性优化将把全天实际数据当作已知，那么惩罚机制无法触发，若采用鲁棒优化则只给出最坏情景下的可行解，无法量化期望费用，也就回答不了题目需要回答的 “是否需要引入其他预报指定计划” 的问题；本文考虑到序贯决策的价值恰恰在于把每个时点新增的预报信息转化为可货币化的收益，故选择多阶段滚动随机规划求解[10]，以预报误差的经验分布作为情景集，在四个决策时点依次求解剩余时段的线性子问题，沿用 HiGHS 求解器逐阶段求全局最优，各阶段子问题为纯线性规划；在求解时，每次求解前重置随机种子，保证各对照配置的情景集逐位相同；设计 A、B、C 三重受控实验把决策机会价值与信息价值分离，避免把决策机制的负效应记成信息的正价值，并辅以风险偏好连续性、阶段模型形式替换等稳健性检验验证。

5.3.3 模型建立

步骤 1：定义决策变量

本文的决策变量按四个决策时刻分组：0:00 确定全天计划购电量，6:00、12:00、18:00 各确定一次剩余时段的调整购电量。为把不对称偏差罚则写成线性形式，另引入两个非负辅助变量——违约电量（计划量高于调整量的部分）与超额电量（调整量高于计划量的部分），四组变量满足

$$g_t^{plan} \geq 0, \quad g_t^{adj} \geq 0, \quad u_t \geq 0, \quad v_t \geq 0 (30)$$

其中，计划购电量为 0:00 确定的承诺量（kWh）；调整购电量为各决策时刻修正后的购电量（kWh）；违约电量为调整量低于计划量的部分（kWh）；超额电量为调整量高于计划量的部分（kWh）。三者的恒等关系为

$$g_t^{adj} = g_t^{plan} - u_t + v_t (31)$$

因此绝对值型的分段罚则被等价地写成线性约束，模型无需引入 0-1 变量。

步骤 2：确立目标函数

$$C^{\text{plan}} = \sum_{d}\sum_{t} \pi_t \cdot min\left(g_{d,t}^{\text{plan}},\, g_{d,t}^{\text{adj}}\right) (32)$$

$$C^{\text{emg}} = \beta_{\text{emg}} \sum_{d}\sum_{t} \pi_t\, e_{d,t} \qquad (33)$$

$$C^{\text{adj}} = \sum_{d}\sum_{t} \pi_t \left(\beta_{\text{def}}\, u_{d,t} + \beta_{\text{exc}}\, v_{d,t}\right) \qquad (34)$$

$$C^{\text{total}} = C^{\text{plan}} + C^{\text{emg}} + C^{\text{adj}} (35)$$

其中式（32）为计划购电费用，是按计划量与调整量中的较小者以正常电价计费，式（33）是紧急购电费用，缺口按 5 倍电价补购即 β_{emg}=5；式（34）是调整购电量的相关费用，对计划与调整的差额加收罚金，少买部分按 $0.5\pi_t$ 计违约费，多买部分按 $1.5\pi_t$ 计超额费，即 $\beta_{\text{def}} = 0.5,\ \beta_{\text{exc}} = 1.5$。三项相加即得目标总费用。

因 $v = (g^{\text{adj}} - g^{\text{plan}})^{+}$，有 $min(g^{\text{plan}}, g^{\text{adj}}) = g^{\text{adj}} - v$ 故式（32）可化为 $\pi_t(g_{d,t}^{\text{adj}} - v_{d,t})$ 的纯线性形式，模型无需引入任何非线性运算。故目标函数可写成以下形式：

$$min\ C^{\text{total}} = \sum_{d}\sum_{t} \pi_t \left(g_{d,t}^{\text{adj}} + 0.5\, u_{d,t} + 0.5\, v_{d,t} + 5\, e_{d,t}\right) (36)$$

步骤 3：设立约束条件

约束 1：偏差罚则的线性化

$$u_{d,t} \geq g_{d,t}^{\text{plan}} - g_{d,t}^{\text{adj}}, \qquad u_{d,t} \geq 0 (37)$$

$$v_{d,t} \geq g_{d,t}^{\text{adj}} - g_{d,t}^{\text{plan}}, \qquad v_{d,t} \geq 0 (38)$$

由于目标函数中 u 与 v 的系数即 $0.5\pi_t$ 与 $1.5\pi_t$ 均严格为正，最优解中 u 与 v 会自动取到其下界，从而恒有 $u = (g^{\text{plan}} - g^{\text{adj}})^{+}$、$v = (g^{\text{adj}} - g^{\text{plan}})^{+}$。避免引入绝对值或取大运算这类非线性项，使模型保持线性，可用线性规划求解器求得全局最优。

约束 2：取用量的松弛

$$0 \leq g_{d,t}^{\text{draw}} \leq g_{d,t}^{\text{adj}} (39)$$

承诺的计划量 g^{plan} 与实际取自电网的电量 g^{draw} 是两个不同的量。计费基准是计划量与调整量中的较小者，故取用量不可能超过调整量。

约束 3：多阶段决策的非预期性

决策变量按决策时刻划分为 4 组，每组只能依赖于当时已获得的信息：

表 **11**　问题三的多阶段信息结构

阶段 k	决策时刻	可观测信息	决策变量
0	0:00	预报 F_0（全天 24 小时）	$g_{d,t}^{\text{plan}},\ \forall t$
1	6:00	预报 F_1 ＋前 6 小时实际	$g_{d,t}^{\text{adj}},\ t \geq 37$

阶段 k	决策时刻	可观测信息	决策变量
2	12:00	预报 F_2 ＋前 12 小时实际	$g_{d,t}^{\text{adj}},\ t \geq 73$
3	18:00	预报 F_3 ＋前 18 小时实际	$g_{d,t}^{\text{adj}},\ t \geq 109$

根据非预期性约束，即阶段 k 的决策不得依赖于阶段 k 之后才会揭晓的信息。

约束 4：与其他问题共用的约束骨架

母线功率平衡（式（19））、跨日储电量传递（式（20））、日内储电量递推（式（22））、储电量运行区间与充放电功率上限（式（23））以及全部变量的非负约束，在本问中沿用问题二的形式，只需把下标替换为多阶段决策的记号，这里便不展开赘述。

汇总以上各部分，针对问题三的多阶段滚动随机决策模型的完整模型如式 (40)。式中 $t = 1, \ldots, 144$，n 遍历全部仿真日，τ_k 为第 k 次预报的发布时刻，$\mathcal{I}_k$ 为阶段 k 已获得的信息集。

$$
\begin{aligned}
&\min\ Z = \sum_{n} \sum_{t=1}^{144} p_t \left(g_{n,t}^{adj} + \beta_{def} u_{n,t} + \beta_{exc} v_{n,t} + \beta_{em} g_{n,t}^{em} \right) \\
&st. \begin{cases}
g_{n,t}^{adj} + g_{n,t}^{em} + d_{n,t} = N_{n,t} + c_{n,t} + w_{n,t} \\
e_{n,t} = e_{n,t-1} + \eta_c \, c_{n,t} - \dfrac{d_{n,t}}{\eta_d} \\
e_{n,0} = e_{n-1,144}, \qquad e_{1,0} = 6000 \\
1200 \leq e_{n,t} \leq 10800, \qquad 0 \leq c_{n,t},\, d_{n,t} \leq 833.33 \\
u_{n,t} \geq g_{n,t}^{plan} - g_{n,t}^{adj}, \qquad u_{n,t} \geq 0 \\
v_{n,t} \geq g_{n,t}^{adj} - g_{n,t}^{plan}, \qquad v_{n,t} \geq 0 \\
0 \leq g_{n,t}^{draw} \leq g_{n,t}^{adj}, \qquad 0 \leq w_{n,t} \leq P_{n,t}^{pv,fc} \Delta t \\
P_{n,t}^{pv,fc} = f_k \left(\lfloor (t-1)\Delta t \rfloor \right) \\
\text{阶段} k \text{的决策不依赖阶段} k \text{之后的信息 } (\mathcal{I}_k) k = 0, \ldots, 3
\end{cases}
\end{aligned}
\tag{40}
$$

5.3.4 模型求解

步骤 1：预报降尺度与误差统计

读入附件 3 的 $365 \times 4 \times 24$ 条整点预报，按式 (27)、(29) 展开为逐日 144 段的预报序列；与附件 2 的实际值相减，得到误差画像与各窗口的均方根误差。

步骤 2：四阶段滚动求解

对每一天，依次在 0:00、6:00、12:00、18:00 求解 “剩余时段” 的子问题，用实际已发生的数据更新储电量状态，用当次预报处理未来时段，已发生的购电行为不可追溯修改，因此每个阶段只优化剩余时段。

步骤 3：结算与费用分解

按当日实际的负载与光伏结算取用量、充放电量与紧急购电量，分别统计基准购电费、违

约费、超额费与紧急购电费，写入 result3.xlsx。

步骤 4：信息价值受控实验

在每次求解前重置随机种子，保证各配置的情景集逐位相同，再分别求解方案 A、B、C 以及只替换单个时点预报的 C1、C2、C3。

5.3.5 结果分析

表 **12**　报告期费用分解

费用项	金额 (元)	对应电量
基准购电费用	11 815 574.31	—
违约费用	496 941.64	1 234 024.55 kWh
超额费用	390 424.71	371 686.77 kWh
紧急购电费用	1 836 413.28	618 226.53 kWh
合计	14 539 353.94 元	
相对无储能节省	1 867 965.69 元（11.38%）	
非零少买 / 多买 / 紧急时段数	8 877 / 2 071 / 7 070（总 48 096）	
发生紧急购电的天数	300 / 334（89.8%）	
调整量 / 计划量	95.81%	

表 **13**　指定日期的紧急购电量

日期	紧急购电量 (kWh)	计划购电量 (kWh)	调整购电量 (kWh)	偏差 (kWh)
2025.3.20（春分）	6 039.20	65 892.64	58 683.83	−7 208.81
2025.6.21	2 212.86	31 869.50	30 044.37	−1 825.12
2025.9.23	3 266.60	67 630.51	61 861.13	−5 769.37
2025.12.21	1 706.30	92 892.99	90 539.54	−2 353.46

表 **14**　偏差量的统计描述

变量	均值	标准差	最大值	变异系数
少买量 u（kWh/段）	25.66	88.61	1 314.38	3.454
多买量 v（kWh/段）	7.73	60.87	1 219.85	7.877
紧急购电量 e（kWh/段）	12.85	65.62	971.36	5.105
日总偏差（kWh）	4 807.52	3 423.92	25 736.28	0.712

表 **15**　信息价值受控实验

编号	配置	报告期总费用（元）
A	仅 0:00 决策	14 634 776.93
B	三时刻改签，但预报始终用 0:00 版	15 165 327.94
C	三时刻改签，各用当期预报	14 539 353.94
C1	仅 6:00 改用 0:00 版预报	14 697 259.30
C2	仅 12:00 改用 6:00 版预报	14 781 793.85
C3	仅 18:00 改用 12:00 版预报	14 539 355.72

如图 15 预报误差画像与计划-调整偏差图所示，左图为白天时段预报误差分布图，中图为计划与调整偏差分布图，右图为 2025-12-21 计划和调整对比图。分析可得，预报整体不偏，但单点误差可达大，而调整的实际动作以“少买”为主，说明模型倾向在获得新预报后下调承诺量以减少沉没成本。

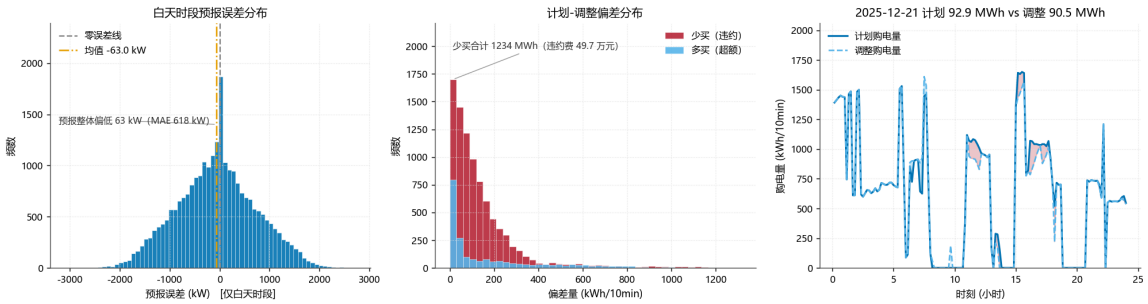


图 **15**　预报误差画像与计划-调整偏差图

如图 16 信息价值分析图所示，左图为 A、B、C 三种配置的费用分布图，B 比 A 贵 53.1 万元，C 比 B 省 62.6 万元，分析可得，单纯增加改签机会是负价值，而信息更新是正价值，两者相抵净赚约 9.5 万元；右图为各发布时刻的边际信息价值图，分时看，12:00 预报的价值最高、6:00 次之、18:00 几乎为零，说明傍晚再增加预报时刻没有意义，说明发布频率不等于有效信息。可得结论，预报建设的重点应从“提高发布频率”向“确保每次发布都带来真正的新信息”转变。

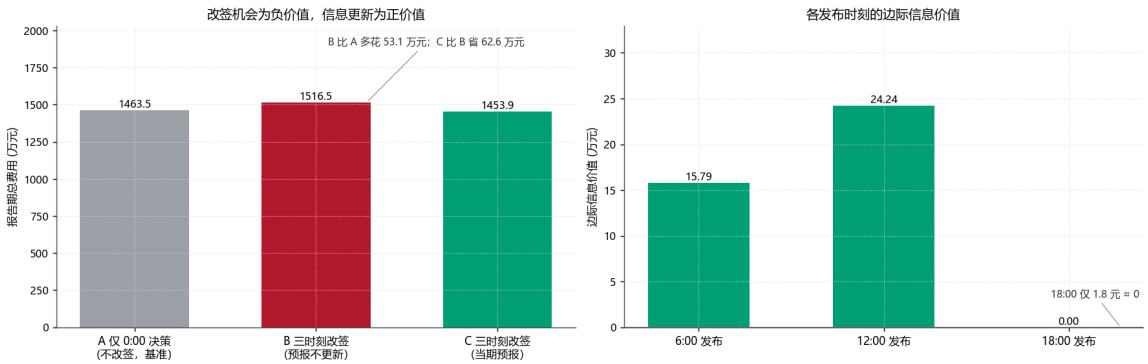


图 **16**　信息价值分析图

5.4 问题四的模型的建立和求解

5.4.1 具体分析

问题四指出“实际上，外网的电价也是实时波动的”，并要求“在波动电价下重新计算问题二和问题三”。依据题目所说，是“重新计算”而非“重新建模”，因此本问的首要工作是复用问题二、三的模型骨架，仅更换电价数据源。在此之上，应该注意到实时电价这一真实情况是需要被纳入决策，可选择引入新的风险控制模型来优化原本的两阶段随机规划模型。

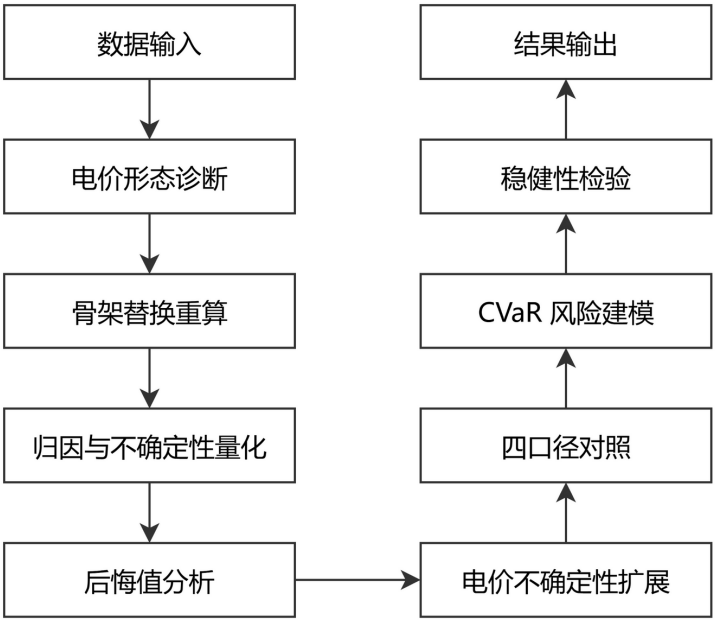


图 **17**　问题四分析流程图

5.4.2 模型准备

1. 数据预处理。

问题四的数据处理以电价数据为核心，主要开展电价形态特征诊断与仿真基准的界定工作。通过逐时段比对附件 1 固定电价与附件 4 实时电价数据可知，两类电价日内峰谷形态高度一致、均值基本等同、单点偏差极小，核心差异体现在波动离散程度上；实时电价极差为固定电价约 1.7 倍，价格分布显著拓宽，且正午光伏高发时段出现多处极端低价点，契合新能源并网下的电价鸭形曲线市场特征，属于真实电价波动现象。整体而言，固定电价仅包含稳定日内峰谷特性，而实时电价在其基础上叠加了逐日随机波动特征，为后续分析电价波动对调度成本与储能套利策略的影响提供了数据基础。

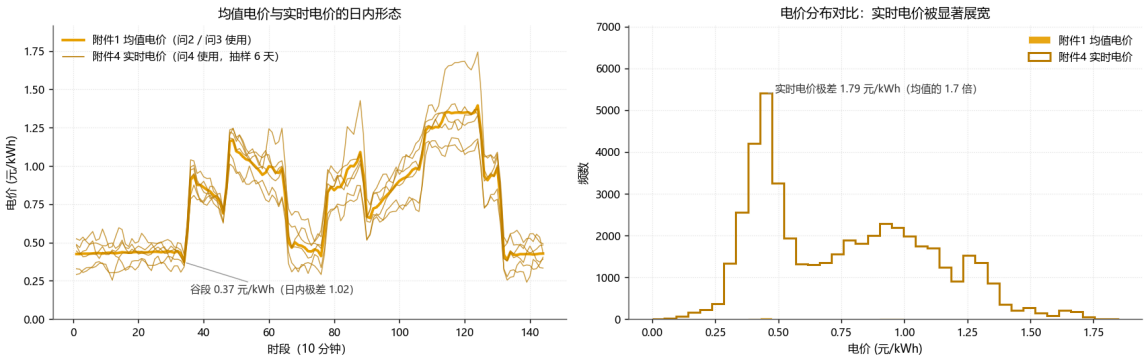


图 **18**　电价形态对比图

2. 算法选择

针对问题四，由于线性规划可在多项式时间内求得精确全局最优，启发式只能得到近似解且需要反复调参，反而降低结果的可复现性，本文将沿用问题二和问题三的模型结构。在此基础上，本文将建立电价不确定性两阶段随机规划与 CVaR 风险模型来作为模型优化，以实现实时电价纳入决策的想法，并且在模型检验中引入后悔值分析来检验费用变化究竟源于电价波动的客观影响，还是决策策略本身的偏差。

5.4.3 模型建立

步骤 1：确立目标函数

（1）确定性基准模型

4-2 模型与问题二相同，仅将 π_t 替换为 $\pi_{d,t}$：

$$min\ C^{\text{total}} = \sum_{d}\sum_{t} \pi_{d,t}\, g_{d,t} \ + \ 5 \sum_{d}\sum_{t} \pi_{d,t}\, e_{d,t} (41)$$

4-3 模型与问题三相同，仅将 π_t 替换为 $\pi_{d,t}$：

$$min\ C^{\text{total}} = \sum_{d}\sum_{t} \pi_{d,t} \left(g_{d,t}^{\text{adj}} + 0.5\, u_{d,t} + 0.5\, v_{d,t} + 5\, e_{d,t} \right) (42)$$

（2）不确定性增强模型

设情景 ω 下的电价、负载与光伏实现分别为 $\pi_{d,t}^{\omega}$、$L_{d,t}^{\omega}$、$P_{d,t}^{\omega}$，则

$$min\ \mathbb{E}[C] = \sum_{\omega} p_{\omega} \sum_{d} \left[\sum_{t} \pi_{d,t}^{\omega}\, g_{d,t}^{\text{plan}} + \sum_{t} \pi_{d,t}^{\omega} \left(0.5\, u_{d,t} + 0.5\, v_{d,t} + 5\, e_{d,t}^{\omega} \right) \right] (43)$$

其中第一阶段费用 $\sum_t \pi_{d,t}^{\omega} g_{d,t}^{\text{plan}}$ 的是不确定揭晓前就锁定的计划量，故它对 ω 取期望；第二阶段费用 $\sum_t \pi_{d,t}^{\omega}(0.5u + 0.5v + 5e^{\omega})$ 随情景实现而变化。

（3）风险厌恶目标

为控制极端价格情景造成的巨额损失，可进一步引入条件风险价值，称为 CVaR[11]：

$$min\ \mathbb{E}[C] + \lambda \cdot CVaR_{\alpha}(C) (44)$$

$$CVaR_{\alpha}(C) = \min_{\eta} \left\{ \eta + \frac{1}{1-\alpha} \sum_{\omega} p_{\omega} (C_{\omega} - \eta)^{+} \right\} (45)$$

其中 η 为辅助变量，其最优值即费用分布的 α 分位点, 即风险价值 VaR_α。$CVaR_\alpha$ 度量“最坏的 $(1-\alpha)$ 比例情景下的平均费用”，取 $\alpha = 0.95$, 即关注最坏的 5% 情形；$\lambda \geq 0$ 为风险厌恶系数（保守度参数），$\lambda = 0$ 退化为纯期望值模型，λ 越大越保守。CVaR 在此是作为目标函数的惩罚项参与优化，把风险厌恶直接灌输进第一阶段决策；同时也作为评价指标，刻画最优解在费用分布尾部的表现。

步骤 2：设立约束条件

约束 1：取用量不超过计划量。

$$g_{d,t}^{draw,\omega} \leq g_{d,t}^{plan}, \qquad \forall \omega \in \Omega,\ t \in \mathcal{T} \tag{46}$$

其中，该约束是连接第一、二阶段变量的唯一耦合条件：无论源荷与电价情景如何实现，实际取自电网的电量都不得超过事前承诺的计划量，超额采购的部分形成沉没成本。它使计划量成为不确定性揭晓之前的 “承诺—权衡” 变量，模型需在过量采购的浪费与缺额的高额应急成本之间权衡，构成报童结构。

约束 2：CVaR 的线性化。

$$\pi_{d,t}^{\omega}\, g_{d,t}^{plan} + 5\pi_{d,t}^{\omega}\, g_{d,t}^{em,\omega} + c_{cyc}\, d_{d,t}^{\omega} - \eta - z_\omega \leq 0, \qquad z_\omega \geq 0 \tag{47}$$

其中，左端前三项为对应情景的总费用，第四项为可选的循环成本项；该式与配套的辅助变量非负条件共同实现了条件风险价值中正部函数的精确线性化，使含风险项的目标函数仍然保持线性规划形式。

约束 3：与其他问题共用的约束骨架

母线功率平衡（式（19））、跨日储电量传递（式（20））、储电量运行区间与充放电功率上限（式（23））以及全部变量的非负约束，在本问中沿用问题二的形式，只需把下标替换为多阶段决策的记号，这里便不展开赘述。

汇总以上所有部分，问题四的波动电价调度模型的完整模型如下：

$$min\ C^{\text{total}} = \sum_d \sum_t \pi_{d,t}\, g_{d,t} \ +\ 5 \sum_d \sum_t \pi_{d,t}\, e_{d,t}$$

$$st. \begin{cases} g_{n,t}^{adj} + g_{n,t}^{em} + d_{n,t} = N_{n,t} + c_{n,t} + s_{n,t} \\ e_{n,t} = e_{n,t-1} + \eta_c\, c_{n,t} - \dfrac{d_{n,t}}{\eta_d} \\ g_{\omega,t}^{\text{draw}} \leq g_t^{\text{plan}} \\ \underline{E} \leq E_{\omega,t} \leq \bar{E}, \quad 0 \leq c_{\omega,t},\, d_{\omega,t} \leq \bar{E}_{\text{seg}} \\ \sum_t \pi_t^{\omega}\, g_t^{\text{plan}} + 5 \sum_t \pi_t^{\omega}\, e_t^{\omega} + c_{\text{cyc}} \sum_t d_t^{\omega} - \eta - z_\omega \leq 0 \\ u_{n,t} \geq 0, \qquad v_{n,t} \geq 0 \\ R = C^{\text{det}}\left(\pi^{\text{actual}}\right) - C^{*}\left(\pi^{\text{actual}}\right), \qquad \Delta = \dfrac{R}{C^{*}\left(\pi^{\text{actual}}\right)} \times 100\% \\ 0 \leq s_{\omega,t} \leq P_{\omega,t}^{\text{pv}}\, \Delta t, \quad g_t^{\text{plan}},\, g_{\omega,t}^{\text{draw}},\, e_{\omega,t} \ \geq 0 \end{cases} \tag{48}$$

5.4.4 模型求解

步骤 1：模型求解

读入附件 4 电价矩阵、诊断与附件 1 形态差异，采用 SciPy 调用 HiGHS 求解器求全局最优，嵌入逐日滚动框架，报告期 334 天每日先生成第一阶段决策，再按当日实际源荷做实时结算，末储电量传至次日；4-2、4-3 分别复用问题二、三的随机规划程序，仅替换价格矩阵。

步骤 2：线性规划的矩阵化实现

变量按固定分块拼成一维向量，索引由常量统一维护；等式约束逐情景、逐时段装配母线平衡与 SOC 递推，不等式约束含取用量上限及 CVaR 辅助项，上下界经 bounds 参数直传而非写成约束行；系数矩阵每行仅 4～5 个非零元，采用 COO→CSR 稀疏格式装配。

步骤 3：冷启动与储电量传递

报告期初前 5 天历史样本不足，直接用当日实际数据构造情景、以当日已知电价为价格依据，第 6 天起启用相似日预测与残差池。跨日耦合通过储电量传递实现，当日起始储电量作为参数代入递推约束首行，结算后取当日末储电量为次日初值；若误用固定初始值，计划量会与电池真实状态脱节，使费用虚高。

步骤 4：实时结算

承诺购电量锁定后，按当日实际源荷单独求解一个结算 LP：目标仅为最小化紧急购电费用，计划购电费为沉没成本，不进入目标，约束为母线平衡、SOC 递推与取用量上限，求解后确认多买的部分不被取用，并按实际电价核算当日费用。

步骤 5：情景生成与计算方案设置

电价点预测采用相似日加权，三类残差池按同一历史日对齐构造；联合方案三类情景共用抽样索引，独立方案分别抽样。为分离电价信息与抽样方式两种效应，设置 A、B、C、D 四种方案，共用同一随机种子与情景数，保证差异只来自方案本身。

表 **16**　电价形态对比

方案	电价处理	情景抽样方式	用途
A	按当日实际电价	负载与光伏各自独立抽取	复现本文交付的 4-2 结果，作为一致性基准
B	按当日实际电价	三源按同一块索引联合抽取	隔离 “抽样方式” 的影响
C	相似日电价情景	源荷独立抽取	隔离 “电价处理” 的影响
D	相似日电价情景	三源联合抽取	电价不确定性的正式约定与 CVaR 扫描基准

5.4.5 结果分析

表 **17**　问题四求解结果

模型	报告期费用（334 天）	紧急购电量 (kWh)
4-2	14 274 280.36 元	345 905.66
4-3	15 239 867.10 元	669 885.00

模型 4-3 的费用为：基准 12281078.67 ＋违约 489373.82 ＋超额 411427.45 ＋紧急 2057987.15 = 15239 867.10 5 元。

表 **18**　与固定电价下的对照

模型	报告期费用（元）	相对固定电价
问题二 → 4-2	13234106 → 14274280	＋ 1040173.65
问题三 → 4-3	14539353→ 15239867	＋ 700513.16

相对无储能：4-2 节省约 16%、4-3 节省约 10%。

表 **19**　后悔值与随机解的价值

方案	报告期费用（元）
方案 A（均值电价计划）：用均值电价做计划，按实时电价完整结算	13941420.44
方案 B（实时电价优化）：直接用实时电价做最优调度	14274280.36
后悔值 R	−332859.92 元

表 **20**　随机解的价值

策略	报告期费用（元）
期望值策略	17510771.14
4-2 两阶段随机规划	14274280.36
4-3 滚动决策	15239867.10
VSS(4-2)	＋ 3236490.78 元
VSS(4-3)	＋ 2270904.05 元

如图 19 后悔值分析图所示，左图为两种规划方案的费用对比图，均值电价计划（图中方案 A）为 1 394.1 万元，实时电价优化（图中方案 B）为 1 427.4 万元，后悔值约为 −33.3 万元，出现按均值电价做计划反而更省的情况。右图为两种方案与均价的平均计划购电量对比，两条购电曲线在大部分时段几乎重合，分离只出现在尾段。同时说明用错误的均值价格做计划未必是坏事，均值电价计划本身带有对冲价值。

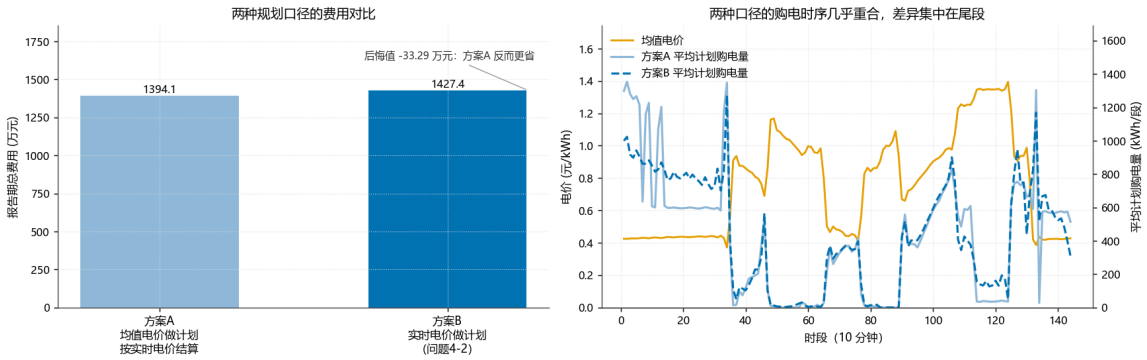


图 **19**　后悔值分析图

如图 20 电价不确定性与 CVaR 分析图所示，左图为四种方案的报告期费用图，每个方案的差距不是很明显；右图为 CVaR 风险前沿分析图，λ 为风险厌恶系数，横轴往右表示尾部风险降低，纵轴为报告期期望费用。分析可得，把电价真正当作随机变量，即 D 方案只比“已知电价”多花 1.07%，说明“不知道电价”的代价有限。引入 CVaR 后，模型可以显式地在“期望费用”与“尾部风险”之间做转换，当前的 λ 在 0.2 时尾部风险下降 37.9%，代价是期望费用上升 14.1%，且曲线在 λ>0.15 后出现明显拐点，说明过度保守的性价比开始变差。

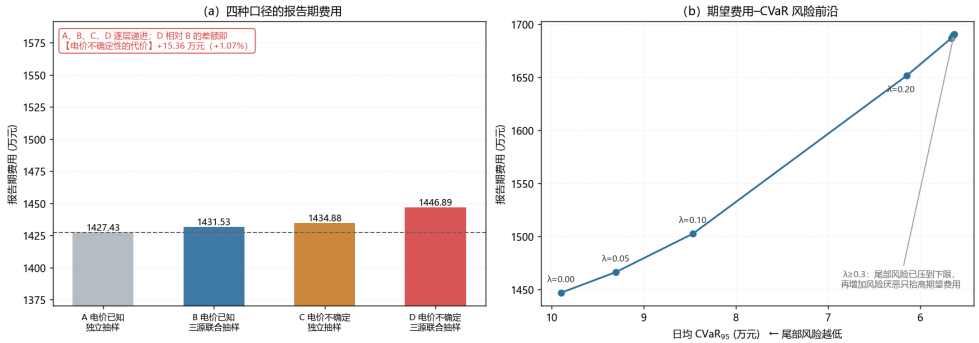


图 **20**　电价不确定性与 **CVaR** 分析图

六、 模型检验与分析

6.1 四问求解结果汇总对比

如图 21 四问求解结果对比图所示，储能装置把费用从 1 605 万元压到 1 129 万元下降约 29%，而信息不完全与计费规则让费用逐级回升，即问题二的 1 323 万元到问题三的 1 454 万元，实时波动电价再让问四比对应的问二、问三各多花约 104 万元与 70 万元；可清楚看出“储能价值”与“信息代价”是如何共同决定最终费用的。

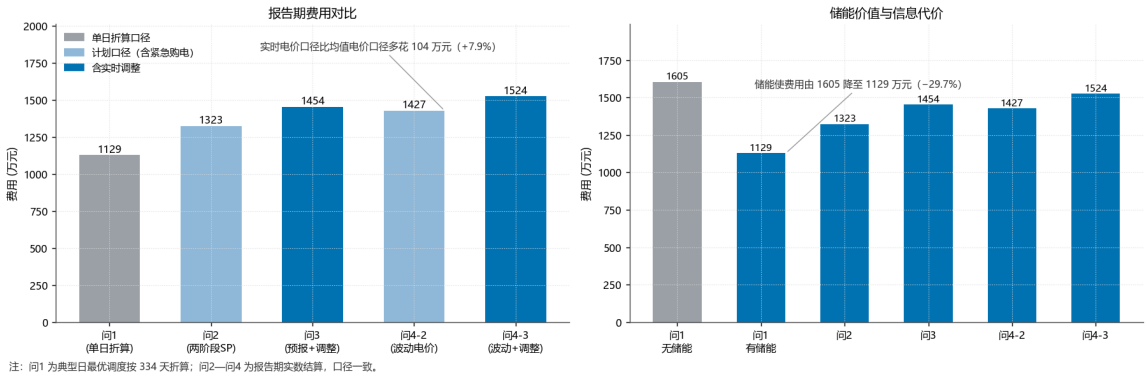


图 **21**　四问求解结果对比图

6.2 各项模型检验与分析

（1）报童分位与情景集检验。逐日复算，实际净负荷在情景分布中的平均分位为 56.0%（理论 50%），超出 P80 阈值的天数占 28.7%（理论 20%），情景集近似无偏、宽度合理；由 5 倍紧急购电导出的理论临界分位 0.8 与最优计划量的隐含分位一致，报童结构成立；情景数由 10 增至 50 时最优费用仅漂移 1.14%，独立与成对抽样的费用差仅 +0.287%。

（2）KKT 与最优性检验。最优解代回问题一全部约束的最大违反量为 9.1×10^{-13}，互补松弛条件吻合率 100%，强对偶成立；单纯形法与内点法目标值相对差为 0。

（3）EVPI 双路径互验。分别按“一次实际实现”与“对分布取期望”两种路径计算完美信息价值，得 143.76 万元与 151.76 万元，量级与结论一致；实时电价环境下该价值放大到 248.77 万元，约为均值电价环境的 1.6 倍。

（4）灵敏度分析。容量、峰谷价差、效率、功率与电价水平五维扫描结果与模型结构性质一致，其中电价水平维度呈严格一次齐次性（弹性 +1.00）；惩罚倍数 β 的两条流水线重跑表明 β≥2 时后悔值恒为负（临界 $\beta^* = 2$）；循环成本两层扫描给出临界值 0.92 / 1.10 / 1.40 元/kWh，低于 1.10 时“弃光全部消纳”不变；降尺度方式与保守度 ρ 的结论同样稳健。

（5）一致性与退化检验。分季度成本变异系数 CV = 0.02；阶段 LP 情景化后决策机会价值仍为负（−42.15 万元；基准 −53.1 万元）；λ=0 退化对拍计划量逐位相同，CVaR 线性化无误差，四问模型各项检验全部通过。

七、 模型的评价

7.1 模型的优点

• **优点 1：**全模型保持线性规划形式，可保证全局最优。本文证明同一时段同时充放电为严格劣策略，因此无需引入整数变量，各问模型均可由求解器在多项式时间内收敛到精确全局最优，避免了启发式算法的局部最优与参数调优问题，且模型不含随机成分，结果完全可复现。

• **优点 2：**模型骨架与数据源解耦，扩展性强。四问仅在电价数据源、预报来源与是否滚

动调整三处做局部替换，即可复用同一套功率平衡与储能递推约束，问题四一次替换即产出两套全量结果，四问结果天然自洽、可交叉验证。

• 优点 3：结论带有明确且可核查的适用边界。题目中会改变数值结果的约定歧义被逐项识别并转化为对照分析，模型的稳健性以参数扫描的方式给出，明确了结论在何种条件下成立、在何种条件下翻转，避免了结论被过度外推。

• 优点 4：将决策机会与信息更新两类价值分离评价，发现增加改签机会反而推高费用、收益只来自信息更新，故预报发布频率不等于信息量。

7.2 模型的缺点

• 缺点 1：忽略电池循环寿命损耗，模型给出的循环强度偏高。最优解下电池长期处于高频次充放状态，而模型未计入由此产生的容量衰减与更换成本，并假设电池容量与效率恒定，与长期运行中容量衰减、效率漂移的实际情况不符；此外，部分定性结论对循环成本这一外生参数较为敏感，取值偏高时会发生翻转。

• 缺点 2：多日问题采用逐日滚动结算，跨日耦合实际退化。结算层以当日为单位独立求解，当日末的储电量在本日内不产生价值，最优解遂每天把电池放至下界，日初与日末储电量恒相等且停在下界；跨日递推约束虽被满足，但已不承载能量搬运，紧急购电被系统性高估。该偏差方向保守，但在日间价差更大的数据上可能放大。

八、 模型的改进与推广

8.1 模型的改进

当前模型仍存在可优化的空间。在信息层面，可将阶段优化的确定性处理升级为情景化随机规划，以情景集刻画光伏预报误差，使滚动决策在购电经济性与供电可靠性之间显式权衡；对问题四则可将电价由已知参数升级为随机变量，采用分布鲁棒优化[12]，只假设真实分布落在经验分布附近的不确定集内，给出期望费用与风险之间的有效前沿，使决策可按风险偏好选择。在求解层面，可将逐日独立结算升级为全期联合结算，使储能真正跨日搬运能量，消除结算层短视带来的保守偏差。在参数层面，可引入容量衰减状态变量，使电池可用容量随循环历史单调下降并计入更换时点，把单年度费用最小扩展为全生命周期成本最小。

8.2 模型的推广

本文的模型体系不局限于本题给定的微网。在场景层面，凡同时具备储能、价差与不确定性的系统均可复用本文的建模路径，如工商业用户侧储能的峰谷套利与电动汽车充电站的购储协同。在方法层面，信息价值分解框架不依赖具体模型，可用于任何信息随时间到达的决策问题，如供应链滚动补货与电力现货市场滚动申报，其要点在于决策灵活性与信息更新必须分开评价。在数据层面，模型只依赖电价、负荷、光伏三类输入矩阵与一组电池参数，更换数据源即可运行，无需调整模型结构。

九、 参考文献

[1] LIU X, ZHAO T, DENG H, et al. Microgrid energy management with energy storage systems: a review[J]. CSEE Journal of Power and Energy Systems, 2023, 9(2): 483-504. DOI: 10.17775/cseejpes.2022.04290.

[2] HOSSAIN LIPU M S, ANSARI S, MIAH M S, et al. A review of controllers and optimizations based scheduling operation for battery energy storage system towards decarbonization in microgrid: challenges and future directions[J]. Journal of Cleaner Production, 2022, 360: 132188. DOI: 10.1016/j.jclepro.2022.132188.

[3] 马恺珧, 王国庆, 于雷. 不确定性环境下微电网优化调度综述 [J]. 工程研究——跨学科视野中的工程, 2023, 15(2): 93-103.

[4] 姜启源, 谢金星, 叶俊. 数学模型 [M]. 5 版. 北京: 高等教育出版社, 2018: 1-40.

[5] BIRGE J R, LOUVEAUX F. Introduction to stochastic programming[M]. 2nd ed. New York: Springer, 2011: 1-35. DOI: 10.1007/978-1-4614-0237-4.

[6] HERDING R, ROSS E, JONES W R, et al. Stochastic programming approach for optimal day-ahead market bidding curves of a microgrid[J]. Applied Energy, 2023, 336: 120847. DOI: 10.1016/j.apenergy.2023.120847.

[7] SALEEM M I, SAHA S, IZHAR U, et al. Bi-layer model predictive control strategy for techno-economic operation of grid-connected microgrids[J]. Renewable Energy, 2024, 236: 121441. DOI: 10.1016/j.renene.2024.121441.

[8] SUN X, LIU J, OU M, et al. Joint bidding decision of wind farms and energy storage based on newsvendor model[J]. Journal of Systems Science and Information, 2022, 10(1): 51-64. DOI: 10.21078/jssi-2022-051-14.

[9] GUO S, CUI X, ZHANG J, et al. Techno-economic assessment of the realisable economic value and value capture gap of probabilistic PV forecasting in PV-battery microgrid dispatch[J]. Sustainable Energy Technologies and Assessments, 2026, 93: 105319. DOI: 10.1016/j.seta.2026.105319.

[10] FUSCO A, GIOFFRÈ D, CASTELLI A F, et al. A multi-stage stochastic programming model for the unit commitment of conventional and virtual power plants bidding in the day-ahead and ancillary services markets[J]. Applied Energy, 2023, 336: 120739. DOI: 10.1016/j.apenergy.2023.120739.

[11] ZOU Y, XU Y, ZHANG C. A risk-averse adaptive stochastic optimization method for transactive energy management of a multi-energy microgrid[J]. IEEE Transactions on Sustainable Energy, 2023, 14(3): 1599-1611. DOI: 10.1109/TSTE.2023.3240184.

[12] PRADANA A, YANG F, LU J, et al. Unified robust–stochastic dispatch for reliable net-zero microgrids under PV uncertainty[J]. Applied Energy, 2026, 420: 128113. DOI: 10.1016/j.apenergy.2026.128113.

十、 AI 工具使用声明

AI 使用情况说明

本文在写作过程中使用了 AI 辅助工具，具体说明如下。

1. 使用环节：（1）赛题理解与问题分析；（2）模型假设与符号定义的参考；其余环节均未使用 AI。

2. 人工主导：AI 仅作辅助与初稿参考。问题的数学建模思路、模型选择与公式推导、参数取值与约束设定、程序代码编写、结果解释与结论判断，均由本队提出并逐项审定。

3. 核心内容核验：全部模型、公式与数值结果均由本队逐项复核；问题一至问题四的费用与电量结果均与题目附件数据及 result1.xlsx～result4-3.xlsx 重新核对一致；全部源程序由本队独立编写并经运行验证。

4. 工具清单：DeepSeek（版本 DeepSeek-V4-Flash），仅用于第 1 条所列环节；使用方式为网页对话框交互与上传赛题文件对话。所有 AI 输出均经本队审查、核实，采纳部分均经修改后使用；未被采纳的输出未进入论文。

十一、 附录

附录A支持材料文件列表

序号	文件 / 目录	内容	备注
1	参赛论文（本文件）	摘要、正文、附录	电子版另附 PDF，内容与纸质版完全一致
2	代码/	建模所用的全部完整、可运行的源程序	本论展示核心代码：1.lp_core.py,2.stoch.py,3.数据预处理.py,4.问题1求解.py,5. 问题2求解.py,6.问题3求解.py,7.问题4求解.py,8.敏感性分析.py,9.模型检验.py
3	附件/	题目原始数据	附件 1 ～ 附件 5
4	result1.xlsx ～ result4-3.xlsx	五份结果文件	按附件5模板生成，分别对应问题一、问题二、问题三、问题四-二、问题四-三
5	图片/	全部图表 PNG，共 28 张	其中21张嵌入正文

附录B核心程序代码

附录B-1 lp_core.py:

```
"""读入附件1~4，装配单日/多日调度 LP 并求解。"""
```

```
import os

import numpy as np

import pandas as pd

from scipy.optimize import linprog

from scipy.sparse import coo_matrix



DT       = 1.0 / 6.0     # 单段时长(小时), 10 分钟

N_T      = 144           # 一天的段数

N_DAY    = 365           # 附件2/3/4 覆盖天数

# 往返效率 0.9, 充放各 √0.9

ETA_C    = 0.9 ** 0.5    # 充电效率 = sqrt(0.9)

ETA_D    = 0.9 ** 0.5    # 放电效率 = sqrt(0.9)

P_MAX    = 5000.0        # 最大充放电功率 (kW)

E_MAX    = P_MAX * DT    # 单段最大充/放电电量 (kWh)

SOC_MIN  = 1200.0        # 储电量运行下界 (kWh)

SOC_MAX  = 10800.0       # 储电量运行上界 (kWh)

SOC_INIT = 6000.0        # 2025-01-01 00:00 初始储电量 (kWh)



# 末端储电量下界

SOC_TERM = 6000.0
```

```
# 吞吐量正则化：取充放电最少的解
THRU_PEN = 1e-6

BETA_EMG = 5.0           # 紧急购电电价倍数
BETA_DEF = 0.5           # 违约（少买）电价倍数
BETA_EXC = 1.5           # 超额（多买）电价倍数

# 报告期 2025-02-01 ~ 12-31
REPORT_START = pd.Timestamp("2025-02-01")
REPORT_END   = pd.Timestamp("2025-12-31")

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
ATT  = os.path.join(ROOT, "附件")
RES  = os.path.join(ROOT, "结果")
IMG  = os.path.join(ROOT, "图片")
for folder in (RES, IMG):
    os.makedirs(folder, exist_ok=True)

# 六大时段口径：每段 4 小时 = 24 段
```

```
BLOCKS = [("0:00-4:00", 0, 24), ("4:00-8:00", 24, 48), ("8:00-12:00", 48, 72),
          ("12:00-16:00", 72, 96), ("16:00-20:00", 96, 120), ("20:00-24:00", 120,
144)]




def load_all(verbose=True):
    """读入全部附件，返回各数据 dict。"""
    df1 = pd.read_excel(os.path.join(ATT, "附件1.xlsx"))
    # 显式转 float
    typical = {
        "price":   pd.to_numeric(df1["电价"], errors="raise").to_numpy(float),
        "load_kw": pd.to_numeric(df1["小区负载"], errors="raise").to_numpy(float),
        "pv_kw":   pd.to_numeric(df1["光伏发电预测功率"],
errors="raise").to_numpy(float),
    }


    xl = pd.ExcelFile(os.path.join(ATT, "附件2.xlsx"))
    ld = xl.parse("小区负载", header=0)
    pv = xl.parse("光伏发电实际功率", header=0)
    dates = pd.to_datetime(ld.iloc[:, 0]).reset_index(drop=True)
    actual = {
```

```
        "dates":   dates,
        "load_kw": ld.iloc[:, 1:1 + N_T].apply(pd.to_numeric,
errors="raise").to_numpy(float),
        # 光伏负值截零（夜间浮点噪声）
        "pv_kw":   np.clip(pv.iloc[:, 1:1 + N_T].apply(pd.to_numeric,
errors="raise")
                           .to_numpy(float), 0.0, None),
    }

    df4 = pd.read_excel(os.path.join(ATT, "附件4.xlsx"))
    price_rt = {
        "dates": pd.to_datetime(df4.iloc[:, 0]).reset_index(drop=True),
        "price": df4.iloc[:, 1:1 + N_T].apply(pd.to_numeric,
errors="raise").to_numpy(float),
    }

    # 附件3：全年光伏预报 365x4x24, 第2维为发布时刻
    df3 = pd.read_excel(os.path.join(ATT, "附件3.xlsx"))
    date_col = df3.iloc[:, 0].ffill()      # 日期列向下填充
    time_col = df3.iloc[:, 1].astype(str).str.strip()
    stamps = pd.to_datetime(date_col.astype(str).str.strip(), format="mixed",
errors="coerce")
```

```
    days = pd.DatetimeIndex(sorted(stamps.dropna().unique()))

    day_index = {day: i for i, day in enumerate(days)}

    fc_3d = np.full((N_DAY, 4, 24), np.nan)

    for row in range(len(df3)):

        day = stamps.iloc[row]

        if pd.isna(day):

            continue

        label = time_col.iloc[row]

        if label in ("0:00", "6:00", "12:00", "18:00"):

            fc_3d[day_index[day], ("0:00", "6:00", "12:00",
"18:00").index(label), :] = \

                pd.to_numeric(df3.iloc[row, 2:26], errors="coerce").to_numpy(float)

    forecast = {"dates": days, "fc": fc_3d}



    if verbose:

        print("典型日", typical["price"].shape, "附件2", actual["load_kw"].shape,

              "附件3", fc_3d.shape, "附件4", price_rt["price"].shape)

    return {"typical": typical, "actual": actual, "forecast": forecast, "price_rt":
price_rt}
```

```
def forecast_to_10min(fc_day, k, method="step"):
    """第 k 个发布时刻 24 点预报还原为 144 段 10 分钟。"""
    base = (0, 6, 12, 18)[k]
    vals = np.asarray(fc_day[k], float)
    hourly = np.zeros(24)
    # j = h + 1 - base（时间戳为区间终点）
    for h in range(24):
        j = (h + 1 - base) % 24
        if j == 0:
            j = 24            # 发布时刻小时取"预报24小时"
        hourly[h] = vals[j - 1]
    if method == "step":
        # 段 t 覆盖 ((t-1)/6, t/6]，整点取左端点
        idx = np.floor(np.arange(N_T) * DT).astype(int)
        return hourly[np.clip(idx, 0, 23)]
    elif method == "linear":
        grid = np.arange(1, N_T + 1) * DT
        xp = np.concatenate([[0.0], np.arange(1, 24).astype(float), [24.0]])
        fp = np.concatenate([[hourly[0]], hourly[1:24], [hourly[0]]])
        return np.interp(grid, xp, fp)
```

```
    raise ValueError("method 只能是 'step' 或 'linear'")




def solve_day_lp(price, load_kw, pv_kw, soc_start=SOC_INIT, soc_end=None,
                 use_grid=True, verbose=False, c_cyc=0.0):
    """单日调度 LP：未知量分 5 块 g,c,d,s,E。"""
    n = N_T
    # 目标系数：购电项 + 循环成本
    c_obj = np.concatenate([price, np.zeros(4 * n)])
    if c_cyc:
        c_obj[2 * n:3 * n] = c_cyc        # 第 3 个变量块 = 放电量 d_t


    rows, cols, vals, b_eq = [], [], [], []


    # 母线平衡 g_t + pv*dt + d_t = L*dt + c_t + s_t
    for t in range(n):
        r = t
        rows += [r, r, r, r]; cols += [t, n + t, 2 * n + t, 3 * n + t]
        vals += [1.0, -1.0, 1.0, -1.0]
        b_eq.append(load_kw[t] * DT - pv_kw[t] * DT)
```

```
# SOC 递推 E_t = E_{t-1} + eta_c*c_t - d_t/eta_d
for t in range(n):
    r = n + t
    rows.append(r); cols.append(4 * n + t); vals.append(1.0)
    if t == 0:
        b_eq.append(soc_start)        # E_0 - eta_c*c_0 + d_0/eta_d = E_init
    else:
        rows.append(r); cols.append(4 * n + t - 1); vals.append(-1.0)
        b_eq.append(0.0)
    rows.append(r); cols.append(n + t); vals.append(-ETA_C)
    rows.append(r); cols.append(2 * n + t); vals.append(1.0 / ETA_D)

A_eq = coo_matrix((vals, (rows, cols)), shape=(2 * n, 5 * n)).tocsr()

lb = np.zeros(5 * n)
ub = np.empty(5 * n)
ub[0:n]         = np.inf                  # 购电量只买不卖
ub[n:2 * n]     = E_MAX                   # 充电功率折算上限
ub[2 * n:3 * n] = E_MAX                   # 放电功率折算上限
```

```
    ub[3 * n:4 * n] = pv_kw * DT                # 弃光不得超过光伏发电量
    ub[4 * n:5 * n] = SOC_MAX
    lb[4 * n:5 * n] = SOC_MIN
    if soc_end is not None:
        # 末端储电量收紧上下界
        ub[4 * n + n - 1] = soc_end
        lb[4 * n + n - 1] = soc_end

    bounds = list(zip(lb, ub))

    res = linprog(c_obj, A_eq=A_eq, b_eq=np.array(b_eq), bounds=bounds,
                  method="highs")         # HiGHS 求解器

    if not res.success:
        return {"status": res.status, "message": res.message}

    x = res.x
    out = {
        "status": res.status, "message": res.message, "cost": float(res.fun),
        "g": x[0:n].copy(), "c": x[n:2 * n].copy(), "d": x[2 * n:3 * n].copy(),
```

```
        "s": x[3 * n:4 * n].copy(), "E": x[4 * n:5 * n].copy(),
        # 平衡约束对偶：边际供电成本
        "dual_balance": (res.eqlin.marginals[:n].copy()
                         if getattr(res, "eqlin", None) is not None else None),
    }
    if verbose:
        print("单日LP", "status", res.status, "费用", "%.2f" % out["cost"],
              "购电量", "%.1f" % out["g"].sum(), "弃光", "%.1f" % out["s"].sum())
    return out



def block_sum(arr, lo, hi):
    """按 4 小时区块求和（result 模板口径）。"""
    return sum(arr[lo:hi] for (_, lo, hi) in BLOCKS)



def block_sums(arr):
    """6 个 4 小时区块的合计（元组）。"""
    return [float(arr[lo:hi].sum()) for (_, lo, hi) in BLOCKS]
```

```
def solve_multiday_lp(price_2d, load_2d, pv_2d, soc_init=SOC_INIT,
                      use_emg=True, n_days=None, verbose=True, c_cyc=0.0):
    """多日耦合调度 LP：展平 i=d*144+t，返回 (D,144)。"""
    n_day = price_2d.shape[0] if n_days is None else n_days
    price = price_2d[:n_day].ravel()
    load = load_2d[:n_day].ravel()
    pv = pv_2d[:n_day].ravel()
    n_seg = n_day * N_T
    n_blk = 6 if use_emg else 5       # 变量块数：g,c,d,s,E(,e)

    off_g, off_c, off_d, off_s, off_e = 0, n_seg, 2 * n_seg, 3 * n_seg, 4 * n_seg
    off_e2 = 5 * n_seg if use_emg else None

    c_obj = np.zeros(n_blk * n_seg)
    c_obj[off_g:off_g + n_seg] = price
    if use_emg:
        c_obj[off_e2:off_e2 + n_seg] = BETA_EMG * price
    if c_cyc:
        # 循环成本按放电量计提
```

```
        c_obj[off_d:off_d + n_seg] = c_cyc



    idx = np.arange(n_seg)

    rows, cols, vals, b_eq = [], [], [], []



    # 母线平衡，n_seg 条等式

    rows.append(idx);            cols.append(off_g + idx);
vals.append(np.ones(n_seg))

    rows.append(idx);            cols.append(off_c + idx); vals.append(-
np.ones(n_seg))

    rows.append(idx);            cols.append(off_d + idx);
vals.append(np.ones(n_seg))

    rows.append(idx);            cols.append(off_s + idx); vals.append(-
np.ones(n_seg))

    if use_emg:

        rows.append(idx);        cols.append(off_e2 + idx);
vals.append(np.ones(n_seg))

    b_eq.append(load * DT - pv * DT)



    # SOC 递推：n_seg 条等式，初始值在右端项

    soc_r = n_seg + idx

    rows.append(soc_r);      cols.append(off_e + idx); vals.append(np.ones(n_seg))
```

```
    rows.append(soc_r);      cols.append(off_c + idx); vals.append(-ETA_C *
np.ones(n_seg))

    rows.append(soc_r);      cols.append(off_d + idx); vals.append((1.0 / ETA_D) *
np.ones(n_seg))

    rows.append(soc_r[1:]);  cols.append(off_e + idx[1:] - 1); vals.append(-
np.ones(n_seg - 1))

    b_eq.append(np.zeros(n_seg)); b_eq[-1][0] = soc_init



    A_eq = coo_matrix((np.concatenate(vals), (np.concatenate(rows),
np.concatenate(cols))),

                      shape=(2 * n_seg, n_blk * n_seg)).tocsr()

    b_vec = np.concatenate(b_eq)



    lb = np.zeros(n_blk * n_seg); ub = np.empty(n_blk * n_seg)

    ub[off_g:off_g + n_seg]    = np.inf

    ub[off_c:off_c + n_seg]    = E_MAX

    ub[off_d:off_d + n_seg]    = E_MAX

    ub[off_s:off_s + n_seg]    = pv * DT

    lb[off_e:off_e + n_seg]    = SOC_MIN

    ub[off_e:off_e + n_seg]    = SOC_MAX

    if use_emg:

        ub[off_e2:off_e2 + n_seg] = np.inf
```

```
    if verbose:
        print("多日LP", "天数", n_day, "变量", n_blk * n_seg, "等式约束", 2 *
n_seg,
              "非零元", A_eq.nnz)
    res = linprog(c_obj, A_eq=A_eq, b_eq=b_vec, bounds=list(zip(lb, ub)),
method="highs")

    if not res.success:
        return {"status": res.status, "message": res.message}

    x = res.x
    out = {
        "status": res.status, "message": res.message,
        "g": x[off_g:off_g + n_seg].reshape(n_day, N_T).copy(),
        "e": (x[off_e2:off_e2 + n_seg].reshape(n_day, N_T).copy()
              if use_emg else np.zeros((n_day, N_T))),
        "c": x[off_c:off_c + n_seg].reshape(n_day, N_T).copy(),
        "d": x[off_d:off_d + n_seg].reshape(n_day, N_T).copy(),
        "s": x[off_s:off_s + n_seg].reshape(n_day, N_T).copy(),
        "E": x[off_e:off_e + n_seg].reshape(n_day, N_T).copy(),
```

```
    }

    out["cost_plan"] = float((price.reshape(n_day, N_T) * out["g"]).sum())

    out["cost_emg"]  = float(BETA_EMG * (price.reshape(n_day, N_T) *
out["e"]).sum())

    out["cost"]      = out["cost_plan"] + out["cost_emg"]

    if verbose:

        print("多日LP", "status", res.status, "总费用", "%.2f" % out["cost"],

              "计划", "%.2f" % out["cost_plan"], "紧急", "%.2f" % out["cost_emg"],

              "紧急购电", "%.6f" % out["e"].sum())

    return out




def soc_day_boundary(E_2d):

    """由 (D,144) 逐段储电量提取每日首末值。"""

    n_day = E_2d.shape[0]

    soc0 = np.empty(n_day); soc24 = E_2d[:, -1].copy()

    soc0[0] = SOC_INIT

    soc0[1:] = E_2d[:-1, -1]

    return soc0, soc24
```

```
N_T_ = N_T
_DECIDE_T = {0: 0, 1: 36, 2: 72, 3: 108}   # 6:00->36段, 12:00->72段, 18:00->108段
_STAGE_CACHE = {}



def _stage_structure(n=N_T):
    """阶段 LP 稀疏结构，缓存复用。"""
    if "A" in _STAGE_CACHE:
        return _STAGE_CACHE["I"], _STAGE_CACHE["A_eq"], _STAGE_CACHE["A_ub"]
    blk = {k: i * n for i, k in enumerate(["GD", "C", "D", "S", "E", "E2", "GA",
"U", "V"])}
    idx = np.arange(n)
    eq_r, eq_c, eq_v = [], [], []
    for name, sign in [("GD", 1.0), ("E2", 1.0), ("D", 1.0), ("C", -1.0), ("S", -
1.0)]:
        eq_r.append(idx); eq_c.append(blk[name] + idx); eq_v.append(sign *
np.ones(n))
    soc_r = n + idx
    eq_r.append(soc_r); eq_c.append(blk["E"] + idx); eq_v.append(np.ones(n))
    eq_r.append(soc_r); eq_c.append(blk["C"] + idx); eq_v.append(-ETA_C *
np.ones(n))
    eq_r.append(soc_r); eq_c.append(blk["D"] + idx); eq_v.append((1.0 / ETA_D) *
```

```
np.ones(n))

    eq_r.append(soc_r[1:]); eq_c.append(blk["E"] + idx[1:] - 1); eq_v.append(-
np.ones(n - 1))

    A_eq = coo_matrix((np.concatenate(eq_v), (np.concatenate(eq_r),
np.concatenate(eq_c))),

                      shape=(2 * n, 9 * n)).tocsr()

    u_r, u_c, u_v = [], [], []

    u_r.append(idx); u_c.append(blk["GD"] + idx); u_v.append(np.ones(n))

    u_r.append(idx); u_c.append(blk["GA"] + idx); u_v.append(-np.ones(n))

    r_u = n + idx

    u_r.append(r_u); u_c.append(blk["GA"] + idx); u_v.append(-np.ones(n))

    u_r.append(r_u); u_c.append(blk["U"] + idx);  u_v.append(-np.ones(n))

    r_v = 2 * n + idx

    u_r.append(r_v); u_c.append(blk["GA"] + idx); u_v.append(np.ones(n))

    u_r.append(r_v); u_c.append(blk["V"] + idx);  u_v.append(-np.ones(n))

    A_ub = coo_matrix((np.concatenate(u_v), (np.concatenate(u_r),
np.concatenate(u_c))),

                      shape=(3 * n, 9 * n)).tocsr()

    _STAGE_CACHE.update({"I": blk, "A_eq": A_eq, "A_ub": A_ub})

    return blk, A_eq, A_ub
```

```
_STAGE2_CACHE = {}




def _stage2_structure(t_decide):
    """按 t_decide 缓存的阶段 LP 稀疏结构。"""
    if t_decide in _STAGE2_CACHE:
        return _STAGE2_CACHE[t_decide]
    m = N_T - t_decide
    blk = {k: i * m for i, k in enumerate(["GA", "C", "D", "S", "E", "E2", "U",
"V"])}
    idx = np.arange(m)
    rows, cols, vals = [], [], []
    for name, sign in [("GA", 1.0), ("E2", 1.0), ("D", 1.0), ("C", -1.0), ("S", -
1.0)]:
        rows.append(idx); cols.append(blk[name] + idx); vals.append(sign *
np.ones(m))
    soc_r = m + idx
    rows.append(soc_r); cols.append(blk["E"] + idx); vals.append(np.ones(m))
    rows.append(soc_r); cols.append(blk["C"] + idx); vals.append(-ETA_C *
np.ones(m))
    rows.append(soc_r); cols.append(blk["D"] + idx); vals.append((1.0 / ETA_D) *
```

```
np.ones(m))
    rows.append(soc_r[1:]); cols.append(blk["E"] + idx[1:] - 1); vals.append(-
np.ones(m - 1))
    A_eq = coo_matrix((np.concatenate(vals), (np.concatenate(rows),
np.concatenate(cols))),
                      shape=(2 * m, 8 * m)).tocsr()
    # U/V 两条约束分占不同行号
    ru, cu, vu = [], [], []
    ru.append(idx); cu.append(blk["GA"] + idx); vu.append(-np.ones(m))
    ru.append(idx); cu.append(blk["U"] + idx); vu.append(-np.ones(m))
    rv = m + idx
    ru.append(rv); cu.append(blk["GA"] + idx); vu.append(np.ones(m))
    ru.append(rv); cu.append(blk["V"] + idx); vu.append(-np.ones(m))
    A_ub = coo_matrix((np.concatenate(vu), (np.concatenate(ru),
np.concatenate(cu))),
                      shape=(2 * m, 8 * m)).tocsr()
    _STAGE2_CACHE[t_decide] = (blk, A_eq, A_ub)
    return blk, A_eq, A_ub



def solve_stage_lp(price, load_forecast, pv_forecast, g_plan, t_decide, soc_at):
```

```
    """阶段 LP：只重优化 [t_decide,144)。"""
    blk, A_eq, A_ub = _stage2_structure(t_decide)
    m = N_T - t_decide
    t = np.arange(t_decide, N_T)
    idx = np.arange(m)
    c_obj = np.zeros(8 * m)
    c_obj[blk["GA"] + idx] = BETA_DEF * price[t]
    c_obj[blk["V"] + idx] = (BETA_EXC - BETA_DEF) * price[t]
    c_obj[blk["E2"] + idx] = BETA_EMG * price[t]
    b_eq = np.concatenate([load_forecast[t] * DT - pv_forecast[t] * DT,
                           np.r_[soc_at, np.zeros(m - 1)]])
    b_ub = np.concatenate([-g_plan[t], g_plan[t]])
    lb = np.zeros(8 * m); ub = np.empty(8 * m)
    ub[blk["GA"]:blk["GA"] + m] = np.inf
    ub[blk["C"]:blk["C"] + m] = E_MAX
    ub[blk["D"]:blk["D"] + m] = E_MAX
    ub[blk["S"]:blk["S"] + m] = pv_forecast[t] * DT
    lb[blk["E"]:blk["E"] + m] = SOC_MIN; ub[blk["E"]:blk["E"] + m] = SOC_MAX
    ub[blk["E2"]:blk["E2"] + m] = np.inf
    ub[blk["U"]:blk["U"] + m] = np.inf
```

```
    ub[blk["V"]:blk["V"] + m] = np.inf
    res = linprog(c_obj, A_eq=A_eq, b_eq=b_eq, A_ub=A_ub, b_ub=b_ub,
                  bounds=list(zip(lb, ub)), method="highs")
    return res, blk, m




def forecast_bias_table(fc10_all, pv_2d):
    """扩张窗口偏差表 BIAS[d,k,t]；pv 校正用。"""
    n_day, n_ver, n_seg = fc10_all.shape
    bias = np.zeros((n_day, n_ver, n_seg))
    for k in range(n_ver):
        cum_bias = np.zeros(n_seg); count = 0
        for day in range(n_day):
            if count > 0:
                bias[day, k] = cum_bias / count
            fc_row = fc10_all[day, k]
            if not np.isnan(fc_row).any():
                cum_bias = cum_bias + (fc_row - pv_2d[day]); count += 1
    return bias
```

```
def _settle_structure(n=N_T):
    """实际结算 LP 稀疏结构，缓存复用。"""
    if "AS" in _STAGE_CACHE:
        return _STAGE_CACHE["IS"], _STAGE_CACHE["AS_eq"], _STAGE_CACHE["AS_ub"]
    blk = {k: i * n for i, k in enumerate(["GD", "C", "D", "S", "E", "E2"])}
    idx = np.arange(n)
    eq_r, eq_c, eq_v = [], [], []
    for name, sign in [("GD", 1.0), ("E2", 1.0), ("D", 1.0), ("C", -1.0), ("S", -
1.0)]:
        eq_r.append(idx); eq_c.append(blk[name] + idx); eq_v.append(sign *
np.ones(n))
    soc_r = n + idx
    eq_r.append(soc_r); eq_c.append(blk["E"] + idx); eq_v.append(np.ones(n))
    eq_r.append(soc_r); eq_c.append(blk["C"] + idx); eq_v.append(-ETA_C *
np.ones(n))
    eq_r.append(soc_r); eq_c.append(blk["D"] + idx); eq_v.append((1.0 / ETA_D) *
np.ones(n))
    eq_r.append(soc_r[1:]); eq_c.append(blk["E"] + idx[1:] - 1); eq_v.append(-
np.ones(n - 1))
    A_eq = coo_matrix((np.concatenate(eq_v), (np.concatenate(eq_r),
np.concatenate(eq_c))),
```

```
                      shape=(2 * n, 6 * n)).tocsr()
    # 结算购电量不超过已承诺量
    A_ub = coo_matrix((np.concatenate([np.ones(n), -np.ones(n)]),
                       (np.concatenate([idx, idx]),
                        np.concatenate([blk["GD"] + idx,
                                        blk["GA"] if False else np.zeros(n,
dtype=int)]))),
                      shape=(n, 6 * n)).tocsr()
    one_rows = list(range(n)); gd_cols = list(blk["GD"] + idx); one_vals = [1.0] *
n
    A_ub = coo_matrix((np.array(one_vals), (np.array(one_rows),
np.array(gd_cols))),
                      shape=(n, 6 * n)).tocsr()
    _STAGE_CACHE.update({"IS": blk, "AS_eq": A_eq, "AS_ub": A_ub})
    return blk, A_eq, A_ub



def settle_actual_lp(price, load_kw, pv_actual, committed, soc_start):
    """按当日实际光伏与负载做最终结算。"""
    n = N_T
    blk, A_eq, A_ub = _settle_structure(n)
```

```
    idx = np.arange(n)
    c_obj = np.zeros(6 * n)
    c_obj[blk["E2"] + idx] = BETA_EMG * price
    # 吞吐量正则化：取充放电最少的解
    c_obj[blk["C"] + idx] = THRU_PEN
    c_obj[blk["D"] + idx] = THRU_PEN
    b_eq = np.concatenate([load_kw * DT - pv_actual * DT,
                           np.r_[soc_start, np.zeros(n - 1)]])
    b_ub = committed.copy()
    lb = np.zeros(6 * n); ub = np.empty(6 * n)
    ub[blk["GD"]:blk["GD"] + n] = committed
    ub[blk["C"]:blk["C"] + n] = E_MAX
    ub[blk["D"]:blk["D"] + n] = E_MAX
    ub[blk["S"]:blk["S"] + n] = pv_actual * DT
    lb[blk["E"]:blk["E"] + n] = SOC_MIN; ub[blk["E"]:blk["E"] + n] = SOC_MAX
    ub[blk["E2"]:blk["E2"] + n] = np.inf
    return linprog(c_obj, A_eq=A_eq, b_eq=b_eq, A_ub=A_ub, b_ub=b_ub,
                   bounds=list(zip(lb, ub)), method="highs")
```

```
def run_rolling(price_2d, load_2d, pv_2d, fc10, active_stages=(1, 2, 3),
                soc_init=SOC_INIT, verbose=False):
    """全年逐日滚动求解（问3 主模型）。"""
    n_day = price_2d.shape[0]
    g_plan = np.zeros((n_day, N_T)); g_adj = np.zeros((n_day, N_T))
    charge = np.zeros((n_day, N_T)); discharge = np.zeros((n_day, N_T))
    curtail = np.zeros((n_day, N_T)); soc_all = np.zeros((n_day, N_T))
    emg = np.zeros((n_day, N_T))
    soc = soc_init
    for day in range(n_day):
        plan = solve_day_lp(price_2d[day], load_2d[day], fc10[day, 0],
soc_start=soc, soc_end=None)
        if plan["status"] != 0:
            raise RuntimeError("第 %d 天计划阶段失败" % day)
        g_plan[day] = plan["g"]
        committed = g_plan[day].copy()
        for k in active_stages:
            t_dec = _DECIDE_T[k]
            # t_dec 时刻的真实储电量作为阶段初值
            settle0 = settle_actual_lp(price_2d[day], load_2d[day], pv_2d[day],
committed, soc)
```

```
            blk = _STAGE_CACHE["IS"]

            soc_at = float(settle0.x[blk["E"] + t_dec - 1]) if settle0.success else
soc

            stage, stage_blk, m_stage = solve_stage_lp(price_2d[day], load_2d[day],
fc10[day, k],

                                                       g_plan[day], t_dec, soc_at)

            if not stage.success:

                raise RuntimeError("第 %d 天阶段%d 失败" % (day, k))

            committed[t_dec:] = stage.x[stage_blk["GA"]:stage_blk["GA"] + m_stage]

        g_adj[day] = committed

        settle = settle_actual_lp(price_2d[day], load_2d[day], pv_2d[day],
committed, soc)

        if not settle.success:

            raise RuntimeError("第 %d 天实际结算失败" % day)

        y = settle.x

        blk = _STAGE_CACHE["IS"]

        charge[day] = y[blk["C"]:blk["C"] + N_T]; discharge[day] =
y[blk["D"]:blk["D"] + N_T]

        curtail[day] = y[blk["S"]:blk["S"] + N_T]; soc_all[day] =
y[blk["E"]:blk["E"] + N_T]

        emg[day] = y[blk["E2"]:blk["E2"] + N_T]

        soc = soc_all[day, -1]
```

```
        if verbose and (day + 1) % 90 == 0:
            print("已完成 %d/%d 天" % (day + 1, n_day))
    return {"g_plan": g_plan, "g_adj": g_adj, "c": charge, "d": discharge,
            "s": curtail, "E": soc_all, "e": emg}



def cost_breakdown(price_2d, gp, ga, ee):
    """按 Reading A 口径核算四项费用。"""
    short_kwh = np.maximum(gp - ga, 0.0)
    over_kwh = np.maximum(ga - gp, 0.0)
    return {
        "base": float((price_2d * np.minimum(gp, ga)).sum()),
        "def":  float((price_2d * BETA_DEF * short_kwh).sum()),
        "exc":  float((price_2d * BETA_EXC * over_kwh).sum()),
        "emg":  float(BETA_EMG * (price_2d * ee).sum()),
        "u_kwh": float(short_kwh.sum()), "v_kwh": float(over_kwh.sum()), "e_kwh":
float(ee.sum()),
    }
```

附录B-2 stoch.py

```
"""随机优化模块：点预测、情景生成与两阶段随机规划。"""
```

```
import numpy as np

import pandas as pd

from scipy.optimize import linprog

from scipy.sparse import coo_matrix



DT = 1.0 / 6.0

N_T = 144







def day_type(dates):

    """日型分类：周五、周六为 0，其余为 1。"""

    dow = pd.DatetimeIndex(dates).dayofweek        # 周一=0 ... 周日=6

    return np.where((dow == 4) | (dow == 5), 0, 1)







def point_forecast(hist_2d, dtypes, d, k=8, decay=0.85):

    """相似日加权点预测：最近 k 天，按距离指数衰减。"""

    cands = [j for j in range(d) if dtypes[j] == dtypes[d]]

    if not cands:

        cands = list(range(d))
```

```
    cands = cands[-k:]
    weights = np.array([decay ** (len(cands) - 1 - i) for i in range(len(cands))])
    weights = weights / weights.sum()
    return np.tensordot(weights, hist_2d[cands], axes=(0, 0))



def residual_pool(actual_2d, dtypes, d, k=40, eps=0.0):
    """相对残差池：同型日 实际/预测 的逐段比值，eps 平滑。"""
    pool = []
    if d == 0:                          # 无历史时返回全 1 曲线
        return np.ones((1, actual_2d.shape[1]))
    cands = [j for j in range(d) if dtypes[j] == dtypes[d]]
    if len(cands) < 5:
        cands = list(range(d))
    for j in cands[-k:]:
        fc = point_forecast(actual_2d, dtypes, j, k=8)
        # 分母加极小正数
        ratio = (actual_2d[j] + eps) / np.maximum(fc + eps, 1e-9)
        pool.append(np.clip(ratio, 0.2, 2.5))
    return np.array(pool)
```

```
def sample_scenarios(pool, n_scen, rng):
    """整日成块抽取情景。"""
    idx = rng.integers(0, len(pool), size=n_scen)
    return pool[idx]



def solve_two_stage(price, load_fc, pv_fc, load_scen, pv_scen,
                    soc_start, soc_term, beta_emg=5.0, n_t=N_T, c_cyc=0.0):
    """两阶段随机规划：阶段一定计划购电量，阶段二情景内调度。"""
    n_scen = load_scen.shape[0]
    n = n_t
    # 变量分块：g_plan(n) + 每情景 6 块 (g_draw,c,d,s,E,e)
    n_var = n + n_scen * 6 * n
    cost = np.zeros(n_var)
    cost[0:n] = price                                          # 第一阶段：计划购电费用
    for w in range(n_scen):
        base = n + w * 6 * n
        cost[base + 5 * n: base + 6 * n] = (beta_emg / n_scen) * price
```

```
        if c_cyc:
            # 循环成本按第二阶段放电量期望计提
            cost[base + 2 * n: base + 3 * n] = c_cyc / n_scen

    rows, cols, vals, b_eq = [], [], [], []
    rows_u, cols_u, vals_u, b_ub = [], [], [], []
    idx = np.arange(n)
    for w in range(n_scen):
        base = n + w * 6 * n
        off_gd, off_c, off_d, off_s, off_e, off_emg = (base + 0 * n, base + 1 * n,
                                                       base + 2 * n, base + 3 * n,
                                                       base + 4 * n, base + 5 * n)
        row = w * 2 * n + idx
        for off, co in [(off_gd, 1.0), (off_emg, 1.0), (off_d, 1.0), (off_c, -1.0),
(off_s, -1.0)]:
            rows.append(row); cols.append(off + idx); vals.append(co * np.ones(n))
        b_eq.append(load_scen[w] * DT - pv_scen[w] * DT)
        row_soc = w * 2 * n + n + idx
        rows.append(row_soc); cols.append(off_e + idx); vals.append(np.ones(n))
        rows.append(row_soc); cols.append(off_c + idx)
        vals.append(-__import__("lp_core").ETA_C * np.ones(n))
```

```
        rows.append(row_soc); cols.append(off_d + idx)

        vals.append((1.0 / __import__("lp_core").ETA_D) * np.ones(n))

        rows.append(row_soc[1:]); cols.append(off_e + idx[1:] - 1); vals.append(-
np.ones(n - 1))

        rhs = np.zeros(n); rhs[0] = soc_start

        b_eq.append(rhs)

        rows_u.append(w * n + idx); cols_u.append(off_gd + idx);
vals_u.append(np.ones(n))

        rows_u.append(w * n + idx); cols_u.append(idx); vals_u.append(-np.ones(n))

        b_ub.append(np.zeros(n))



    A_eq = coo_matrix((np.concatenate(vals), (np.concatenate(rows),
np.concatenate(cols))),

                      shape=(2 * n_scen * n, n_var)).tocsr()

    A_ub = coo_matrix((np.concatenate(vals_u), (np.concatenate(rows_u),
np.concatenate(cols_u))),

                      shape=(n_scen * n, n_var)).tocsr()



    import lp_core as L

    lb = np.zeros(n_var); ub = np.empty(n_var)

    ub[0:n] = np.inf

    for w in range(n_scen):
```

```
        base = n + w * 6 * n
        ub[base + 0 * n: base + 1 * n] = np.inf
        ub[base + 1 * n: base + 2 * n] = L.E_MAX
        ub[base + 2 * n: base + 3 * n] = L.E_MAX
        ub[base + 3 * n: base + 4 * n] = pv_scen[w] * DT
        lb[base + 4 * n: base + 5 * n] = L.SOC_MIN
        ub[base + 4 * n: base + 5 * n] = L.SOC_MAX
        ub[base + 5 * n: base + 6 * n] = np.inf
    res = linprog(cost, A_eq=A_eq, b_eq=np.concatenate(b_eq),
                  A_ub=A_ub, b_ub=np.concatenate(b_ub),
                  bounds=list(zip(lb, ub)), method="highs")
    if not res.success:
        return None
    # d_mean：情景平均放电量（kWh）
    sol = res.x
    d_sum = np.zeros(n)
    for w in range(n_scen):
        base = n + w * 6 * n
        d_sum += sol[base + 2 * n: base + 3 * n]
    return {"g_plan": res.x[0:n].copy(), "obj": float(res.fun), "x": res.x, "S":
n_scen, "n": n,
```

```
            "d_mean": d_sum / n_scen}



def extract_scenario(res, w):
    """取第 w 个情景的第二阶段结果。"""
    n, n_scen = res["n"], res["S"]
    base = n + w * 6 * n
    x = res["x"]
    return {"g_draw": x[base + 0 * n: base + 1 * n],
            "c": x[base + 1 * n: base + 2 * n],
            "d": x[base + 2 * n: base + 3 * n],
            "s": x[base + 3 * n: base + 4 * n],
            "E": x[base + 4 * n: base + 5 * n],
            "e": x[base + 5 * n: base + 6 * n]}



def advance_window(price, g_adj, load_act, pv_act, soc_start, i0, i1):
    """已发生窗口 [i0,i1) 的确定性调度，返回末端储能。"""
    import lp_core as L
    m = i1 - i0
```

```
    if m <= 0:

        return soc_start, np.zeros(0), np.zeros(0), np.zeros(0), np.zeros(0)

    idx = np.arange(m)

    # g_draw 松弛量：计划量是计费量而非取用量

    blocks = {k: i * m for i, k in enumerate(["GD", "C", "D", "S", "E", "E2"])}

    cost = np.zeros(6 * m)

    cost[blocks["E2"] + idx] = L.BETA_EMG * price[i0:i1]

    cost[blocks["C"] + idx] = L.THRU_PEN

    cost[blocks["D"] + idx] = L.THRU_PEN

    eq_r, eq_c, eq_v, b_eq = [], [], [], []

    for name, co in [("GD", 1.0), ("E2", 1.0), ("D", 1.0), ("C", -1.0), ("S", -
1.0)]:

        eq_r.append(idx); eq_c.append(blocks[name] + idx); eq_v.append(co *
np.ones(m))

    b_eq.append(load_act[i0:i1] * L.DT - pv_act[i0:i1] * L.DT)

    row_soc = m + idx

    eq_r.append(row_soc); eq_c.append(blocks["E"] + idx); eq_v.append(np.ones(m))

    eq_r.append(row_soc); eq_c.append(blocks["C"] + idx); eq_v.append(-L.ETA_C *
np.ones(m))

    eq_r.append(row_soc); eq_c.append(blocks["D"] + idx); eq_v.append((1.0 /
L.ETA_D) * np.ones(m))

    eq_r.append(row_soc[1:]); eq_c.append(blocks["E"] + idx[1:] - 1); eq_v.append(-
```

```
np.ones(m - 1))
    rhs = np.zeros(m); rhs[0] = soc_start
    b_eq.append(rhs)
    A_eq = coo_matrix((np.concatenate(eq_v), (np.concatenate(eq_r),
np.concatenate(eq_c))),
                      shape=(2 * m, 6 * m)).tocsr()
    lb = np.zeros(6 * m); ub = np.empty(6 * m)
    ub[blocks["GD"]:blocks["GD"] + m] = g_adj[i0:i1]      # 抽取量不超过计划量
    ub[blocks["C"]:blocks["C"] + m] = L.E_MAX
    ub[blocks["D"]:blocks["D"] + m] = L.E_MAX
    ub[blocks["S"]:blocks["S"] + m] = pv_act[i0:i1] * L.DT
    lb[blocks["E"]:blocks["E"] + m] = L.SOC_MIN
    ub[blocks["E"]:blocks["E"] + m] = L.SOC_MAX
    ub[blocks["E2"]:blocks["E2"] + m] = np.inf
    res = linprog(cost, A_eq=A_eq, b_eq=np.concatenate(b_eq),
                  bounds=list(zip(lb, ub)), method="highs")
    if not res.success:
        return soc_start, np.zeros(m), np.zeros(m), np.zeros(m), np.zeros(m)
    x = res.x
    return (x[blocks["E"] + m - 1],
            x[blocks["C"]:blocks["C"] + m], x[blocks["D"]:blocks["D"] + m],
```

```
            x[blocks["S"]:blocks["S"] + m], x[blocks["E2"]:blocks["E2"] + m])



def solve_stage_scen(price, g_plan, g_adj_prev, tdec, soc_start,
                     load_fc, pv_fc, load_scen, pv_scen, soc_term=None,
                     beta_def=None, beta_exc=None, beta_emg=None):
    """问题三情景化调整阶段 LP：未来时段用历史误差生成情景。"""
    import lp_core as L
    if beta_def is None: beta_def = L.BETA_DEF
    if beta_exc is None: beta_exc = L.BETA_EXC
    if beta_emg is None: beta_emg = L.BETA_EMG
    m = N_T - tdec
    if m <= 0:
        return None
    n_scen = load_scen.shape[0]
    # 变量分块：g_adj | u | v | 每情景 6 块
    n_adj, n_uv = m, 2 * m
    base_scen = n_adj + n_uv
    n_var = base_scen + n_scen * 6 * m
    idx = np.arange(m)
```

```
    price_f = price[tdec:]

    g_plan_f = g_plan[tdec:]



    cost = np.zeros(n_var)

    # 目标 pi*g_adj+(beta_exc-1)*pi*v；GA/U/V 顺序固定

    cost[0:m] = price_f

    cost[n_adj: n_adj + m] = beta_def * price_f

    cost[n_adj + m: n_adj + 2 * m] = (beta_exc - 1.0) * price_f



    rows, cols, vals, b_eq = [], [], [], []

    rows_u, cols_u, vals_u, b_ub = [], [], [], []

    # u、v 两条约束分占不同行号

    rows_u.append(idx); cols_u.append(0 + idx); vals_u.append(-np.ones(m))

    rows_u.append(idx); cols_u.append(n_adj + idx); vals_u.append(-np.ones(m))

    b_ub.append(-g_plan_f)

    rows_u.append(m + idx); cols_u.append(0 + idx); vals_u.append(np.ones(m))

    rows_u.append(m + idx); cols_u.append(n_adj + 2 * m + idx); vals_u.append(-
np.ones(m))

    b_ub.append(g_plan_f)



    for w in range(n_scen):
```

```
        base = base_scen + w * 6 * m
        off_gd, off_c, off_d, off_s, off_e, off_emg = (base, base + m, base + 2 *
m,
                                                       base + 3 * m, base + 4 * m,
base + 5 * m)
        cost[off_emg + idx] = (beta_emg / n_scen) * price_f
        row = w * 2 * m + idx
        for off, co in [(off_gd, 1.0), (off_emg, 1.0), (off_d, 1.0), (off_c, -1.0),
(off_s, -1.0)]:
            rows.append(row); cols.append(off + idx); vals.append(co * np.ones(m))
        b_eq.append(load_scen[w][tdec:] * L.DT - pv_scen[w][tdec:] * L.DT)
        row_soc = w * 2 * m + m + idx
        rows.append(row_soc); cols.append(off_e + idx); vals.append(np.ones(m))
        rows.append(row_soc); cols.append(off_c + idx); vals.append(-L.ETA_C *
np.ones(m))
        rows.append(row_soc); cols.append(off_d + idx); vals.append((1.0 / L.ETA_D)
* np.ones(m))
        rows.append(row_soc[1:]); cols.append(off_e + idx[1:] - 1); vals.append(-
np.ones(m - 1))
        rhs = np.zeros(m); rhs[0] = soc_start
        b_eq.append(rhs)
        rows_u.append(2 * m + w * m + idx); cols_u.append(off_gd + idx)
```

```
        vals_u.append(np.ones(m))
        rows_u.append(2 * m + w * m + idx); cols_u.append(0 + idx); vals_u.append(-
np.ones(m))
        b_ub.append(np.zeros(m))


    A_eq = coo_matrix((np.concatenate(vals), (np.concatenate(rows),
np.concatenate(cols))),
                      shape=(2 * n_scen * m, n_var)).tocsr()
    A_ub = coo_matrix((np.concatenate(vals_u), (np.concatenate(rows_u),
np.concatenate(cols_u))),
                      shape=((2 + n_scen) * m, n_var)).tocsr()


    lb = np.zeros(n_var); ub = np.empty(n_var)
    ub[0:m] = np.inf                        # g_adj 未来可自由调整
    ub[n_adj: n_adj + 2 * m] = np.inf       # u, v
    for w in range(n_scen):
        base = base_scen + w * 6 * m
        ub[base:base + m] = np.inf
        ub[base + m:base + 2 * m] = L.E_MAX
        ub[base + 2 * m:base + 3 * m] = L.E_MAX
        ub[base + 3 * m:base + 4 * m] = pv_scen[w][tdec:] * L.DT
```

```
        lb[base + 4 * m:base + 5 * m] = L.SOC_MIN
        ub[base + 4 * m:base + 5 * m] = L.SOC_MAX
        ub[base + 5 * m:base + 6 * m] = np.inf
        if soc_term is not None:
            lb[base + 4 * m + m - 1] = soc_term
    res = linprog(cost, A_eq=A_eq, b_eq=np.concatenate(b_eq), A_ub=A_ub,
                  b_ub=np.concatenate(b_ub), bounds=list(zip(lb, ub)),
method="highs")
    if not res.success:
        return None
    return {"g_adj": res.x[0:m].copy(), "tdec": tdec, "S": n_scen, "m": m,
            "base": base_scen, "obj": float(res.fun), "x": res.x}



def fc_residual_pool(fc10_all, actual_2d, d, k, eps=50.0, kk=40):
    """按发布时刻 k 分层的预报误差残差池，样本外。"""
    pool = []
    for j in range(d):
        f = fc10_all[j, k]
        if np.nanmax(f) < 1.0:            # 预报异常日跳过
            continue
```

```
        ratio = (actual_2d[j] + eps) / (f + eps)
        pool.append(np.clip(ratio, 0.2, 2.5))
    if not pool:
        pool = [np.ones(N_T)]
    return np.array(pool[-kk:])



def _same_type_days(dtypes, d, k):
    """目标日之前同型的最近 k 天。"""
    cands = [j for j in range(d) if dtypes[j] == dtypes[d]]
    if len(cands) < 5:
        cands = list(range(d))
    return cands[-k:]



def load_forecast(load_2d, dtypes, d, k=12, decay=0.88):
    """分解式负载点预测：日总量与日内形状两层外推。"""
    cands = _same_type_days(dtypes, d, k)
    weights = np.array([decay ** (len(cands) - 1 - i) for i in range(len(cands))])
    weights = weights / weights.sum()
```

```
    tot = load_2d[cands].sum(axis=1) * DT                    # 各历史日的日总量
    tot_hat = float(weights @ tot)
    shapes = load_2d[cands] / np.maximum(tot[:, None] / 24.0, 1e-9)
    shape_hat = weights @ shapes
    return (tot_hat / 24.0) * shape_hat



def load_residual_pool(load_2d, dtypes, d, k=60):
    """分解式残差池：第 0 列 rho, 其余列 sigma。"""
    types = np.asarray(dtypes)
    key = np.zeros(load_2d.shape[1] + 1)
    key[0] = 1.0
    pool = [key]
    cands = [j for j in range(d) if types[j] == types[d]]
    if len(cands) < 5:
        cands = list(range(d))
    for j in cands[-k:]:
        if j < 3:
            continue
        f = load_forecast(load_2d, dtypes, j, k=12)
```

```
        tot_f = f.sum() * DT
        tot_a = load_2d[j].sum() * DT
        rho = tot_a / max(tot_f, 1e-9)
        sig = load_2d[j] / np.maximum(f, 1e-9)
        row = np.empty(load_2d.shape[1] + 1)
        row[0] = np.clip(rho, 0.5, 1.8)
        row[1:] = np.clip(sig, 0.5, 1.8)
        pool.append(row)
    return np.array(pool)



def load_scenarios(pool, load_pt, n_scen, rng):
    """负荷情景 (T_hat*rho/24)*(s_hat*sigma)"""
    idx = rng.integers(0, len(pool), size=n_scen)
    rows = pool[idx]
    tot_pt = load_pt.sum() * DT                      # 点预测的日总量
    shape_pt = load_pt / np.maximum(tot_pt / 24.0, 1e-9)
    out = (tot_pt * rows[:, 0:1] / 24.0) * (shape_pt[None, :] * rows[:, 1:])
    return out
```

```
def clearsky_ref(pv_2d, d, k=45):
    """晴空参考：同季窗口光伏出力的上包络。"""
    lo, hi = max(0, d - k), d
    if hi - lo < 10:
        lo, hi = 0, max(d, 1)
    win = pv_2d[lo:hi]
    ref = np.percentile(win, 92, axis=0)
    # 轻度平滑
    ker = np.ones(5) / 5.0
    ref = np.convolve(np.r_[ref[0], ref[0], ref, ref[-1], ref[-1]], ker,
mode="same")[2:-2]
    return np.maximum(ref, 1.0)



def pv_forecast_persistence(pv_2d, dtypes, d, k=12, decay=0.88):
    """光伏点预测：晴空指数持续性法。"""
    ref_d = clearsky_ref(pv_2d, d)
    cands = _same_type_days(dtypes, d, k)
    clearness = []
    for j in cands:
```

```
        ref_j = clearsky_ref(pv_2d, j)
        clearness.append(pv_2d[j] / np.maximum(ref_j, 1.0))
    weights = np.array([decay ** (len(cands) - 1 - i) for i in range(len(cands))])
    weights = weights / weights.sum()
    clearness_hat = weights @ np.array(clearness)
    return np.clip(clearness_hat, 0.0, 1.05) * ref_d



def pv_residual_pool_persist(pv_2d, dtypes, d, k=60, eps=50.0):
    """光伏持续性预报残差池。"""
    pool = []
    cands = [j for j in range(d) if dtypes[j] == dtypes[d]]
    if len(cands) < 5:
        cands = list(range(d))
    for j in cands[-k:]:
        if j < 3:
            continue
        f = pv_forecast_persistence(pv_2d, dtypes, j, k=12)
        pool.append(np.clip((pv_2d[j] + eps) / (f + eps), 0.3, 2.2))
    if not pool:
```

```
        pool = [np.ones(pv_2d.shape[1])]
    return np.array(pool)



def soc_trajectory(price, g_fixed, load_kw, pv_kw, soc_start):
    """给定已承诺购电量，按当日实际负载/光伏做全天优化。"""
    # 物理常量统一从 lp_core 取
    import lp_core as L
    n = N_T
    blocks = {k: i * n for i, k in enumerate(["GD", "C", "D", "S", "E", "E2"])}
    idx = np.arange(n)
    cost = np.zeros(6 * n)
    cost[blocks["E2"] + idx] = L.BETA_EMG * price
    cost[blocks["C"] + idx] = L.THRU_PEN
    cost[blocks["D"] + idx] = L.THRU_PEN
    eq_r, eq_c, eq_v, b_eq = [], [], [], []
    for name, co in [("GD", 1.0), ("E2", 1.0), ("D", 1.0), ("C", -1.0), ("S", -
1.0)]:
        eq_r.append(idx); eq_c.append(blocks[name] + idx); eq_v.append(co *
np.ones(n))
    b_eq.append(load_kw * DT - pv_kw * DT)
```

```
    row_soc = n + idx

    eq_r.append(row_soc); eq_c.append(blocks["E"] + idx); eq_v.append(np.ones(n))

    eq_r.append(row_soc); eq_c.append(blocks["C"] + idx); eq_v.append(-L.ETA_C *
np.ones(n))

    eq_r.append(row_soc); eq_c.append(blocks["D"] + idx); eq_v.append((1.0 /
L.ETA_D) * np.ones(n))

    eq_r.append(row_soc[1:]); eq_c.append(blocks["E"] + idx[1:] - 1); eq_v.append(-
np.ones(n - 1))

    rhs = np.zeros(n); rhs[0] = soc_start

    b_eq.append(rhs)

    A_eq = coo_matrix((np.concatenate(eq_v), (np.concatenate(eq_r),
np.concatenate(eq_c))),

                      shape=(2 * n, 6 * n)).tocsr()

    lb = np.zeros(6 * n); ub = np.empty(6 * n)

    ub[blocks["GD"]:blocks["GD"] + n] = g_fixed

    ub[blocks["C"]:blocks["C"] + n] = L.E_MAX

    ub[blocks["D"]:blocks["D"] + n] = L.E_MAX

    ub[blocks["S"]:blocks["S"] + n] = pv_kw * DT

    lb[blocks["E"]:blocks["E"] + n] = L.SOC_MIN

    ub[blocks["E"]:blocks["E"] + n] = L.SOC_MAX

    ub[blocks["E2"]:blocks["E2"] + n] = np.inf

    res = linprog(cost, A_eq=A_eq, b_eq=np.concatenate(b_eq),
```

```
                  bounds=list(zip(lb, ub)), method="highs")
    if not res.success:
        return None
    x = res.x
    return (x[blocks["E"]:blocks["E"] + n], x[blocks["C"]:blocks["C"] + n],
            x[blocks["D"]:blocks["D"] + n],
            x[blocks["S"]:blocks["S"] + n], x[blocks["E2"]:blocks["E2"] + n])
```

附录B-3 数据预处理.py

```
"""附件数据清点：缺失值 / 异常值检测与清洗后导出，产出图1~图8。"""
import os, sys, io
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt


sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8")


plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
```

```
plt.rcParams["axes.unicode_minus"] = False

plt.rcParams["figure.dpi"] = 130



ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

ATT  = os.path.join(ROOT, "附件")

IMG  = os.path.join(ROOT, "图片")

RES  = os.path.join(ROOT, "结果")

for d in (IMG, RES):

    os.makedirs(d, exist_ok=True)



N_T, N_DAY, DT = 144, 365, 1.0/6.0

MONTHS = ["1月","2月","3月","4月","5月","6月","7月","8月","9月","10月","11月","12月
"]



def load(fn, **kw):

    return pd.read_excel(os.path.join(ATT, fn), **kw)



typical = load("附件1.xlsx")

load_year = load("附件2.xlsx", sheet_name="小区负载")

pv_year   = load("附件2.xlsx", sheet_name="光伏发电实际功率")

fcst = load("附件3.xlsx")
```

```
price_year = load("附件4.xlsx")



inventory = [

    ("附件1", "Sheet1",           typical.shape,    "某典型日 电价/负载/光伏预测",
"144 个 10 分钟时段"),

    ("附件2", "小区负载",          load_year.shape,  "全年 小区负载(实际)",
"365 天 x 144 时段"),

    ("附件2", "光伏发电实际功率",  pv_year.shape,    "全年 光伏发电(实际)",
"365 天 x 144 时段"),

    ("附件3", "Sheet1",           fcst.shape,       "全年 光伏预报",

     "365 天 x 4 发布时刻 x 24 小时"),

    ("附件4", "Sheet1",           price_year.shape, "全年 实时电价",
"365 天 x 144 时段"),

]

inv_rows = []

for fn, sheet, shp, desc, gran in inventory:

    print("%-8s %-18s %-14s %-22s %s" % (fn, sheet, str(shp), desc, gran))

    inv_rows.append({"文件":fn, "工作表":sheet, "行":shp[0], "列":shp[1], "内容
":desc, "粒度":gran})

pd.DataFrame(inv_rows).to_csv(os.path.join(RES, "数据清单.csv"), index=False,
encoding="utf-8-sig")
```

```
price1 = pd.to_numeric(typical["电价"], errors="coerce").values

load1  = pd.to_numeric(typical["小区负载"], errors="coerce").values

pv1    = pd.to_numeric(typical["光伏发电预测功率"], errors="coerce").values

load2  = load_year.iloc[:, 1:1+N_T].apply(pd.to_numeric, errors="coerce").values

pv2    = pv_year.iloc[:, 1:1+N_T].apply(pd.to_numeric, errors="coerce").values

price4 = price_year.iloc[:, 1:1+N_T].apply(pd.to_numeric, errors="coerce").values



def stat_line(name, x, unit):

    print("%-26s min=%10.4f  max=%10.4f  mean=%10.4f  std=%9.4f  %s"

          % (name, np.nanmin(x), np.nanmax(x), np.nanmean(x), np.nanstd(x), unit))

stat_line("附件1 电价",     price1, "元/kWh")

stat_line("附件1 小区负载", load1,  "kW")

stat_line("附件1 光伏预测", pv1,    "kW")

stat_line("附件2 小区负载", load2,  "kW")

stat_line("附件2 光伏实际", pv2,    "kW")

stat_line("附件4 实时电价", price4, "元/kWh")



miss_rows = []

def count_missing(name, df, skip_col0=False):

    sub = df.iloc[:, 1:] if skip_col0 else df
```

```
    n = int(sub.isna().sum().sum())
    tot = int(sub.size)
    miss_rows.append({"数据体":name, "单元格总数":tot, "缺失单元格":n,
                      "缺失率": round(n/tot*100, 6) if tot else 0.0})
    return n, tot

checks = [
    ("附件1", typical, False),
    ("附件2/小区负载", load_year, True),
    ("附件2/光伏实际", pv_year, True),
    ("附件3", fcst, True),
    ("附件4", price_year, True),
]
for name, df, sk in checks:
    count_missing(name, df, sk)

# 附件3 日期列为合并单元格空白
date_col = fcst.iloc[:, 0]
n_blank = int(date_col.isna().sum())
filled = date_col.ffill()
print("显式缺失单元格", sum(r["缺失单元格"] for r in miss_rows), "个；附件3 日期列
```

```
空白", n_blank,
      "行, ffill 后", filled.nunique(), "个日期, 每天行数",
sorted(filled.value_counts().unique().tolist()))

pd.DataFrame(miss_rows).to_csv(os.path.join(RES, "缺失值报告.csv"), index=False,
encoding="utf-8-sig")

def zscore_flag(x, thr=3.0):
    x = np.asarray(x, float)
    sd = x.std(ddof=0)
    if sd == 0:
        return np.zeros(x.size, bool), np.zeros(x.size)
    z = (x - x.mean()) / sd
    return np.abs(z) > thr, z

def iqr_flag(x, k=1.5):
    x = np.asarray(x, float)
    q1, q3 = np.percentile(x, [25, 75])
    iqr = q3 - q1
    lo, hi = q1 - k*iqr, q3 + k*iqr
    f = (x < lo) | (x > hi)
```

```
    return f, (lo, hi)


def both_flags(x):
    fz, z = zscore_flag(x)
    fi, _ = iqr_flag(x)
    return fz, fi, z


for name, x in [("电价", price1), ("小区负载", load1), ("光伏预测", pv1)]:
    fz, fi, z = both_flags(x)
    q1, q3 = np.percentile(x, [25, 75])
    iqr_ = q3 - q1
    print("%-10s  Z-score(|z|>3) 检出 %3d 点 | IQR(1.5倍) 检出 %3d 点 | Q1=%.4f
Q3=%.4f IQR=%.4f"
          % (name, int(fz.sum()), int(fi.sum()), q1, q3, iqr_))
print("附件1 电价为峰谷分时电价，双峰结构下 IQR / Z-score 必然把合法峰段判为异常，
属结构性误报，保留原值")


dates = pd.to_datetime(load_year.iloc[:, 0])
month = dates.dt.month.values


def global_vs_monthly(name, X, unit):
```

```
    fz, fi, _ = both_flags(X.ravel())
    mz, mi = [], []
    for m in range(1, 13):
        sub = X[month == m].ravel()
        a, _, _ = both_flags(sub)
        b, _ = iqr_flag(sub)
        mz.append(int(a.sum())); mi.append(int(b.sum()))
    print("%s(%s) 全局 Z 检出 %6d 点 (%.2f%%) / 分月合计 %d 点；全局 IQR 检出 %6d
点 (%.2f%%) / 分月合计 %d 点"
          % (name, unit, int(fz.sum()), fz.mean()*100, sum(mz),
             int(fi.sum()), fi.mean()*100, sum(mi)))
    return mz, mi


mz_load, mi_load = global_vs_monthly("附件2 小区负载", load2, "kW")
mz_pv, mi_pv = global_vs_monthly("附件2 光伏实际", pv2, "kW")
mz_price, mi_price = global_vs_monthly("附件4 实时电价", price4, "元/kWh")


neg_load = int((load2 < 0).sum())
neg_pv = int((pv2 < 0).sum())
neg_price = int((price4 <= 0).sum())
print("物理上不可能的点数：负载为负 %d, 光伏为负 %d, 电价<=0 %d" % (neg_load,
```

```
neg_pv, neg_price))



night = np.r_[0:24, 132:144]

night_max = float(np.nanmax(pv2[:, night]))

day = np.arange(36, 108)

day_min = float(np.nanmin(pv2[:, day]))

print("光伏 夜间(22:00-04:00)最大值 %.4f kW, 日间(06:00-18:00)最小值 %.4f kW" %
(night_max, day_min))



mono = np.abs(np.diff(load2, axis=1))

low_c = np.where(price4 < 0.05)

print("负载相邻时段最大跳变 %.1f kW (%.2f%% of 均值)；电价 < 0.05 元/kWh 的点 %d 个
(%.4f%%)"

      % (mono.max(), mono.max()/load2.mean()*100, len(low_c[0]),
len(low_c[0])/price4.size*100))



fig, axes = plt.subplots(1, 3, figsize=(13, 4.2))

data1 = [("电价 (元/kWh)", price1, "#2E6F9E"),

         ("小区负载 (kW)", load1, "#3F8F5B"),

         ("光伏预测 (kW)", pv1, "#C77D2B")]

for ax, (name, x, col) in zip(axes, data1):
```

```
    bp = ax.boxplot(x, vert=True, patch_artist=True, widths=0.5,
                    flierprops=dict(marker="o", markerfacecolor="#D64545",
markersize=5, alpha=0.8))
    bp["boxes"][0].set_facecolor(col); bp["boxes"][0].set_alpha(0.55)
    for med in bp["medians"]: med.set_color("#222222"); med.set_linewidth(1.6)
    ax.set_title("附件1 典型日 · " + name, fontsize=11)
    ax.set_xticks([]); ax.grid(axis="y", alpha=0.25, linestyle=":")
fig.suptitle("附件1 典型日三变量箱线图（红点为疑似离群点）", fontsize=12.5)
fig.tight_layout(rect=[0, 0, 1, 0.94])
fig.savefig(os.path.join(IMG, "图2_附件1三变量箱线图.png"), bbox_inches="tight")
plt.close(fig)


def monthly_box(X, title, ylab, fname, color):
    fig, ax = plt.subplots(figsize=(13, 4.4))
    data = [X[month == m].ravel() for m in range(1, 13)]
    bp = ax.boxplot(data, patch_artist=True, widths=0.6, showfliers=True,
                    flierprops=dict(marker="o", markerfacecolor="#D64545",
                                    markersize=2.2, alpha=0.35),
                    medianprops=dict(color="#222222", linewidth=1.4))
    for b in bp["boxes"]:
        b.set_facecolor(color); b.set_alpha(0.55)
```

```
    ax.set_xticklabels(MONTHS, fontsize=10)
    ax.set_ylabel(ylab, fontsize=11)
    ax.set_title(title, fontsize=12.5)
    ax.grid(axis="y", alpha=0.25, linestyle=":")
    fig.tight_layout()
    fig.savefig(os.path.join(IMG, fname), bbox_inches="tight")
    plt.close(fig)


monthly_box(load2, "2025 年小区负载分月箱线图", "负载（kW)", "图3_全年负载分月箱线图.png", "#3F8F5B")
monthly_box(pv2, "2025 年光伏实际功率分月箱线图", "光伏功率（kW)", "图4_全年光伏分月箱线图.png", "#C77D2B")
monthly_box(price4, "2025 年实时电价分月箱线图", "电价（元/kWh)", "图5_全年电价分月箱线图.png", "#2E6F9E")


fig, axes = plt.subplots(1, 2, figsize=(13, 4.2))
hprof = pv2.reshape(N_DAY*N_T//6, 6).mean(axis=1).reshape(N_DAY, 24).mean(axis=0) if False else None
P_hour = pv2.reshape(N_DAY, 24, 6).mean(axis=2)   # (365, 24)
mean_h = P_hour.mean(axis=0); p95_h = np.percentile(P_hour, 95, axis=0)
hours = np.arange(24)
axes[0].fill_between(hours, 0, p95_h, color="#C77D2B", alpha=0.25, label="95 分位")
```

```
axes[0].plot(hours, mean_h, color="#8A4B12", linewidth=2.2, marker="o",
markersize=4, label="日均值")

axes[0].set_xticks(hours[::2]); axes[0].set_xlabel("小时"); axes[0].set_ylabel("光
伏功率（kW)")

axes[0].set_title("（a）全年光伏日内平均曲线", fontsize=11.5)

axes[0].legend(fontsize=9); axes[0].grid(alpha=0.25, linestyle=":")



night_vals = P_hour[:, list(range(0,5)) + list(range(22,24))].ravel()

axes[1].hist(night_vals, bins=40, color="#2E6F9E", alpha=0.8, edgecolor="white")

axes[1].set_xlabel("夜间光伏功率（kW)"); axes[1].set_ylabel("频数")

axes[1].set_title("（b）夜间(22:00-05:00)光伏功率分布，最大 %.2f kW" %
night_vals.max(), fontsize=11.5)

axes[1].grid(alpha=0.25, linestyle=":")

fig.suptitle("光伏数据的昼夜一致性校验", fontsize=12.5)

fig.tight_layout(rect=[0, 0, 1, 0.94])

fig.savefig(os.path.join(IMG, "图6_光伏昼夜一致性.png"), bbox_inches="tight")

plt.close(fig)



fig, ax = plt.subplots(figsize=(12, 4.4))

labels = ["附件1\n电价", "附件1\n负载", "附件1\n光伏", "全年\n负载", "全年\n光伏",
"全年\n电价"]

gz, gi = [], []
```

```
for x in [price1, load1, pv1, load2.ravel(), pv2.ravel(), price4.ravel()]:
    a, _, _ = both_flags(x); b, _ = iqr_flag(x)
    gz.append(a.mean()*100); gi.append(b.mean()*100)
xpos = np.arange(len(labels)); w = 0.38
b1 = ax.bar(xpos - w/2, gz, w, label="Z-score 法 (|z|>3)", color="#2E6F9E",
alpha=0.85)
b2 = ax.bar(xpos + w/2, gi, w, label="IQR 法 (1.5 倍)", color="#D64545",
alpha=0.85)
ax.set_xticks(xpos); ax.set_xticklabels(labels, fontsize=10)
ax.set_ylabel("检出比例 (%)")
ax.set_title("两种异常值检出方法的检出率对比", fontsize=12.5)
ax.legend(fontsize=9.5); ax.grid(axis="y", alpha=0.25, linestyle=":")
for bars in (b1, b2):
    for r in bars:
        h = r.get_height()
        ax.annotate("%.2f%%" % h, (r.get_x()+r.get_width()/2, h),
                    ha="center", va="bottom", fontsize=8.5)
fig.tight_layout()
fig.savefig(os.path.join(IMG, "图7_Zscore与IQR检出对比.png"), bbox_inches="tight")
plt.close(fig)
```

```
fig, axes = plt.subplots(3, 1, figsize=(12, 8), sharex=True)

t = np.arange(1, N_T+1) * 10 / 60.0

axes[0].step(t, price1, where="post", color="#2E6F9E", linewidth=1.6)

axes[0].set_ylabel("电价（元/kWh)", fontsize=10.5)

axes[0].set_title("附件1 典型日：电价 / 负载 / 光伏预测", fontsize=12.5)

axes[0].grid(alpha=0.25, linestyle=":")

axes[1].plot(t, load1, color="#3F8F5B", linewidth=1.6)

axes[1].set_ylabel("小区负载（kW)", fontsize=10.5); axes[1].grid(alpha=0.25,
linestyle=":")

axes[2].fill_between(t, 0, pv1, color="#C77D2B", alpha=0.45)

axes[2].plot(t, pv1, color="#8A4B12", linewidth=1.6)

axes[2].set_ylabel("光伏预测（kW)", fontsize=10.5)

axes[2].set_xlabel("时刻（小时)", fontsize=10.5)

axes[2].set_xticks(np.arange(0, 25, 2))

axes[2].grid(alpha=0.25, linestyle=":")

fig.tight_layout()

fig.savefig(os.path.join(IMG, "图1_附件1典型日曲线.png"), bbox_inches="tight")

plt.close(fig)



fig, axes = plt.subplots(1, 2, figsize=(13, 4.2))

axes[0].hist(price4.ravel(), bins=80, color="#2E6F9E", alpha=0.85,
```

```
edgecolor="white")

axes[0].axvline(0.05, color="#D64545", linestyle="--", linewidth=1.6, label="0.05
元/kWh")

axes[0].set_xlabel("电价（元/kWh)"); axes[0].set_ylabel("频数")

axes[0].set_title("（a）附件4 电价全局分布", fontsize=11.5)

axes[0].legend(fontsize=9); axes[0].grid(alpha=0.25, linestyle=":")



axes[1].plot(price4.ravel()[:288], color="#2E6F9E", linewidth=0.9)

axes[1].set_xlabel("时段序号（前 2 天，共 288 段）"); axes[1].set_ylabel("电价（元
/kWh)")

axes[1].set_title("（b）电价时序片段（可见剧烈日内波动）", fontsize=11.5)

axes[1].grid(alpha=0.25, linestyle=":")

fig.suptitle("附件4 实时电价特征诊断", fontsize=12.5)

fig.tight_layout(rect=[0, 0, 1, 0.94])

fig.savefig(os.path.join(IMG, "图8_附件4电价诊断.png"), bbox_inches="tight")

plt.close(fig)



from scipy.optimize import linprog

# 尺度变换 x = D*y，D 含 1000 倍量纲差

c0 = np.array([0.4, 1.3])

A0 = np.array([[-1.0, -1.0]]); b0 = np.array([-10.0])
```

```
bnd0 = [(0, 100)] * 2

r_raw = linprog(c0, A_ub=A0, b_ub=b0, bounds=bnd0)



D  = np.array([1000.0, 0.001])          # 人为制造 10^6 量级的变量尺度差

c1 = c0 * D                             # 目标系数同步缩放

A1 = A0 * D                             # 约束系数同步缩放

bnd1 = [(0, 100/D[0]), (0, 100/D[1])]

r_scaled = linprog(c1, A_ub=A1, b_ub=b0, bounds=bnd1)

x_back = r_scaled.x * D                 # 还原到原始尺度



print("LP 最优值 原始 %.6f / 缩放 %.6f, 差 %.3e；还原后 x = %s"

      % (r_raw.fun, r_scaled.fun, abs(r_raw.fun - r_scaled.fun), np.round(x_back,
6).tolist()))



out1 = typical.copy()

out1["段号"] = np.arange(1, N_T+1)

out1["区间"] = ["%02d:%02d-%02d:%02d"

                % (((i-1)*10)//60, ((i-1)*10)%60, (i*10)//60 if i*10 < 1440 else 0,
(i*10) % 60)

                for i in range(1, N_T+1)]

out1.to_csv(os.path.join(RES, "清洗后_附件1典型日.csv"), index=False,
```

```
encoding="utf-8-sig")



pv2_clip = np.clip(pv2, 0.0, None)

np.save(os.path.join(RES, "附件2_负载.npy"), load2)

np.save(os.path.join(RES, "附件2_光伏.npy"), pv2_clip)

np.save(os.path.join(RES, "附件4_电价.npy"), price4)



figs = sorted([f for f in os.listdir(IMG) if f.endswith(".png")])

print("数据自足无需补充；已写 结果/ 下 3 个 csv 与 3 个 npy，图片目录当前共有",
len(figs), "张 png")
```

附录B-4 问题1求解.py

```
"""问题1 求解：单日确定性 LP，写 result1.xlsx。"""

import os, sys, shutil

import numpy as np

import openpyxl



sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import lp_core as L



D = L.load_all(verbose=False)
```

```
price = D["typical"]["price"]

load_kw = D["typical"]["load_kw"]

pv_kw = D["typical"]["pv_kw"]



SOC0 = SOC_END = L.SOC_INIT                  # 日内闭环：末端钉回初值



sol = L.solve_day_lp(price, load_kw, pv_kw, soc_start=SOC0, soc_end=SOC_END)

assert sol["status"] == 0, "LP 未求得最优解: %s" % sol.get("message")

g, c, d, s, E = sol["g"], sol["c"], sol["d"], sol["s"], sol["E"]



# 命题1：纯 LP 不会同时充放电

simul = float(np.minimum(c, d).max())



tpl = os.path.join(L.ATT, "附件5", "result1.xlsx")

out_path = os.path.join(L.ROOT, "result1.xlsx")

shutil.copy(tpl, out_path)                   # 保留模板表头，只填数值

wb = openpyxl.load_workbook(out_path)



ws = wb["计划购电量"]

for i in range(L.N_T):
```

```
    ws.cell(row=2 + i, column=2, value=float(round(g[i], 6)))


# 充放电量按六大时段汇总
ws2 = wb["充放电量"]
chg = L.block_sums(c)
dis = L.block_sums(d)
for i in range(6):
    ws2.cell(row=2 + i, column=2, value=float(round(chg[i], 6)))    # 充电量
    ws2.cell(row=2 + i, column=3, value=float(round(dis[i], 6)))    # 放电量
ws2.cell(row=2, column=5, value=float(SOC0))        # 0:00 储电量
ws2.cell(row=3, column=5, value=float(E[-1]))       # 24:00 储电量
wb.save(out_path)


# 无储能基准：每段直接买净负荷
net = np.maximum(load_kw * L.DT - pv_kw * L.DT, 0.0)
cost_base = float((price * net).sum())
save = cost_base - sol["cost"]


np.savez(os.path.join(L.RES, "问1_结果.npz"), g=g, c=c, d=d, s=s, E=E,
         price=price, load_kw=load_kw, pv_kw=pv_kw, dual=sol["dual_balance"],
```

```
          cost=sol["cost"], cost_base=cost_base)



print("问题1 单日 LP")

print("同时充放电最大重叠 %.10f kWh" % simul)

print("购电 %.2f kWh / %.2f 元；弃光 %.2f kWh；SOC %.1f~%.1f"

      % (g.sum(), sol["cost"], s.sum(), E.min(), E.max()))

print("无储能基准 %.2f 元，储能节省 %.2f 元（%.2f%%）"

      % (cost_base, save, save / cost_base * 100))

print("已写 result1.xlsx 与 结果/问1_结果.npz")
```

附录B-5 问题2求解.py

```
"""问题2 求解：两阶段随机规划，写 result2.xlsx。"""

import os, sys, time

import numpy as np

import pandas as pd

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

import openpyxl, shutil, datetime as dt

from scipy.optimize import linprog
```

```
from scipy.sparse import coo_matrix



sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import lp_core as L

import stoch as ST



plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]

plt.rcParams["axes.unicode_minus"] = False

plt.rcParams["figure.dpi"] = 130

rng = np.random.default_rng(42)



N = L.N_T

N_SCEN = 20                       # 情景数 20

REP0 = 31

KEY_DATES = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]



D = L.load_all(verbose=False)

price = D["typical"]["price"]

load_2d = D["actual"]["load_kw"]

pv_2d = D["actual"]["pv_kw"]
```

```
dates = D["actual"]["dates"]

DTYPE = ST.day_type(dates)

print("报告期 %s ~ %s, 情景数 %d"

      % (L.REPORT_START.date(), L.REPORT_END.date(), N_SCEN))




def settle_day(g_plan, load_kw, pv_kw, soc_start, soc_term=None):

    """按实际负载与光伏结算已承诺计划量。"""

    n = N

    base = {k: i * n for i, k in enumerate(["GD", "C", "D", "S", "E", "E2"])}

    idx = np.arange(n)

    obj = np.zeros(6 * n)

    obj[base["E2"] + idx] = L.BETA_EMG * price

    obj[base["C"] + idx] = L.THRU_PEN        # 吞吐量正则化

    obj[base["D"] + idx] = L.THRU_PEN



    row, col, val, rhs = [], [], [], []

    for name, coef in [("GD", 1.0), ("E2", 1.0), ("D", 1.0), ("C", -1.0), ("S", -
1.0)]:

        row.append(idx)

        col.append(base[name] + idx)
```

```
        val.append(coef * np.ones(n))
    rhs.append(load_kw * L.DT - pv_kw * L.DT)
    bal = n + idx
    row.append(bal); col.append(base["E"] + idx); val.append(np.ones(n))
    row.append(bal); col.append(base["C"] + idx); val.append(-L.ETA_C * np.ones(n))
    row.append(bal); col.append(base["D"] + idx); val.append((1.0 / L.ETA_D) *
np.ones(n))
    row.append(bal[1:]); col.append(base["E"] + idx[1:] - 1); val.append(-np.ones(n
- 1))
    soc0 = np.zeros(n)
    soc0[0] = soc_start
    rhs.append(soc0)
    A_eq = coo_matrix((np.concatenate(val), (np.concatenate(row),
np.concatenate(col))),
                      shape=(2 * n, 6 * n)).tocsr()
    # g_draw <= g_plan 由变量上界施加
    lb = np.zeros(6 * n)
    ub = np.empty(6 * n)
    ub[base["GD"]:base["GD"] + n] = g_plan
    ub[base["C"]:base["C"] + n] = L.E_MAX
    ub[base["D"]:base["D"] + n] = L.E_MAX
```

```
    ub[base["S"]:base["S"] + n] = pv_kw * L.DT
    lb[base["E"]:base["E"] + n] = L.SOC_MIN
    ub[base["E"]:base["E"] + n] = L.SOC_MAX
    ub[base["E2"]:base["E2"] + n] = np.inf
    # 每日末端不闭环
    if soc_term is not None:
        lb[base["E"] + n - 1] = soc_term
    return linprog(obj, A_eq=A_eq, b_eq=np.concatenate(rhs),
                   bounds=list(zip(lb, ub)), method="highs")



t_start = time.time()
plan_2d = np.zeros((L.N_DAY, N))
chg_2d = np.zeros((L.N_DAY, N))
dis_2d = np.zeros((L.N_DAY, N))
spill_2d = np.zeros((L.N_DAY, N))
soc_2d = np.zeros((L.N_DAY, N))
emg_2d = np.zeros((L.N_DAY, N))
pct_of_plan = np.full(L.N_DAY, np.nan)      # 计划量在情景净负载分布中的分位
soc = L.SOC_INIT
```

```
for d in range(L.N_DAY):
    if d < 5:                                   # 热身期用当日实际
        plan_2d[d] = np.maximum(load_2d[d] * L.DT - pv_2d[d] * L.DT, 0.0)
        settled = settle_day(plan_2d[d], load_2d[d], pv_2d[d], soc)
        x = settled.x
        chg_2d[d] = x[N:2*N]; dis_2d[d] = x[2*N:3*N]; spill_2d[d] = x[3*N:4*N]
        soc_2d[d] = x[4*N:5*N]; emg_2d[d] = x[5*N:6*N]; soc = soc_2d[d, -1]
        continue
    load_fc = ST.point_forecast(load_2d, DTYPE, d)
    pv_fc = ST.point_forecast(pv_2d, DTYPE, d)
    load_res = ST.residual_pool(load_2d, DTYPE, d, eps=0.0)
    pv_res = ST.residual_pool(pv_2d, DTYPE, d, eps=50.0)
    load_scen = ST.sample_scenarios(load_res, N_SCEN, rng) * load_fc
    pv_scen = np.clip(ST.sample_scenarios(pv_res, N_SCEN, rng) * pv_fc, 0.0, None)
    two_stage = ST.solve_two_stage(price, load_fc, pv_fc, load_scen, pv_scen, soc,
None)
    if two_stage is None:
        plan_2d[d] = np.maximum(load_fc * L.DT - pv_fc * L.DT, 0.0)
    else:
        plan_2d[d] = two_stage["g_plan"]
        net_sc = (load_scen - pv_scen).sum(axis=1) * L.DT
```

```
        net_act = float((load_2d[d] - pv_2d[d]).sum() * L.DT)

        pct_of_plan[d] = float((net_sc < net_act).mean() * 100)

    settled = settle_day(plan_2d[d], load_2d[d], pv_2d[d], soc, None)   # 储能跨日自由传递

    assert settled.success, "第 %d 天结算失败" % d

    x = settled.x

    chg_2d[d] = x[N:2*N]

    dis_2d[d] = x[2*N:3*N]

    spill_2d[d] = x[3*N:4*N]

    soc_2d[d] = x[4*N:5*N]

    emg_2d[d] = x[5*N:6*N]

    soc = soc_2d[d, -1]

    if (d + 1) % 40 == 0:

        print("已完成 %3d/%d 天, 用时 %.0f s" % (d + 1, L.N_DAY, time.time() - t_start))

print("滚动完成, 用时 %.1f s" % (time.time() - t_start))



report = slice(REP0, L.N_DAY)

rep_dates = dates.iloc[report].reset_index(drop=True)

plan_rep = plan_2d[report]

emg_rep = emg_2d[report]
```

```
chg_rep = chg_2d[report]

dis_rep = dis_2d[report]

spill_rep = spill_2d[report]

soc_rep = soc_2d[report]

cost_plan = float((price * plan_rep).sum())

cost_emg = float(L.BETA_EMG * (price * emg_rep).sum())

print("报告期计划购电 %.2f 元，紧急购电 %.2f 元（%.2f kWh，涉及 %d 天）"

      % (cost_plan, cost_emg, emg_rep.sum(), int((emg_rep.sum(axis=1) > 1e-
6).sum())))

print("报告期合计 %.2f 元" % (cost_plan + cost_emg))



pct_vals = pct_of_plan[REP0:]

pct_vals = pct_vals[~np.isnan(pct_vals)]

above_p80 = pct_vals > 80

print("净负载相对情景分布的平均分位 %.1f%%，超出 P80 的天数占比 %.1f%%"

      % (pct_vals.mean(), above_p80.mean() * 100))



net_base = np.maximum(load_2d[report] * L.DT - pv_2d[report] * L.DT, 0)

cost_nostore = float((price * net_base).sum())

cost_perfect = float((price * np.maximum(load_2d[report] * L.DT - pv_2d[report] *
L.DT, 0)).sum())
```

```
print("无储能基准 %.2f 元，两阶段随机规划 %.2f 元" % (cost_nostore, cost_plan + cost_emg))
print("相对无储能节省 %.2f 元（%.2f%%）"
      % (cost_nostore - cost_plan - cost_emg,
         (cost_nostore - cost_plan - cost_emg) / cost_nostore * 100))


template = os.path.join(L.ATT, "附件5", "result2.xlsx")
out = os.path.join(L.ROOT, "result2.xlsx")
shutil.copy(template, out)
wb = openpyxl.load_workbook(out)
ws = wb["计划购电量"]
for i in range(len(rep_dates)):
    for t in range(N):
        ws.cell(row=2 + i, column=2 + t, value=float(round(plan_rep[i, t], 6)))
    ws.cell(row=2 + i, column=146, value=float(round(plan_rep[i].sum(), 6)))
    ws.cell(row=2 + i, column=147, value=float(round((price * plan_rep[i]).sum(), 6)))
cf_pos = wb.sheetnames.index("充放电量")
del wb["充放电量"]
ws2 = wb.create_sheet("充放电量", cf_pos)
ws2.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
```

```
for i in range(len(rep_dates)):

    chg = L.block_sums(chg_rep[i])

    dis = L.block_sums(dis_rep[i])

    first_row = 2 + i * 6

    for k, (block_name, _, _) in enumerate(L.BLOCKS):

        blk_row = first_row + k

        if k == 0:

            ws2.cell(row=blk_row, column=1,
value=rep_dates.iloc[i].to_pydatetime())

            ws2.cell(row=blk_row, column=5, value=dt.time(0, 0))

            ws2.cell(row=blk_row, column=6, value=float(round(

                soc_2d[REP0 - 1, -1] if i == 0 else soc_rep[i - 1, -1], 6)))

        if k == 1:

            ws2.cell(row=blk_row, column=5, value="24:00")

            ws2.cell(row=blk_row, column=6, value=float(round(soc_rep[i, -1], 6)))

        ws2.cell(row=blk_row, column=2, value=block_name)

        ws2.cell(row=blk_row, column=3, value=float(round(chg[k], 6)))

        ws2.cell(row=blk_row, column=4, value=float(round(dis[k], 6)))

em_pos = wb.sheetnames.index("紧急购电量")

del wb["紧急购电量"]

ws3 = wb.create_sheet("紧急购电量", em_pos)
```

```
ws3.append(["日期", "购电时间段", "购电量"])

excel_row = 2

for i in range(len(rep_dates)):

    active = np.where(emg_rep[i] > 1e-6)[0]

    ws3.cell(row=excel_row, column=1, value=rep_dates.iloc[i].to_pydatetime())

    if len(active) == 0:

        ws3.cell(row=excel_row, column=2, value="无")

        ws3.cell(row=excel_row, column=3, value=0.0)

        excel_row += 1

    else:

        # 连续为正的时段合并成区间

        runs = []

        start = active[0]

        for a, b in zip(active[:-1], active[1:]):

            if b != a + 1:

                runs.append((start, a))

                start = b

        runs.append((start, active[-1]))

        for j, (a, b) in enumerate(runs):

            if j > 0:
```

```
                ws3.cell(row=excel_row, column=1, value=None)
            ws3.cell(row=excel_row, column=2, value="%02d:%02d-%02d:%02d"
                     % ((a * 10) // 60, (a * 10) % 60, ((b + 1) * 10) // 60 % 24,
                        ((b + 1) * 10) % 60))
            ws3.cell(row=excel_row, column=3, value=float(round(emg_rep[i, a:b +
1].sum(), 6)))
            excel_row += 1
wb.save(out)
print("已写入", out)


print("表3 指定日期的紧急购电量")
for key_date in KEY_DATES:
    pos = int(np.where(rep_dates == pd.Timestamp(key_date))[0][0])
    print(key_date, "紧急 %.2f kWh" % emg_rep[pos].sum(), "计划 %.2f kWh" %
plan_rep[pos].sum(),
          "当日 %.2f 元" % (price @ plan_rep[pos] + L.BETA_EMG * price @
emg_rep[pos]))


np.savez(os.path.join(L.RES, "问2_结果.npz"),
         g=plan_rep, e=emg_rep, c=chg_rep, d=dis_rep, s=spill_rep, E=soc_rep,
price=price,
         g_all=plan_2d, E_all=soc_2d, e_all=emg_2d, pct=pct_of_plan,
```

```
        dates=rep_dates.values.astype("datetime64[D]"),
        cost_plan=cost_plan, cost_emg=cost_emg, cost_nostore=cost_nostore,
n_scen=N_SCEN)
print("已生成数据：结果/问2_结果.npz")
```

附录B-6 问题3求解.py

```
"""问题3 求解：滚动时域随机决策，写 result3.xlsx。"""
import os, sys, time
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import openpyxl, shutil, datetime as _dt

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import lp_core as L
import stoch as ST

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
```

```
plt.rcParams["figure.dpi"] = 130

rng = np.random.default_rng(42)



N = L.N_T

N_SCEN = 20

DECIDE_T = {1: 36, 2: 72, 3: 108}

REP0 = 31

KEY_DATES = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]



print("问题3 滚动时域随机决策模型")



D = L.load_all(verbose=False)

price = D["typical"]["price"]

load_2d = D["actual"]["load_kw"]

pv_2d = D["actual"]["pv_kw"]

FC = D["forecast"]["fc"]

dates = D["actual"]["dates"]

DTYPE = ST.day_type(dates)

FC10 = np.zeros((L.N_DAY, 4, N))

for d in range(L.N_DAY):
```

```
    for k in range(4):
        FC10[d, k] = L.forecast_to_10min(FC[d], k)
# 偏差订正表：扩张窗口，样本外
BIAS = L.forecast_bias_table(FC10, pv_2d)
print("预报已降尺度为 144 段，午后各版本平均偏低 %.1f~%.1f kW"
      % (BIAS[REP0:, 0, 72:].mean(), BIAS[REP0:, 3, 72:].mean()))



def settle_day(g_plan, load_kw, pv_kw, soc_start):
    """核算充放电量；GD 为承诺购电上限。"""
    from scipy.optimize import linprog
    from scipy.sparse import coo_matrix
    n = N
    ofs = {name: i * n for i, name in enumerate(["GD", "C", "D", "S", "E", "E2"])}
    seg = np.arange(n)
    cost = np.zeros(6 * n)
    cost[ofs["E2"] + seg] = L.BETA_EMG * price
    cost[ofs["C"] + seg] = L.THRU_PEN
    cost[ofs["D"] + seg] = L.THRU_PEN
    eq_row, eq_col, eq_val, eq_rhs = [], [], [], []
    for name, coef in [("GD", 1.0), ("E2", 1.0), ("D", 1.0), ("C", -1.0), ("S", -
```

```
1.0)]:

        eq_row.append(seg)

        eq_col.append(ofs[name] + seg)

        eq_val.append(coef * np.ones(n))

    eq_rhs.append(load_kw * L.DT - pv_kw * L.DT)

    soc_row = n + seg

    eq_row.append(soc_row)

    eq_col.append(ofs["E"] + seg)

    eq_val.append(np.ones(n))

    eq_row.append(soc_row)

    eq_col.append(ofs["C"] + seg)

    eq_val.append(-L.ETA_C * np.ones(n))

    eq_row.append(soc_row)

    eq_col.append(ofs["D"] + seg)

    eq_val.append((1.0 / L.ETA_D) * np.ones(n))

    eq_row.append(soc_row[1:])

    eq_col.append(ofs["E"] + seg[1:] - 1)

    eq_val.append(-np.ones(n - 1))

    e0 = np.zeros(n)

    e0[0] = soc_start
```

```
    eq_rhs.append(e0)

    A_eq = coo_matrix((np.concatenate(eq_val), (np.concatenate(eq_row),
np.concatenate(eq_col))),

                      shape=(2 * n, 6 * n)).tocsr()

    lb = np.zeros(6 * n)

    ub = np.empty(6 * n)

    ub[ofs["GD"]:ofs["GD"] + n] = g_plan

    ub[ofs["C"]:ofs["C"] + n] = L.E_MAX

    ub[ofs["D"]:ofs["D"] + n] = L.E_MAX

    ub[ofs["S"]:ofs["S"] + n] = pv_kw * L.DT

    lb[ofs["E"]:ofs["E"] + n] = L.SOC_MIN

    ub[ofs["E"]:ofs["E"] + n] = L.SOC_MAX

    ub[ofs["E2"]:ofs["E2"] + n] = np.inf

    return linprog(cost, A_eq=A_eq, b_eq=np.concatenate(eq_rhs),

                   bounds=list(zip(lb, ub)), method="highs")




def run(active_stages, vint=None, verbose=False):

    """全年滚动：阶段0 定计划，阶段1~3 调整；vint 为各阶段预报版本。"""

    if vint is None:

        vint = {k: k for k in (1, 2, 3)}
```

```
    rng = np.random.default_rng(42)

    gp = np.zeros((L.N_DAY, N))

    ga = np.zeros((L.N_DAY, N))

    cc = np.zeros((L.N_DAY, N))

    dd = np.zeros((L.N_DAY, N))

    ss = np.zeros((L.N_DAY, N))

    EE = np.zeros((L.N_DAY, N))

    ee = np.zeros((L.N_DAY, N))

    soc = L.SOC_INIT

    for d in range(L.N_DAY):

        load_fc = ST.load_forecast(load_2d, DTYPE, d) if d >= 5 else load_2d[d]

        pv_fc = FC10[d, 0] if d >= 5 else pv_2d[d]

        if d >= 5:

            load_pool = ST.load_residual_pool(load_2d, DTYPE, d)

            pv_pool = ST.fc_residual_pool(FC10[:, 0], pv_2d, d, 0, eps=50.0)

            load_scen = ST.load_scenarios(load_pool, load_fc, N_SCEN, rng)

            pv_scen = np.clip(ST.sample_scenarios(pv_pool, N_SCEN, rng) * pv_fc, 0,
None)

            res0 = ST.solve_two_stage(price, load_fc, pv_fc, load_scen, pv_scen,
soc, None)

            plan = (res0["g_plan"] if res0 is not None
```

```
                    else np.maximum(load_fc * L.DT - pv_fc * L.DT, 0))
        else:
            plan = np.maximum(load_2d[d] * L.DT - pv_2d[d] * L.DT, 0)
        gp[d] = plan
        committed = plan.copy()
        for k in active_stages:
            t_dec = DECIDE_T[k]
            # 阶段初值取自全天结算轨迹
            soc_path = ST.soc_trajectory(price, committed, load_2d[d], pv_2d[d],
soc)
            soc_at = float(soc_path[0][t_dec - 1]) if soc_path is not None else soc
            ver = vint.get(k, k)
            pv_stage = (np.clip(FC10[d, ver] - BIAS[d, ver], 0.0, None)
                        if d >= 5 else pv_2d[d])
            # 只优化 [t_dec, 144)，负载用预报
            rk, ofs, m = L.solve_stage_lp(price, load_fc, pv_stage, plan, t_dec,
soc_at)
            if rk.success:
                committed[t_dec:] = rk.x[ofs["GA"]:ofs["GA"] + m]
        ga[d] = committed
        res = settle_day(committed, load_2d[d], pv_2d[d], soc)
```

```
        x = res.x
        cc[d] = x[N:2*N]
        dd[d] = x[2*N:3*N]
        ss[d] = x[3*N:4*N]
        EE[d] = x[4*N:5*N]
        ee[d] = x[5*N:6*N]
        soc = EE[d, -1]
    return gp, ga, cc, dd, ss, EE, ee




def cost_of(gp, ga, ee_):
    rp = slice(REP0, L.N_DAY)
    under = np.maximum(gp[rp] - ga[rp], 0)
    over = np.maximum(ga[rp] - gp[rp], 0)
    return {"base": float((price * np.minimum(gp[rp], ga[rp])).sum()),
            "def": float((price * L.BETA_DEF * under).sum()),
            "exc": float((price * L.BETA_EXC * over).sum()),
            "emg": float(L.BETA_EMG * (price * ee_[rp]).sum()),
            "u": float(under.sum()), "v": float(over.sum()), "e":
float(ee_[rp].sum()),
            "tot": (float((price * np.minimum(gp[rp], ga[rp])).sum())
```

```
                     + float((price * L.BETA_DEF * under).sum())
                     + float((price * L.BETA_EXC * over).sum())
                     + float(L.BETA_EMG * (price * ee_[rp]).sum()))}



t0 = time.time()
gp, ga, cc, dd, ss, EE, ee = run((1, 2, 3), verbose=True)
C = cost_of(gp, ga, ee)
print("全年滚动完成，用时 %.1f s" % (time.time() - t0))
print("基准购电 %.2f 元，违约 %.2f 元（少买 %.2f kWh）" % (C["base"], C["def"],
C["u"]))
print("超额 %.2f 元（多买 %.2f kWh），紧急购电 %.2f 元（紧急 %.2f kWh）"
      % (C["exc"], C["v"], C["emg"], C["e"]))
print("报告期合计 %.2f 元" % C["tot"])


sl = slice(REP0, L.N_DAY)
dates_r = dates.iloc[sl].reset_index(drop=True)
out = os.path.join(L.ROOT, "result3.xlsx")
shutil.copy(os.path.join(L.ATT, "附件5", "result3.xlsx"), out)
wb = openpyxl.load_workbook(out)
for name, mat in [("计划购电量", gp[sl]), ("调整购电量", ga[sl])]:
```

```
    ws = wb[name]
    for i in range(len(dates_r)):
        for t in range(N):
            ws.cell(row=2 + i, column=2 + t, value=float(round(mat[i, t], 6)))
        ws.cell(row=2 + i, column=146, value=float(round(mat[i].sum(), 6)))
        ws.cell(row=2 + i, column=147, value=float(round((price * mat[i]).sum(),
6)))
idx_cf = wb.sheetnames.index("充放电量")
del wb["充放电量"]
ws2 = wb.create_sheet("充放电量", idx_cf)
ws2.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
for i in range(len(dates_r)):
    chg = L.block_sums(cc[sl][i])
    dis = L.block_sums(dd[sl][i])
    b0 = 2 + i * 6
    for k, (name, _, _) in enumerate(L.BLOCKS):
        r = b0 + k
        if k == 0:
            ws2.cell(row=r, column=1, value=dates_r.iloc[i].to_pydatetime())
            ws2.cell(row=r, column=5, value=_dt.time(0, 0))
            ws2.cell(row=r, column=6, value=float(round(EE[REP0 - 1, -1], 6)))
```

```
        if k == 1:
            ws2.cell(row=r, column=5, value="24:00")
            ws2.cell(row=r, column=6, value=float(round(EE[sl][i, -1], 6)))
        ws2.cell(row=r, column=2, value=name)
        ws2.cell(row=r, column=3, value=float(round(chg[k], 6)))
        ws2.cell(row=r, column=4, value=float(round(dis[k], 6)))
idx_em = wb.sheetnames.index("紧急购电量")
del wb["紧急购电量"]
ws3 = wb.create_sheet("紧急购电量", idx_em)
ws3.append(["日期", "购电时间段", "购电量"])
row = 2
for i in range(len(dates_r)):
    nz = np.where(ee[sl][i] > 1e-6)[0]
    ws3.cell(row=row, column=1, value=dates_r.iloc[i].to_pydatetime())
    if len(nz) == 0:
        ws3.cell(row=row, column=2, value="无")
        ws3.cell(row=row, column=3, value=0.0)
        row += 1
    else:
        segs, s0 = [], nz[0]
```

```
        for a, b in zip(nz[:-1], nz[1:]):
            if b != a + 1:
                segs.append((s0, a))
                s0 = b
        segs.append((s0, nz[-1]))
        for j, (a, b) in enumerate(segs):
            if j > 0:
                ws3.cell(row=row, column=1, value=None)
            ws3.cell(row=row, column=2, value="%02d:%02d-%02d:%02d"
                     % ((a*10)//60, (a*10) % 60, ((b+1)*10)//60 % 24, ((b+1)*10) %
60))
            ws3.cell(row=row, column=3, value=float(round(ee[sl][i, a:b+1].sum(),
6)))
            row += 1
wb.save(out)
print("结果已写入", out)


for ds in KEY_DATES:
    di = int(np.where(dates_r == pd.Timestamp(ds))[0][0])
    print(ds, "紧急 %.2f kWh" % ee[sl][di].sum(), "计划 %.2f" % gp[sl][di].sum(),
          "调整 %.2f" % ga[sl][di].sum(), "偏差 %+.2f" % (ga[sl][di].sum() -
```

```
gp[sl][di].sum()))



# VOI 对照：固定决策时刻，只换预报版本

print("价值分解（各配置共用同一情景集）")

CASES = [

    ("A 仅0:00决策（不改签）", (), {}),

    ("B 三时刻改签，预报始终用0:00版", (1, 2, 3), {1: 0, 2: 0, 3: 0}),

    ("C1 仅6:00用0:00版预报", (1, 2, 3), {1: 0, 2: 2, 3: 3}),

    ("C2 仅12:00用6:00版预报", (1, 2, 3), {1: 1, 2: 1, 3: 3}),

    ("C3 仅18:00用12:00版预报", (1, 2, 3), {1: 1, 2: 2, 3: 2}),

]

MAIN = "C 三时刻改签，各用当期预报（本方案）"

vcost = {MAIN: C["tot"]}

for lab, act, vt in CASES:

    plan2, adj2, _, _, _, _, emg2 = run(act, vt)

    vcost[lab] = cost_of(plan2, adj2, emg2)["tot"]

    print(lab, "总费用 %.2f 元" % vcost[lab])



CA = vcost["A 仅0:00决策（不改签）"]

CB = vcost["B 三时刻改签，预报始终用0:00版"]
```

```
CC = vcost[MAIN]
print("决策机会价值 %.2f 元" % (CA - CB), "信息更新价值 %.2f 元" % (CB - CC),
      "净效应 %.2f 元" % (CA - CC))
print("分次信息价值（用上一版预报的代价）")
for tag, key in [("6:00 发布", "C1 仅6:00用0:00版预报"),
                 ("12:00 发布", "C2 仅12:00用6:00版预报"),
                 ("18:00 发布", "C3 仅18:00用12:00版预报")]:
    print(tag, "%+.2f 元" % (vcost[key] - CC))


np.savez(os.path.join(L.RES, "问3_结果.npz"),
         g_plan=gp[sl], g_adj=ga[sl], c=cc[sl], d=dd[sl], s=ss[sl], E=EE[sl],
e=ee[sl],
         price=price, dates=dates_r.values.astype("datetime64[D]"),
         load=load_2d[sl], pv=pv_2d[sl], fc10=FC10[sl], bias=BIAS[sl],
         C_base=C["base"], C_def=C["def"], C_exc=C["exc"], C_emg=C["emg"],
C_tot=C["tot"],
         C_A=CA, C_B=CB, C_C=CC,
         voi_info=np.array([vcost[k] for k in
                            ["C1 仅6:00用0:00版预报", "C2 仅12:00用6:00版预报",
                             "C3 仅18:00用12:00版预报"]]),
         voi_labels=np.array([k for k in vcost]),
```

```
         voi_costs=np.array([vcost[k] for k in vcost]))

print("已生成数据：结果/问3_结果.npz")
```

附录B-7 问题4求解.py

```
"""问题4 求解：实时电价下的两阶段随机规划与滚动决策。"""

import os, sys, time

import numpy as np

import pandas as pd

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

import openpyxl, shutil, datetime as dt



sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import lp_core as L

import stoch as ST



plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]

plt.rcParams["axes.unicode_minus"] = False

plt.rcParams["figure.dpi"] = 130
```

```
rng = np.random.default_rng(42)



N = L.N_T

N_SCEN = 20

N_SCEN_STAGE = 12

DECIDE_T = {1: 36, 2: 72, 3: 108}

REP0 = 31

KEY_DATES = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]



D = L.load_all(verbose=False)

price_mean = D["typical"]["price"]

price_rt = D["price_rt"]["price"]

load_2d = D["actual"]["load_kw"]

pv_2d = D["actual"]["pv_kw"]

FC = D["forecast"]["fc"]

dates = D["actual"]["dates"]

DTYPE = ST.day_type(dates)

fc10 = np.zeros((L.N_DAY, 4, N))

for d in range(L.N_DAY):

    for k in range(4):
```

```
        fc10[d, k] = L.forecast_to_10min(FC[d], k)

# 预报偏差订正表：扩张窗口，样本外

bias = L.forecast_bias_table(fc10, pv_2d)

avg_price = price_rt.mean(axis=0)

print("附件1 电价 vs 附件4 逐时段均值：最大绝对差 %.6f，相关系数 %.6f"

      % (np.abs(price_mean - avg_price).max(), np.corrcoef(price_mean,
avg_price)[0, 1]))




def settle_day(price, g_plan, load_kw, pv_kw, soc_start):

    """实时结算；储能跨日自由传递。"""

    n = N

    base = {k: i * n for i, k in enumerate(["GD", "C", "D", "S", "E", "E2"])}

    idx = np.arange(n)

    obj = np.zeros(6 * n)

    obj[base["E2"] + idx] = L.BETA_EMG * price

    obj[base["C"] + idx] = L.THRU_PEN

    obj[base["D"] + idx] = L.THRU_PEN

    from scipy.optimize import linprog

    from scipy.sparse import coo_matrix

    row, col, val, rhs = [], [], [], []
```

```
    for name, coef in [("GD", 1.0), ("E2", 1.0), ("D", 1.0), ("C", -1.0), ("S", -
1.0)]:
        row.append(idx)
        col.append(base[name] + idx)
        val.append(coef * np.ones(n))
    rhs.append(load_kw * L.DT - pv_kw * L.DT)
    bal = n + idx
    row.append(bal); col.append(base["E"] + idx); val.append(np.ones(n))
    row.append(bal); col.append(base["C"] + idx); val.append(-L.ETA_C * np.ones(n))
    row.append(bal); col.append(base["D"] + idx); val.append((1.0 / L.ETA_D) *
np.ones(n))
    row.append(bal[1:]); col.append(base["E"] + idx[1:] - 1); val.append(-np.ones(n
- 1))
    soc0 = np.zeros(n)
    soc0[0] = soc_start
    rhs.append(soc0)
    A_eq = coo_matrix((np.concatenate(val), (np.concatenate(row),
np.concatenate(col))),
                      shape=(2 * n, 6 * n)).tocsr()
    # g_draw <= g_plan 由变量上界施加
    lb = np.zeros(6 * n)
    ub = np.empty(6 * n)
```

```
    ub[base["GD"]:base["GD"] + n] = g_plan
    ub[base["C"]:base["C"] + n] = L.E_MAX
    ub[base["D"]:base["D"] + n] = L.E_MAX
    ub[base["S"]:base["S"] + n] = pv_kw * L.DT
    lb[base["E"]:base["E"] + n] = L.SOC_MIN
    ub[base["E"]:base["E"] + n] = L.SOC_MAX
    ub[base["E2"]:base["E2"] + n] = np.inf
    return linprog(obj, A_eq=A_eq, b_eq=np.concatenate(rhs),
                   bounds=list(zip(lb, ub)), method="highs")



def run_42(price2d):
    """问题4-2：两阶段随机规划（逐日 0:00 定计划，实时结算）"""
    plan_2d = np.zeros((L.N_DAY, N))
    chg_2d = np.zeros((L.N_DAY, N))
    dis_2d = np.zeros((L.N_DAY, N))
    spill_2d = np.zeros((L.N_DAY, N))
    soc_2d = np.zeros((L.N_DAY, N))
    emg_2d = np.zeros((L.N_DAY, N))
    soc = L.SOC_INIT
```

```
    for d in range(L.N_DAY):
        price_day = price2d[d]
        load_fc = ST.load_forecast(load_2d, DTYPE, d) if d >= 5 else load_2d[d]
        pv_fc = ST.pv_forecast_persistence(pv_2d, DTYPE, d) if d >= 5 else pv_2d[d]
        if d >= 5:
            load_res = ST.load_residual_pool(load_2d, DTYPE, d)
            pv_res = ST.pv_residual_pool_persist(pv_2d, DTYPE, d)
            load_scen = ST.load_scenarios(load_res, load_fc, N_SCEN, rng)
            pv_scen = np.clip(ST.sample_scenarios(pv_res, N_SCEN, rng) * pv_fc, 0,
None)
            two_stage = ST.solve_two_stage(price_day, load_fc, pv_fc, load_scen,
pv_scen, soc, None)
            plan = (two_stage["g_plan"] if two_stage is not None
                    else np.maximum(load_fc * L.DT - pv_fc * L.DT, 0))
        else:
            plan = np.maximum(load_2d[d] * L.DT - pv_2d[d] * L.DT, 0)
        plan_2d[d] = plan
        settled = settle_day(price_day, plan, load_2d[d], pv_2d[d], soc)
        x = settled.x
        chg_2d[d] = x[N:2*N]
        dis_2d[d] = x[2*N:3*N]
```

```
        spill_2d[d] = x[3*N:4*N]
        soc_2d[d] = x[4*N:5*N]
        emg_2d[d] = x[5*N:6*N]
        soc = soc_2d[d, -1]
        if (d + 1) % 60 == 0:
            print("4-2 已完成 %3d/%d 天" % (d + 1, L.N_DAY))
    return plan_2d, chg_2d, dis_2d, spill_2d, soc_2d, emg_2d



def run_43(price2d, active=(1, 2, 3)):
    """问题4-3：情景化滚动随机决策"""
    plan_2d = np.zeros((L.N_DAY, N))
    adj_2d = np.zeros((L.N_DAY, N))
    chg_2d = np.zeros((L.N_DAY, N))
    dis_2d = np.zeros((L.N_DAY, N))
    spill_2d = np.zeros((L.N_DAY, N))
    soc_2d = np.zeros((L.N_DAY, N))
    emg_2d = np.zeros((L.N_DAY, N))
    soc = L.SOC_INIT
    for d in range(L.N_DAY):
```

```
        price_day = price2d[d]
        load_fc = ST.load_forecast(load_2d, DTYPE, d) if d >= 5 else load_2d[d]
        pv_fc = fc10[d, 0] if d >= 5 else pv_2d[d]
        if d >= 5:
            load_res = ST.load_residual_pool(load_2d, DTYPE, d)
            pv_res = ST.fc_residual_pool(fc10[:, 0], pv_2d, d, 0, eps=50.0)
            load_scen = ST.load_scenarios(load_res, load_fc, N_SCEN, rng)
            pv_scen = np.clip(ST.sample_scenarios(pv_res, N_SCEN, rng) * pv_fc, 0,
None)
            two_stage = ST.solve_two_stage(price_day, load_fc, pv_fc, load_scen,
pv_scen, soc, None)
            plan = (two_stage["g_plan"] if two_stage is not None
                    else np.maximum(load_fc * L.DT - pv_fc * L.DT, 0))
        else:
            plan = np.maximum(load_2d[d] * L.DT - pv_2d[d] * L.DT, 0)
        plan_2d[d] = plan
        committed = plan.copy()
        for k in active:
            t_decide = DECIDE_T[k]
            # 阶段初值取自全天结算储能轨迹
            soc_traj = ST.soc_trajectory(price_day, committed, load_2d[d],
```

```
pv_2d[d], soc)
            soc_at = float(soc_traj[0][t_decide - 1]) if soc_traj is not None else
soc
            pv_stage = np.clip(fc10[d, k] - bias[d, k], 0.0, None) if d >= 5 else
pv_2d[d]
            stage, stage_map, stage_n = L.solve_stage_lp(price_day, load_fc,
pv_stage, plan,
                                                         t_decide, soc_at)
            if stage.success:
                committed[t_decide:] = stage.x[stage_map["GA"]:stage_map["GA"] +
stage_n]
        adj_2d[d] = committed
        soc_traj = ST.soc_trajectory(price_day, committed, load_2d[d], pv_2d[d],
soc)
        soc_2d[d], chg_2d[d], dis_2d[d], spill_2d[d], emg_2d[d] = soc_traj
        soc = soc_2d[d, -1]
        if (d + 1) % 60 == 0:
            print("4-3 已完成 %3d/%d 天" % (d + 1, L.N_DAY))
    return plan_2d, adj_2d, chg_2d, dis_2d, spill_2d, soc_2d, emg_2d




t42 = time.time()
```

```
plan42, chg42, dis42, spill42, soc42, emg42 = run_42(price_rt)

print("4-2 完成，用时 %.1f s" % (time.time() - t42))

report = slice(REP0, L.N_DAY)

cost42 = float((price_rt[report] * plan42[report]).sum()

               + L.BETA_EMG * (price_rt[report] * emg42[report]).sum())

print("4-2 报告期费用 %.2f 元（紧急 %.2f kWh）" % (cost42, emg42[report].sum()))



t43 = time.time()

plan43, adj43, chg43, dis43, spill43, soc43, emg43 = run_43(price_rt)

print("4-3 完成，用时 %.1f s" % (time.time() - t43))

short43 = np.maximum(plan43[report] - adj43[report], 0)

over43 = np.maximum(adj43[report] - plan43[report], 0)

parts = {

    "base": float((price_rt[report] * np.minimum(plan43[report],
adj43[report])).sum()),

    "def": float((price_rt[report] * L.BETA_DEF * short43).sum()),

    "exc": float((price_rt[report] * L.BETA_EXC * over43).sum()),

    "emg": float(L.BETA_EMG * (price_rt[report] * emg43[report]).sum())}

cost43 = sum(parts.values())

print("4-3 报告期分解：基准 %.2f，违约 %.2f，超额 %.2f，紧急 %.2f，合计 %.2f"

      % (parts["base"], parts["def"], parts["exc"], parts["emg"], cost43))
```

```
def write_result(tpl_name, out_name, sheets):
    shutil.copy(os.path.join(L.ATT, "附件5", tpl_name), os.path.join(L.ROOT,
out_name))
    wb = openpyxl.load_workbook(os.path.join(L.ROOT, out_name))
    for name, mat in sheets.get("plan", []):
        ws = wb[name]
        for i in range(mat.shape[0]):
            for t in range(N):
                ws.cell(row=2 + i, column=2 + t, value=float(round(mat[i, t], 6)))
            ws.cell(row=2 + i, column=146, value=float(round(mat[i].sum(), 6)))
            ws.cell(row=2 + i, column=147,
                    value=float(round((price_rt[report][i] * mat[i]).sum(), 6)))
    cf_pos = wb.sheetnames.index("充放电量")
    del wb["充放电量"]
    ws2 = wb.create_sheet("充放电量", cf_pos)
    ws2.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
    rep_dates = dates.iloc[report].reset_index(drop=True)
    for i in range(len(rep_dates)):
        chg = L.block_sums(sheets["c"][i])
```

```
        dis = L.block_sums(sheets["d"][i])
        first_row = 2 + i * 6
        for k, (block_name, _, _) in enumerate(L.BLOCKS):
            blk_row = first_row + k
            if k == 0:
                ws2.cell(row=blk_row, column=1,
value=rep_dates.iloc[i].to_pydatetime())
                ws2.cell(row=blk_row, column=5, value=dt.time(0, 0))
                ws2.cell(row=blk_row, column=6, value=float(round(
                    L.SOC_INIT if i == 0 and REP0 == 0 else sheets["E"][max(i - 1,
0), -1], 6)))
            if k == 1:
                ws2.cell(row=blk_row, column=5, value="24:00")
                ws2.cell(row=blk_row, column=6, value=float(round(sheets["E"][i, -
1], 6)))
            ws2.cell(row=blk_row, column=2, value=block_name)
            ws2.cell(row=blk_row, column=3, value=float(round(chg[k], 6)))
            ws2.cell(row=blk_row, column=4, value=float(round(dis[k], 6)))
    em_pos = wb.sheetnames.index("紧急购电量")
    del wb["紧急购电量"]
    ws3 = wb.create_sheet("紧急购电量", em_pos)
```

```
    ws3.append(["日期", "购电时间段", "购电量"])

    excel_row = 2

    for i in range(len(rep_dates)):

        active = np.where(sheets["e"][i] > 1e-6)[0]

        ws3.cell(row=excel_row, column=1, value=rep_dates.iloc[i].to_pydatetime())

        if len(active) == 0:

            ws3.cell(row=excel_row, column=2, value="无")

            ws3.cell(row=excel_row, column=3, value=0.0)

            excel_row += 1

        else:

            runs = []

            start = active[0]

            for a, b in zip(active[:-1], active[1:]):

                if b != a + 1:

                    runs.append((start, a))

                    start = b

            runs.append((start, active[-1]))

            for j, (a, b) in enumerate(runs):

                if j > 0:

                    ws3.cell(row=excel_row, column=1, value=None)
```

```
                ws3.cell(row=excel_row, column=2, value="%02d:%02d-%02d:%02d"
                         % ((a*10)//60, (a*10) % 60, ((b+1)*10)//60 % 24,
((b+1)*10) % 60))
                ws3.cell(row=excel_row, column=3,
                         value=float(round(sheets["e"][i, a:b+1].sum(), 6)))
                excel_row += 1
    wb.save(os.path.join(L.ROOT, out_name))
    print("已写入", out_name)



write_result("result4-2.xlsx", "result4-2.xlsx",
             {"plan": [("计划购电量", plan42[report])], "c": chg42[report], "d":
dis42[report],
              "E": soc42[report], "e": emg42[report]})
write_result("result4-3.xlsx", "result4-3.xlsx",
             {"plan": [("计划购电量", plan43[report]), ("调整购电量",
adj43[report])],
              "c": chg43[report], "d": dis43[report], "E": soc43[report], "e":
emg43[report]})


# 方案A 按实时电价完整结算
res2 = np.load(os.path.join(L.RES, "问2_结果.npz"), allow_pickle=True)
```

```
plan2 = res2["g_all"]

soc_a = L.SOC_INIT

buy_a = 0.0

emg_a = 0.0

for d in range(REP0, L.N_DAY):

    lp_a = settle_day(price_rt[d], plan2[d], load_2d[d], pv_2d[d], soc_a)

    soc_a = lp_a.x[4 * N + N - 1]

    buy_a += float(price_rt[d] @ plan2[d])

    emg_a += float(L.BETA_EMG * (price_rt[d] @ lp_a.x[5 * N:6 * N]))

cost_det_rt = buy_a + emg_a

regret = cost_det_rt - cost42

print("方案A 确定性等价电价规划 + 实时电价完整结算 = 购电 %.2f + 紧急 %.2f = %.2f
元"

      % (buy_a, emg_a, cost_det_rt))

print("方案B 实时电价直接优化（问题4-2）= %.2f 元" % cost42)

print("后悔值 %.2f 元（%.2f%%）" % (regret, regret / cost_det_rt * 100))



np.savez(os.path.join(L.RES, "问4_结果.npz"),

         g42=plan42[report], e42=emg42[report], c42=chg42[report],
d42=dis42[report],

         E42=soc42[report],
```

```
          g43_plan=plan43[report], g43_adj=adj43[report], e43=emg43[report],
c43=chg43[report],

          d43=dis43[report], E43=soc43[report],

          price_rt=price_rt[report], price_mean=price_mean,

          dates=dates.iloc[report].values.astype("datetime64[D]"),

          C42=cost42, C43=cost43, C_det_in_rt=cost_det_rt, regret=regret,
cb43=parts)

print("已生成数据：结果/问4_结果.npz")
```

附录B-8 敏感性分析.py

```
"""四问结果统计描述与关键参数敏感性分析。"""

import os, sys, time

import numpy as np

import pandas as pd

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt



sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import lp_core as L
```

```
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]

plt.rcParams["axes.unicode_minus"] = False

plt.rcParams["figure.dpi"] = 130



N = L.N_T







def desc(name, x, unit=""):

    x = np.asarray(x, float)

    print(name, "均值", "%.2f" % x.mean(), "标准差", "%.2f" % x.std(),

          "最小", "%.2f" % x.min(), "最大", "%.2f" % x.max(),

          "变异系数", "%.3f" % (x.std()/x.mean() if x.mean() != 0 else
float("nan")), unit)







data = L.load_all(verbose=False)

q1 = np.load(os.path.join(L.RES, "问1_结果.npz"), allow_pickle=True)

q2 = np.load(os.path.join(L.RES, "问2_结果.npz"), allow_pickle=True)

q3 = np.load(os.path.join(L.RES, "问3_结果.npz"), allow_pickle=True)

q4 = np.load(os.path.join(L.RES, "问4_结果.npz"), allow_pickle=True)
```

```
buy1, chg1, dis1, spill1, soc1 = q1["g"], q1["c"], q1["d"], q1["s"], q1["E"]

desc("购电量 g_t (kWh)", buy1)

desc("充电量 c_t (kWh)", chg1)

desc("放电量 d_t (kWh)", dis1)

desc("储电量 E_t (kWh)", soc1)

dual = q1["dual"]

price_typ = q1["price"]

same = np.isclose(dual, price_typ, rtol=1e-6, atol=1e-8)

print("问1 不购电时段", int((buy1 < 1e-6).sum()), "充电时段", int((chg1 > 1e-
6).sum()),

      "放电时段", int((dis1 > 1e-6).sum()),

      "空闲时段", int(((chg1 < 1e-6) & (dis1 < 1e-6)).sum()),

      "SOC 触上界", int(np.isclose(soc1, L.SOC_MAX).sum()),

      "触下界", int(np.isclose(soc1, L.SOC_MIN).sum()))

print("问1 SOC 利用率", "%.1f%%" % ((soc1.max()-soc1.min())/(L.SOC_MAX-
L.SOC_MIN)*100),

      "对偶等于电价时段", int(same.sum()), "对偶低于电价时段", int((dual <
price_typ - 1e-8).sum()),

      "其中正在放电", int(((dual < price_typ - 1e-8) & (dis1 > 1e-6)).sum()))



price2 = q2["price"]
```

```
buy2 = q2["g"]

soc2 = q2["E"]

# datetime64 包成 Series 后用 .dt 访问器

dates2 = pd.Series(pd.to_datetime(q2["dates"]))

cost_day = (price2 * buy2).sum(axis=1)

buy_day = buy2.sum(axis=1)

chg_day = q2["c"].sum(axis=1)

dis_day = q2["d"].sum(axis=1)

spill_day = q2["s"].sum(axis=1)

desc("日购电费（元)", cost_day)

desc("日购电量（kWh)", buy_day)

print("问2 报告期总费用", "%.2f" % cost_day.sum(), "元", "日均", "%.2f" %
cost_day.mean(),

      "极差比", "%.2f" % (cost_day.max()/cost_day.min()),

      "总充放", "%.0f/%.0f" % (chg_day.sum(), dis_day.sum()), "总弃光", "%.1f" %
spill_day.sum(),

      "分位数 P5/P25/P50/P75/P95",

      "%.0f %.0f %.0f %.0f %.0f" % tuple(np.percentile(cost_day, [5, 25, 50, 75,
95])))

# 等效循环 = 日放电量 / SOC 可用跨度

cycle = dis_day / (L.SOC_MAX - L.SOC_MIN)
```

```
soc_dawn = np.r_[L.SOC_INIT, soc2[:-1, -1]]

# 实时结算按当日求解，末端取下限

mth = dates2.dt.month.values

print("问2 日等效循环", "%.3f" % cycle.mean(), "每日0:00储电量 均值", "%.1f" %
soc_dawn.mean(),

      "最小", "%.1f" % soc_dawn.min(), "最大", "%.1f" % soc_dawn.max(),

      "恰等于下界 %.0f 的天数" % L.SOC_MIN,

      int((np.abs(soc2[:, -1] - L.SOC_MIN) < 1e-6).sum()), "/", len(soc2),

      "月度费用(万元)", [round(cost_day[mth == m].sum()/10000, 1) for m in range(2,
13)],

      "月度购电量(MWh)", [round(buy_day[mth == m].sum()/100, 0) for m in range(2,
13)])



g_plan, g_adj, emg = q3["g_plan"], q3["g_adj"], q3["e"]

short_buy = np.maximum(g_plan - g_adj, 0)

over_buy = np.maximum(g_adj - g_plan, 0)

desc("紧急购电量 e (kWh/段)", emg)

print("问3 非零少买段", int((short_buy > 1e-6).sum()), "非零多买段",
int((over_buy > 1e-6).sum()),

      "非零紧急段", int((emg > 1e-6).sum()), "/", short_buy.size,

      "紧急天数", int((emg.sum(axis=1) > 1e-6).sum()), "/", emg.shape[0],

      "调整量占计划量", "%.3f%%" % (g_adj.sum()/g_plan.sum()*100),
```

```
      "日均总偏差", "%.1f" % np.abs(g_adj - g_plan).sum(axis=1).mean())



g42 = q4["g42"]

price_rt = q4["price_rt"]

cost42_day = (price_rt * g42).sum(axis=1)

buy_all = q2["g_all"]

cost_a_day = (price_rt * buy_all[31:]).sum(axis=1)

regret_day = cost_a_day - cost42_day

desc("4-2 日购电费（元）", cost42_day)

desc("逐日后悔值（元）", regret_day)

# 0 维 object 数组用 .item() 取出

cost43 = q4["cb43"].item()

print("问4 后悔值为正", int((regret_day > 0).sum()), "天", "为负", int((regret_day
< 0).sum()), "天",

      "中位数", "%.2f" % np.median(regret_day), "总和", "%.2f" % regret_day.sum(),

      "均值/标准差", "%.2f / %.2f" % (regret_day.mean(), regret_day.std()),

      "4-3 费用分解 基准/违约/超额/紧急",

      "%.2f / %.2f / %.2f / %.2f" % (cost43["base"], cost43["def"], cost43["exc"],
cost43["emg"]),

      "紧急购电", "%.2f -> %.2f kWh" % (emg.sum(), q4["e43"].sum()),

      "下降", "%.1f%%" % ((1 - q4["e43"].sum()/emg.sum())*100))
```

```
price2_flat = price2
net = None
g1_base = buy1.copy()



def q1_solve(eta=None, price=None, pmax=None, soc_half=None):
    """按指定参数重解问1，返回（费用，购电量，放电量）。"""
    if eta is not None:
        L.ETA_C = eta
        L.ETA_D = eta
    old_pm, old_lo, old_hi = L.E_MAX, L.SOC_MIN, L.SOC_MAX
    if pmax is not None:
        L.E_MAX = pmax * L.DT
    if soc_half is not None:
        L.SOC_MIN = L.SOC_INIT - soc_half
        L.SOC_MAX = L.SOC_INIT + soc_half
    price_run = price if price is not None else q1["price"]
    res = L.solve_day_lp(price_run, q1["load_kw"], q1["pv_kw"],
                         soc_start=L.SOC_INIT, soc_end=L.SOC_INIT)
```

```
    out = (res["cost"], res["g"].sum(), res["d"].sum())
    L.ETA_C = 0.9 ** 0.5
    L.ETA_D = 0.9 ** 0.5
    L.E_MAX = old_pm
    L.SOC_MIN = old_lo
    L.SOC_MAX = old_hi
    return out



base_cost1, base_g1, base_d1 = q1_solve()
price0 = q1["price"]
price_mean = price0.mean()
cost_a = q1_solve(eta=0.9)
print("S1 充放电效率 往返0.90（本文采用）", "%.2f" % base_cost1, "元", "往返0.81",
"%.2f" % cost_a[0],
      "元", "相对偏差", "%+.2f%%" % ((cost_a[0] - base_cost1) / base_cost1 * 100))
q1_solve()


s2 = []
for k in (0.5, 0.75, 1.0, 1.25, 1.5):
    price_s = price_mean + k * (price0 - price_mean)
```

```
    cost_s, buy_s, dis_s = q1_solve(price=price_s)

    s2.append((k, cost_s, buy_s, dis_s))

print("S2 价差倍数 0.5/1.0/1.5 的单日费用", "%.2f" % s2[0][1], "%.2f" % s2[2][1],
"%.2f" % s2[4][1],

      "元", "相对偏差 %+.2f%%" % ((s2[0][1] - base_cost1) / base_cost1 * 100))

q1_solve()



s3 = []

for k in (0.25, 0.5, 0.75, 1.0, 1.5):

    cost_s, buy_s, dis_s = q1_solve(pmax=L.P_MAX * k)

    s3.append((k, cost_s, buy_s, dis_s))

print("S3 电池功率 0.5x/1.0x 的单日费用", "%.2f" % s3[1][1], "%.2f" % s3[3][1], "元
",

      "相对偏差 %+.2f%%" % ((s3[1][1] - base_cost1) / base_cost1 * 100))

q1_solve()



s4 = []

for k in (0.25, 0.5, 0.75, 1.0):

    cost_s, buy_s, dis_s = q1_solve(soc_half=4800 * k)

    s4.append((k, cost_s, buy_s, dis_s))

print("S4 可用区间半宽 0.5x/1.0x 的单日费用", "%.2f" % s4[1][1], "%.2f" % s4[3][1],
```

```
"元",
      "相对偏差 %+.2f%%" % ((s4[1][1] - base_cost1) / base_cost1 * 100))
q1_solve()


for k in (0.8, 1.0, 1.2):
    cost_s, buy_s, dis_s = q1_solve(price=price0 * k)
    print("S6 电价倍数", "%.2f" % k, "费用", "%.2f" % cost_s, "购电量", "%.0f" %
buy_s,
          "费用/倍数", "%.2f" % (cost_s / k))
q1_solve()
# 价格齐次性：费用随电价等比，购电量不变


# 每次全年求解约 6 s
price_2d = np.tile(q2["price"], (L.N_DAY, 1))
load_2d = data["actual"]["load_kw"]
pv_2d = data["actual"]["pv_kw"]
res2 = []
t0 = time.time()
res_base = L.solve_multiday_lp(price_2d, load_2d, pv_2d)
res2.append(("往返 0.90（本文采用）", res_base["cost"]))
L.ETA_C = L.ETA_D = 0.9
```

```
t0 = time.time()

res_a = L.solve_multiday_lp(price_2d, load_2d, pv_2d)

res2.append(("往返 0.81", res_a["cost"]))

L.ETA_C = L.ETA_D = 0.9 ** 0.5

# 截断到最后 11 天前，估计边界效应

sl_last = slice(0, 354)

r_trunc = L.solve_multiday_lp(price_2d[:354], load_2d[:354], pv_2d[:354])

print("问2 全年 往返0.90", "%.2f" % res_base["cost"], "往返0.81", "%.2f" %
res_a["cost"],

      "元", "相对偏差", "%+.2f%%" % ((res_a["cost"] - res_base["cost"]) /
res_base["cost"] * 100),

      "截到 354 天", "%.2f" % r_trunc["cost"], "元",

      "年末可提前变现", "%.0f" % (L.SOC_MAX - L.SOC_MIN), "kWh")



fig, axes = plt.subplots(2, 2, figsize=(13, 8.6))

k_s2 = [x[0] for x in s2]

cost_s2 = [x[1]/1000 for x in s2]

axes[0, 0].plot(k_s2, cost_s2, marker="o", color="#2E6F9E", linewidth=2.0)

axes[0, 0].set_xlabel("峰谷价差倍数 k")

axes[0, 0].set_ylabel("单日购电费（千元）")

axes[0, 0].set_title("（a）费用对峰谷价差的敏感性", fontsize=11.5)
```

```
axes[0, 0].grid(alpha=0.25, linestyle=":")

for x, y in zip(k_s2, cost_s2):

    axes[0, 0].annotate("%.1f" % y, (x, y), ha="center", va="bottom", fontsize=9)



k_s3 = [x[0]*L.P_MAX for x in s3]

cost_s3 = [x[1]/1000 for x in s3]

axes[0, 1].plot(k_s3, cost_s3, marker="s", color="#3F8F5B", linewidth=2.0)

axes[0, 1].axvline(L.P_MAX, color="#D64545", linestyle="--", linewidth=1.3, label="
基准 5000 kW")

axes[0, 1].set_xlabel("最大充放电功率 (kW)")

axes[0, 1].set_ylabel("单日购电费 (千元)")

axes[0, 1].set_title(" (b) 费用对电池功率的敏感性", fontsize=11.5)

axes[0, 1].legend(fontsize=9)

axes[0, 1].grid(alpha=0.25, linestyle=":")



k_s4 = [x[0]*4800 for x in s4]

cost_s4 = [x[1]/1000 for x in s4]

axes[1, 0].plot(k_s4, cost_s4, marker="^", color="#C77D2B", linewidth=2.0)

axes[1, 0].axvline(4800, color="#D64545", linestyle="--", linewidth=1.3, label="基
准 4800 kWh")

axes[1, 0].set_xlabel("SOC 可用区间半宽 (kWh)")
```

```
axes[1, 0].set_ylabel("单日购电费（千元）")
axes[1, 0].set_title("（c）费用对电池容量的敏感性", fontsize=11.5)
axes[1, 0].legend(fontsize=9)
axes[1, 0].grid(alpha=0.25, linestyle=":")

labs = ["往返0.90\n(本文采用)", "往返0.81"]
vals = [res2[0][1]/1e4, res2[1][1]/1e4]
axes[1, 1].bar(labs, vals, color=["#2E6F9E", "#9EC4DE"], width=0.5)
for i, v in enumerate(vals):
    axes[1, 1].annotate("%.1f" % v, (i, v), ha="center", va="bottom", fontsize=10)
axes[1, 1].set_ylabel("全年费用（万元）")
axes[1, 1].set_title("（d）全年费用对效率口径的敏感性", fontsize=11.5)
axes[1, 1].grid(axis="y", alpha=0.25, linestyle=":")
fig.suptitle("关键参数敏感性分析", fontsize=13)
fig.tight_layout(rect=[0, 0, 1, 0.95])
fig.savefig(os.path.join(L.IMG, "图21_参数敏感性分析.png"), bbox_inches="tight")
plt.close(fig)

fig, axes = plt.subplots(1, 3, figsize=(15.5, 4.3))
axes[0].hist(buy1, bins=40, color="#2E6F9E", alpha=0.85, edgecolor="white")
```

```
axes[0].set_xlabel("分段购电量（kWh）")

axes[0].set_ylabel("频数")

axes[0].set_title("（a）购电量分布\n(%.1f%% 的时段不购电)" % ((buy1 < 1e-
6).mean()*100), fontsize=11)

axes[0].grid(alpha=0.25, linestyle=":")



axes[1].plot(np.arange(1, N+1)*L.DT, dual, color="#D64545", linewidth=1.4, label="
对偶变量")

axes[1].plot(np.arange(1, N+1)*L.DT, price_typ, color="#2E6F9E", linewidth=1.6,
label="实际电价")

axes[1].set_xlabel("时刻（小时）")

axes[1].set_ylabel("元/kWh")

axes[1].set_title("（b）边际电价与购电价的缺口\n(缺口=电池的机会成本)",
fontsize=11)

axes[1].legend(fontsize=9)

axes[1].grid(alpha=0.25, linestyle=":")



gd = pd.to_datetime(q2["dates"])

mc = dates2.dt.month

axes[2].boxplot([cost_day[mc == m]/1000 for m in range(2, 13)], patch_artist=True,

                flierprops=dict(marker="o", markerfacecolor="#D64545",
markersize=3, alpha=0.5),
```

```
                medianprops=dict(color="#222222", linewidth=1.4),
                boxprops=dict(facecolor="#9EC4DE", alpha=0.65))
axes[2].set_xticklabels(["2","3","4","5","6","7","8","9","10","11","12"],
fontsize=9)
axes[2].set_xlabel("月份")
axes[2].set_ylabel("日购电费（千元）")
axes[2].set_title("（c）日购电费的分月分布", fontsize=11)
axes[2].grid(axis="y", alpha=0.25, linestyle=":")
fig.suptitle("求解结果的分布特征", fontsize=12.5)
fig.tight_layout(rect=[0, 0, 1, 0.93])
fig.savefig(os.path.join(L.IMG, "图22_结果分布特征.png"), bbox_inches="tight")
plt.close(fig)

print("敏感性与统计分析完成，已生成 图21_参数敏感性分析.png / 图22_结果分布特
征.png")
```

附录B-9 模型检验.py

```
"""模型检验：可行性、算法一致性、对偶与边界检验。"""
import os, sys, time
import numpy as np
import pandas as pd
```

```
import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

from scipy.optimize import linprog

from scipy.sparse import coo_matrix



sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import lp_core as L



plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]

plt.rcParams["axes.unicode_minus"] = False

plt.rcParams["figure.dpi"] = 130

rng = np.random.default_rng(42)



N = L.N_T



data = L.load_all(verbose=False)

price_typ = data["typical"]["price"]

load_typ = data["typical"]["load_kw"]

pv_typ = data["typical"]["pv_kw"]
```

```
base = L.solve_day_lp(price_typ, load_typ, pv_typ, soc_start=L.SOC_INIT,
soc_end=L.SOC_INIT)

assert base["status"] == 0

buy, charge, discharge, spill, soc = base["g"], base["c"], base["d"], base["s"],
base["E"]

cost_base = base["cost"]

print("基准最优费用", "%.2f" % cost_base, "元")



soc_prev = np.r_[L.SOC_INIT, soc[:-1]]

bal_err = buy + pv_typ * L.DT + discharge - (load_typ * L.DT + charge + spill)

soc_err = soc - (soc_prev + L.ETA_C * charge - discharge / L.ETA_D)

soc_hi_err = np.maximum(soc - L.SOC_MAX, 0)

soc_lo_err = np.maximum(L.SOC_MIN - soc, 0)

chg_err = np.maximum(charge - L.E_MAX, 0)

dis_err = np.maximum(discharge - L.E_MAX, 0)

spill_err = np.maximum(spill - pv_typ * L.DT, 0)

neg_err = np.maximum(-np.minimum.reduce([buy, charge, discharge, spill]), 0)

term_err = abs(soc[-1] - L.SOC_INIT)



checks = [("母线功率平衡", np.abs(bal_err).max()),
```

```
          ("SOC 递推方程", np.abs(soc_err).max()),
          ("SOC 上界", soc_hi_err.max()),
          ("SOC 下界", soc_lo_err.max()),
          ("充电功率上限", chg_err.max()),
          ("放电功率上限", dis_err.max()),
          ("弃光上界", spill_err.max()),
          ("变量非负性", neg_err.max()),
          ("末端条件", term_err)]
ok_all = True
for name, v in checks:
    flag = "通过" if v < 1e-8 else "违反"
    if v >= 1e-8:
        ok_all = False
        print(name, "最大违反量", "%.3e" % v, flag)
print("可行性检验最大违反量", "%.3e" % max(v for _, v in checks),
      "全部约束满足" if ok_all else "存在约束违反")



def solve_with(method, price, load_kw, pv_kw, soc_end=L.SOC_INIT):
    """指定算法求解单日 LP，用于交叉验证。"""
```

```
    n = N

    c_obj = np.concatenate([price, np.zeros(4 * n)])

    rows, cols, vals, b_eq = [], [], [], []

    for t in range(n):

        r = t

        rows += [r, r, r, r]

        cols += [t, n + t, 2 * n + t, 3 * n + t]

        vals += [1.0, -1.0, 1.0, -1.0]

        b_eq.append(load_kw[t] * L.DT - pv_kw[t] * L.DT)

    for t in range(n):

        r = n + t

        rows.append(r)

        cols.append(4 * n + t)

        vals.append(1.0)

        if t == 0:

            b_eq.append(L.SOC_INIT)

        else:

            rows.append(r)

            cols.append(4 * n + t - 1)

            vals.append(-1.0)
```

```
            b_eq.append(0.0)
        rows.append(r)
        cols.append(n + t)
        vals.append(-L.ETA_C)
        rows.append(r)
        cols.append(2 * n + t)
        vals.append(1.0 / L.ETA_D)
    A_eq = coo_matrix((vals, (rows, cols)), shape=(2 * n, 5 * n)).tocsr()
    lb = np.zeros(5 * n)
    ub = np.empty(5 * n)
    ub[0:n] = np.inf
    ub[n:2*n] = L.E_MAX
    ub[2*n:3*n] = L.E_MAX
    ub[3*n:4*n] = pv_kw * L.DT
    ub[4*n:5*n] = L.SOC_MAX
    lb[4*n:5*n] = L.SOC_MIN
    if soc_end is not None:
        ub[5*n-1] = soc_end
        lb[5*n-1] = soc_end
    return linprog(c_obj, A_eq=A_eq, b_eq=np.array(b_eq),
```

```
                    bounds=list(zip(lb, ub)), method=method)




r_ds = solve_with("highs-ds", price_typ, load_typ, pv_typ)

r_ipm = solve_with("highs-ipm", price_typ, load_typ, pv_typ)

gap = abs(r_ds.fun - r_ipm.fun)

rel = gap / cost_base * 100

print("对偶单纯形", "%.6f" % r_ds.fun, "内点法", "%.6f" % r_ipm.fun,

      "lp_core 默认", "%.6f" % cost_base)

print("两法目标值之差", "%.3e" % gap, "相对偏差", "%.2e%%" % rel,

      "购电策略最大逐段差异", "%.3e" % np.abs(r_ds.x[:N] - r_ipm.x[:N]).max())

print("算法交叉验证", "通过" if rel < 1e-6 else "未通过")



res = linprog(price_typ.tolist() + [0.0] * (4 * N), method="highs", **{})

dual = base["dual_balance"]

slack_chg = L.E_MAX - charge

slack_dis = L.E_MAX - discharge

slack_spill = pv_typ * L.DT - spill

slack_soc_hi = L.SOC_MAX - soc

slack_soc_lo = soc - L.SOC_MIN
```

```
print("紧约束时段数 充电", int((slack_chg < 1e-8).sum()),

      "放电", int((slack_dis < 1e-8).sum()), "弃光", int((slack_spill < 1e-
8).sum()),

      "SOC 上界", int((slack_soc_hi < 1e-6).sum()), "SOC 下界", int((slack_soc_lo <
1e-6).sum()))

# 互补松弛检验对象

idle = (charge < 1e-8) & (discharge < 1e-8)

interior = (soc > L.SOC_MIN + 1e-6) & (soc < L.SOC_MAX - 1e-6)

pred = idle & interior

hit = np.isclose(dual[pred], price_typ[pred], rtol=1e-6, atol=1e-8)

print("电池空闲且 SOC 未触界的时段", int(pred.sum()), "其中对偶等于电价",
int(hit.sum()),

      "KKT 条件", "成立" if pred.sum() and hit.all() else "需人工复核")



# 下界：只保留全天总净需求约束

net = (load_typ - pv_typ) * L.DT

total_net = net.sum()

lb = 0.0

if total_net >= 0:

    lb = float(np.min(price_typ) * total_net)

else:
```

```
    lb = float(np.min(price_typ) * total_net)

net_pos = np.maximum(net, 0)

cost_ub = float((price_typ * net_pos).sum())

inside = (lb - 1e-6) <= cost_base <= (cost_ub + 1e-6)

print("下界", "%.2f" % lb, "模型最优解", "%.2f" % cost_base, "上界", "%.2f" %
cost_ub, "元",

      "落在区间内" if inside else "未通过")

print("相对上界改进", "%.2f" % (cost_ub - cost_base), "元 (%.2f%%) "

      % ((cost_ub - cost_base) / cost_ub * 100),

      "距下界", "%.2f" % (cost_base - lb), "元 (%.2f%%) " % ((cost_base - lb) /
cost_base * 100))



robust = []

for sigma in (0.01, 0.03, 0.05, 0.10):

    costs, corrs, l1_g, l1_cd, max_g = [], [], [], [], []

    for _ in range(30):

        load_noisy = load_typ * (1 + rng.normal(0, sigma, N))

        pv_noisy = np.clip(pv_typ * (1 + rng.normal(0, sigma, N)), 0, None)

        rr = L.solve_day_lp(price_typ, load_noisy, pv_noisy,

                            soc_start=L.SOC_INIT, soc_end=L.SOC_INIT)

        if rr["status"] != 0:
```

```
            continue
        costs.append(rr["cost"])
        corrs.append(np.corrcoef(rr["g"], buy)[0, 1])
        l1_g.append(np.abs(rr["g"] - buy).sum() / buy.sum())
        l1_cd.append((np.abs(rr["c"] - charge).sum() + np.abs(rr["d"] -
discharge).sum())
                     / (charge.sum() + discharge.sum()))
        max_g.append(np.abs(rr["g"] - buy).max() / max(buy.max(), 1e-9))
    costs = np.array(costs)
    dev = (costs - cost_base) / cost_base * 100
    robust.append((sigma, costs.mean(), costs.std(), np.abs(dev).max(),
np.mean(corrs),
                   float(np.mean(l1_g)), float(np.mean(l1_cd)),
float(np.mean(max_g))))
    print("噪声", "%.0f%%" % (sigma * 100), "费用均值", "%.2f" % costs.mean(),
          "标准差", "%.2f" % costs.std(), "最大偏差", "%.3f%%" % np.abs(dev).max())
print("10% 噪声下策略重组 购电量 L1", "%.2f%%" % (robust[-1][5] * 100),
      "充放 L1", "%.2f%%" % (robust[-1][6] * 100),
      "单段最大", "%.2f%%" % (robust[-1][7] * 100), "相关系数", "%.4f" % robust[-
1][4])
```

```
def rule_based(a, b):
    """阈值式规则调度：低价满充、高价满放，返回全天费用。"""
    E = L.SOC_INIT
    cost = 0.0
    for t in range(N):
        net = load_typ[t] * L.DT - pv_typ[t] * L.DT
        c = d = 0.0
        if price_typ[t] <= a:
            room = (L.SOC_MAX - E) / L.ETA_C
            c = max(0.0, min(L.E_MAX, room, max(net + 0.0, net)))
            c = min(L.E_MAX, room)
        elif price_typ[t] >= b:
            avail = (E - L.SOC_MIN) * L.ETA_D
            d = max(0.0, min(L.E_MAX, avail))
        g = net + c - d
        if g < 0:
            g2 = 0.0
            surplus = -g
            s = min(surplus, pv_typ[t] * L.DT)
            c = max(0.0, c - (surplus - s))
```

```
        g = max(g, 0.0)
        E = E + L.ETA_C * c - d / L.ETA_D
        E = min(max(E, L.SOC_MIN), L.SOC_MAX)
        cost += price_typ[t] * g
    return cost



best = (np.inf, None, None)
# 充电阈值网格从 0.36 起
for a in np.arange(0.36, 0.95, 0.02):
    for b in np.arange(a + 0.05, 1.45, 0.02):
        cost_rule = rule_based(a, b)
        if cost_rule < best[0]:
            best = (cost_rule, a, b)
print("规则策略族最优 阈值", "%.2f" % best[1], "%.2f" % best[2],
      "费用", "%.2f" % best[0], "元", "LP 最优", "%.2f" % cost_base, "元")
print("LP 相对最优规则策略的改进", "%.2f" % (best[0] - cost_base), "元 (%.2f%%) "
      % ((best[0] - cost_base) / best[0] * 100))


price_2d = np.tile(price_typ, (L.N_DAY, 1))
```

```
load_2d = data["actual"]["load_kw"]

pv_2d = data["actual"]["pv_kw"]

dates = pd.Series(pd.to_datetime(data["actual"]["dates"]))

q2 = np.load(os.path.join(L.RES, "问2_结果.npz"), allow_pickle=True)

# g_all 覆盖全年，季度按日期切分

g_all = q2["g_all"]

for qname, (m0, m1) in [("Q1(1-3月)", (1, 3)), ("Q2(4-6月)", (4, 6)),

                        ("Q3(7-9月)", (7, 9)), ("Q4(10-12月)", (10, 12))]:

    mask = (dates.dt.month.values >= m0) & (dates.dt.month.values <= m1)

    buy_q = g_all[mask]

    load_q = load_2d[mask]

    pv_q = pv_2d[mask]

    unit_cost = float((price_typ * buy_q).sum() / buy_q.sum())

    load_avg = float(load_q.mean())

    pv_avg = float(pv_q.mean())

    print(qname, "天数", mask.sum(), "单位购电成本", "%.4f" % unit_cost,

          "平均负载", "%.1f" % load_avg, "平均光伏", "%.1f" % pv_avg)

units = []

for m0, m1 in [(1, 3), (4, 6), (7, 9), (10, 12)]:

    mask = (dates.dt.month.values >= m0) & (dates.dt.month.values <= m1)
```

```
    buy_q = g_all[mask]
    units.append(float((price_typ * buy_q).sum() / buy_q.sum()))
print("四季度单位购电成本 均值", "%.4f" % np.mean(units), "标准差", "%.4f" %
np.std(units),
      "变异系数", "%.3f" % (np.std(units) / np.mean(units)))


fig, axes = plt.subplots(1, 3, figsize=(15.5, 4.4))
sigmas = [r[0] * 100 for r in robust]
means = [r[1] for r in robust]
stds = [r[2] for r in robust]
axes[0].errorbar(sigmas, np.array(means)/1000, yerr=np.array(stds)/1000,
marker="o",
                 color="#2E6F9E", linewidth=2, capsize=5, markersize=7)
axes[0].axhline(cost_base/1000, color="#D64545", linestyle="--", linewidth=1.4,
label="基准费用")
axes[0].set_xlabel("输入噪声标准差 sigma (%)")
axes[0].set_ylabel("单日购电费（千元)")
axes[0].set_title("（a）费用对输入噪声的鲁棒性", fontsize=11.5)
axes[0].legend(fontsize=9)
axes[0].grid(alpha=0.25, linestyle=":")
```

```
axes[1].bar(["highs-ds\n(对偶单纯形)", "highs-ipm\n(内点法)", "lp_core\n(默认)"],
            [r_ds.fun, r_ipm.fun, cost_base], color=["#3F8F5B", "#C77D2B",
"#2E6F9E"], width=0.55)
axes[1].set_ylabel("目标值（元）")
axes[1].set_ylim(min(r_ds.fun, r_ipm.fun, cost_base) * 0.999,
                 max(r_ds.fun, r_ipm.fun, cost_base) * 1.001)
axes[1].set_title("（b）两条独立算法路径的一致性\n(差异 %.2e 元)" % gap,
fontsize=11.5)
axes[1].grid(axis="y", alpha=0.25, linestyle=":")

cats = ["忽略电池\n物理限制\n(下界)", "LP 最优解", "无储能\n朴素策略\n(上界)"]
vals = [lb, cost_base, cost_ub]
axes[2].bar(cats, np.array(vals)/1000, color=["#B0B7BF", "#2E6F9E", "#D64545"],
width=0.55)
axes[2].set_ylabel("单日购电费（千元）")
for i, v in enumerate(vals):
    axes[2].annotate("%.1f" % (v/1000), (i, v/1000), ha="center", va="bottom",
fontsize=9)
axes[2].set_title("（c）边界检验：最优值落在上下界之间", fontsize=11.5)
axes[2].grid(axis="y", alpha=0.25, linestyle=":")
fig.suptitle("模型有效性检验", fontsize=13)
fig.tight_layout(rect=[0, 0, 1, 0.93])
```

```
fig.savefig(os.path.join(L.IMG, "图23_模型有效性检验.png"), bbox_inches="tight")
plt.close(fig)


fig, axes = plt.subplots(1, 2, figsize=(13, 4.4))
cor = [r[4] for r in robust]
axes[0].plot(sigmas, cor, marker="s", color="#7B4FA0", linewidth=2.2, markersize=8)
axes[0].set_ylim(0.9, 1.005)
axes[0].set_xlabel("输入噪声标准差 sigma (%)")
axes[0].set_ylabel("最优策略与基准的相关系数")
axes[0].set_title("（a）最优策略的稳定性\n(噪声 10%% 时相关系数仍达 %.3f)" % cor[-1], fontsize=11.5)
axes[0].grid(alpha=0.25, linestyle=":")
for x, y in zip(sigmas, cor):
    axes[0].annotate("%.4f" % y, (x, y), ha="center", va="bottom", fontsize=9)


axes[1].bar(["LP 最优解", "最优阈值规则策略"],
            [cost_base/1000, best[0]/1000], color=["#2E6F9E", "#C77D2B"], width=0.5)
for i, v in enumerate([cost_base, best[0]]):
    axes[1].annotate("%.2f 千元" % (v/1000), (i, v/1000), ha="center", va="bottom", fontsize=10)
```

```
axes[1].set_ylabel("单日购电费（千元）")

axes[1].set_title("（b）与朴素阈值策略的对比\n(LP 优 %.2f%%)"

                  % ((best[0]-cost_base)/best[0]*100), fontsize=11.5)

axes[1].grid(axis="y", alpha=0.25, linestyle=":")

fig.suptitle("鲁棒性与策略有效性检验", fontsize=12.5)

fig.tight_layout(rect=[0, 0, 1, 0.93])

fig.savefig(os.path.join(L.IMG, "图24_鲁棒性与策略有效性.png"),
bbox_inches="tight")

plt.close(fig)

print("已生成 图23_模型有效性检验.png / 图24_鲁棒性与策略有效性.png")



np.savez(os.path.join(L.RES, "模型检验_结果.npz"),

         rob=np.array(robust), lb=lb, ub=cost_ub, base=cost_base,

         rule_best=best[0], rule_a=best[1], rule_b=best[2],

         gap=gap, ds=r_ds.fun, ipm=r_ipm.fun)

print("模型检验完成。")
```